

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
ПО ПРОГРАММЕ БАКАЛАВРИАТА**

09.03.04 Программная инженерия

(код, наименование ОПОП ВО: направление подготовки, направленность (профиль))

«Разработка программно-информационных систем»

Бизнес-проект "Онлайн-площадка для сдачи вещей в аренду"

Разработка клиентской части

(название темы)

Дипломный проект

(вид ВКР: дипломная работа или дипломный проект)

Автор ВКР

(подпись, дата)

Н. А. Баранов

(инициалы, фамилия)

Группа ПО-216

Руководитель ВКР

(подпись, дата)

Е. А. Петрик

(инициалы, фамилия)

Нормоконтроль

(подпись, дата)

А. А. Чаплыгин

(инициалы, фамилия)

ВКР допущена к защите:

Заведующий кафедрой

(подпись, дата)

А. В. Малышев

(инициалы, фамилия)

Курск 2026 г.

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

УТВЕРЖДАЮ:

Заведующий кафедрой

(подпись, инициалы, фамилия)

«_____» _____ 20____ г.

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ ПО ПРОГРАММЕ БАКАЛАВРИАТА

Студента Баранова Н.А., шифр 22-06-0230, группа ПО-216

1. Тема «Бизнес-проект "Онлайн-площадка для сдачи вещей в аренду" Разработка клиентской части» утверждена приказом ректора ЮЗГУ от «03» апреля 2026 г. № 1584-с.

2. Срок предоставления работы к защите «09» июня 2026 г.

3. Исходные данные для создания программной системы:

3.1. Перечень решаемых задач:

1) провести анализ предметной области аренды вещей и существующих онлайн-площадок с точки зрения требований к пользовательскому интерфейсу;

2) разработать концептуальную модель клиентской части онлайн-площадки, определить основные экраны, компоненты и сценарии взаимодействия пользователя с системой;

3) спроектировать пользовательский интерфейс онлайн-площадки, обеспечивающий взаимодействие с серверной частью посредством REST API;

4) разработать и протестировать клиентскую часть онлайн-площадки, реализующую отображение данных, навигацию, бизнес-логику на стороне клиента и обработку пользовательских действий.

3.2. Входные данные и требуемые результаты для программы:

1) Входными данными для программной системы являются: действия пользователя в интерфейсе, регистрационные и профильные данные, вводимые в формах, параметры поиска и фильтрации объявлений, данные для создания и редактирования объявлений об аренде, параметры бронирования, текстовые сообщения в чатах, а также ответы REST API серверной части.

2) Выходными данными для программной системы являются: отрисованные страницы и компоненты интерфейса, формы регистрации и авторизации с валидацией, список объявлений с поиском и фильтрацией, карточки объявлений с детальной информацией и ценой с учётом комиссии, форма бронирования с расчётом итоговой стоимости, интерфейс чата, личный кабинет пользователя, а также уведомления об ошибках и результатах операций.

4. Содержание работы (по разделам):

4.1. Введение.

4.1. Анализ предметной области.

4.2. Техническое задание: основание для разработки, назначение разработки, требования к клиентской части программной системы, требования к оформлению документации.

4.3. Технический проект: общие сведения о клиентской части программной системы, проектирование компонентной структуры интерфейса, проектирование взаимодействия с REST API серверной части, проектирование пользовательского интерфейса программной системы.

4.4. Рабочий проект: спецификация компонентов и модулей клиентской части программной системы, тестирование пользовательского интерфейса программной системы, сборка компонентов программной системы.

4.5. Заключение.

4.6. Список использованных источников.

5. Перечень графического материала:

- Лист 1. Сведения о ВКРБ.
- Лист 2. Цель и задачи разработки.
- Лист 3. Сравнение конкурентов.
- Лист 4. Компоненты онлайн-площадки для аренды вещей.
- Лист 5. Диаграмма прецедентов.
- Лист 6. Макет страницы входа в профиль.
- Лист 7. Макет страницы чата с пользователем.
- Лист 8. Макет страницы просмотра объявлений.
- Лист 9. Макет страницы информации об объявлении.
- Лист 10. Заключение.

Руководитель ВКР

(подпись, дата)

Е. А. Петрик

(инициалы, фамилия)

Задание принял к исполнению

(подпись, дата)

Н. А. Баранов

(инициалы, фамилия)

РЕФЕРАТ

Объем работы равен 153 страницам. Работа содержит 19 иллюстраций, 4 таблицы, 50 библиографических источников и 10 листов графического материала. Количество приложений – 2. Графический материал представлен в приложении А. Фрагменты исходного кода представлены в приложении Б.

Перечень ключевых слов: онлайн-площадка, аренда вещей, клиентская часть, фронтенд, JavaScript, HTML, CSS, REST API, пользовательский интерфейс, веб-приложение, адаптивная вёрстка, аутентификация, авторизация, бронирование, чат, отзывы, договор аренды, компонентная структура, валидация форм, подтверждение email.

Объектом разработки является клиентская часть онлайн-площадки для аренды вещей, предназначенная для отображения данных, обеспечения взаимодействия пользователя с системой и обмена данными с серверной частью посредством REST API.

Целью выпускной квалификационной работы является разработка и тестирование клиентской части онлайн-площадки для аренды вещей, обеспечивающей удобный пользовательский интерфейс, корректную обработку пользовательских действий и надёжное взаимодействие с серверной частью приложения.

В процессе разработки клиентской части онлайн-площадки для аренды вещей были проведены анализ предметной области и существующих решений, спроектированы компонентная структура интерфейса и сценарии взаимодействия пользователя с системой, реализованы формы регистрации с подтверждением email через код и авторизации, разработаны модули каталога объявлений с фильтрацией, карточки товара с отображением цены с учётом комиссии платформы, бронирования с автоматическим расчётом стоимости, встроенного чата, договоров аренды и системы отзывов, обеспечено взаимодействие с REST API серверной части, проведена интеграция с компонентами, разработанными другими участниками команды.

ABSTRACT

The volume of work is 153 pages. The work contains 19 illustrations, 4 tables, 50 bibliographic sources and 10 sheets of graphic material. The number of applications is 2. The graphic material is presented in annex A. The layout of the site, including the connection of components, is presented in annex B.

The list of keywords: online marketplace, rental of things, client side, frontend, JavaScript, HTML, CSS, REST API, user interface, web application, responsive layout, authentication, authorization, booking, chat, reviews, rental agreement, component structure, form validation, email confirmation.

The object of development is the client side of an online marketplace for renting things, designed to display data, ensure user interaction with the system, and exchange data with the server side via REST API.

The purpose of the final qualifying work is to develop and test the frontend of an online marketplace for renting things, providing a convenient user interface, correct handling of user actions, and reliable interaction with the backend of the application.

During the development of the client side of the online rental platform, an analysis of the subject area and existing solutions was carried out, the component structure of the interface and user interaction scenarios were designed, registration forms with email confirmation via code and authorization were implemented, modules for the item catalog with filtering, item cards displaying prices including platform commission, booking with automatic cost calculation, built-in chat, rental agreements and a review system were developed, interaction with the REST API of the server side was ensured, and integration with components developed by other team members was carried out.

The developed website was successfully implemented in the company.

СОДЕРЖАНИЕ

РЕЗЮМЕ СТАРТАП-ПРОЕКТА	13
ВВЕДЕНИЕ	16
1 Анализ предметной области	19
1.1 Понятие и принципы онлайн аренды вещей	19
1.2 Классификация онлайн площадок для аренды	20
1.2.1 По типу предлагаемых товаров	20
1.2.2 По модели взаимодействия с пользователями	21
1.2.3 По типу арендодателей	21
1.3 Основные бизнес-процессы системы аренды	22
1.3.1 Регистрация и верификация пользователей	22
1.3.2 Публикация и поиск предложений	22
1.3.3 Бронирование и совершение сделки	22
1.3.4 Бронирование и совершение сделки	22
1.4 Компоненты программно-информационной системы	23
1.4.1 Модуль управления пользователями	23
1.4.2 Модуль каталога и поиска	23
1.4.3 Модуль бронирования и календаря	23
1.4.4 Модуль генерации договоров	23
1.4.5 Модуль коммуникации	24
1.4.6 Модуль аналитики и отчётности	24
1.5 Правовое регулирование сферы онлайн-аренды	24
1.6 Анализ существующих решений и аналогов	25
2 Техническое задание	28
2.1 Основание для разработки	28
2.2 Цель и назначение разработки	28
2.3 Требования к веб-приложению	28
2.3.1 Функциональные требования к веб-приложению	28
2.3.2 Требования пользователя к интерфейсу	30
2.4 Моделирование вариантов использования	40

2.4.1	Регистрация	42
2.4.2	Авторизация	43
2.4.3	Изменение пароля пользователя	43
2.4.4	Просмотр профиля пользователя	44
2.4.5	Редактирование профиля пользователя	44
2.4.6	Просмотр каталога вещей	45
2.4.7	Фильтрация объявлений по категории и дате	46
2.4.8	Создание собственных объявлений	46
2.4.9	Редактирование собственных объявлений	47
2.4.10	Создание бронирования вещи с автоматическим расчетом стоимости	47
2.4.11	Удаление собственных объявлений	48
2.4.12	Загрузка изображений в объявление	49
2.4.13	Удаление изображений в объявлении	49
2.4.14	Просмотр карточки вещи, ее рейтинга, отзывов и занятых периодов аренды	50
2.4.15	Управление статусами бронирования: подтверждение, отмена	50
2.4.16	Просмотр входящих и исходящих бронирований пользователя	51
2.4.17	Оплата бронирования через демонстрационный механизм СБП	52
2.4.18	Формирование и подписание договора аренды участниками сделки	52
2.4.19	Создание отзывов по завершенным или оплаченным бронированиям	53
2.4.20	Чат между пользователями с возможностью отправить изображение	54
2.4.21	Помощь на часто задаваемые вопросы	54
2.5	Требования к оформлению документации	55
3	Технический проект	56
3.1	Архитектура онлайн-площадки для сдачи вещей в аренду	56
3.2	Обоснование выбора технологий проектирования	57
3.3	Архитектура клиентской части	57

3.3.1	Файл index.html	58
3.3.2	Файл style.css	59
3.3.3	Файл config.js	59
3.3.4	Файл auth.js	59
3.3.5	Файл catalog.js	59
3.3.6	Файл bookings.js	59
3.3.7	Файл chats.js	60
3.3.8	Файл profile.js	60
3.3.9	Файл notifications.js	60
3.3.10	Файл contracts.js	60
3.3.11	Файл main.js	60
3.3.12	Связь с API	61
3.4	Реализация ключевых функциональных модулей	61
3.4.1	Модуль каталога вещей	61
3.4.2	Модуль карточки вещи	61
3.4.3	Модуль аренды и бронирования	62
3.4.4	Модуль авторизации и регистрации	62
3.4.5	Модуль личного кабинета	63
3.4.6	Модуль управления объявлениями	63
3.4.7	Модуль чатов	64
3.4.8	Модуль уведомлений	64
3.4.9	Модуль договоров и оплаты	65
3.5	Управление состоянием на клиенте	65
3.6	Пользовательский интерфейс и UX	66
3.7	Взаимодействие с backend API	66
3.8	Требования к запуску клиентской части	68
4	Рабочий проект	69
4.1	Спецификация функций и классов клиентской части	69
4.2	Реализация ключевых фрагментов клиентской части	73
4.2.1	Централизованный модуль взаимодействия с API	73
4.2.2	Модуль бронирования	74

4.2.3	Модуль договоров и демонстрационного подписания ЭЦП	75
4.2.4	Модуль чатов	76
4.2.5	Модуль уведомлений	77
4.2.6	Модуль валидации форм на стороне клиента	78
4.3	Тестирование клиентской части	78
4.3.1	Модульное тестирование	78
4.3.2	Системное тестирование	80
4.4	Сборка и развёртывание клиентской части	82
	ЗАКЛЮЧЕНИЕ	84
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	85
	ПРИЛОЖЕНИЕ А Представление графического материала	92
	ПРИЛОЖЕНИЕ Б Фрагменты исходного кода программы	103
	На отдельных листах (CD-RW в прикрепленном конверте)	153
	Сведения о ВКРБ (Графический материал / Сведения о ВКРБ.png)	Лист 1
	Цель и задачи разработки (Графический материал / Цель и задачи разработки.png)	Лист 2
	Сравнение конкурентов (Графический материал / Сравнение конкурентов.png)	Лист 3
	Компоненты онлайн-площадки для аренды вещей (Графический материал / Компоненты онлайн-площадки для аренды вещей.png)	Лист 4
	Диаграмма прецедентов (Графический материал / Диаграмма прецедентов.png)	Лист 5
	Макет страницы входа в профиль (Графический материал / Макет страницы входа в профиль.png)	Лист 6
	Макет страницы чата с пользователем (Графический материал / Макет страницы чата с пользователем.png)	Лист 7
	Макет страницы просмотра объявлений (Графический материал / Макет страницы просмотра объявлений.png)	Лист 8
	Макет страницы информации об объявлении (Графический материал / Макет страницы информации об объявлении.png)	Лист 9

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

БД – база данных.

ИС – информационная система.

ИТ – информационные технологии.

ПО – программное обеспечение.

РП – рабочий проект.

ТЗ – техническое задание.

ТП – технический проект.

ПО – программное обеспечение.

СУБД – система управления базами данных.

API (Application Programming Interface) – программный интерфейс приложения, набор правил и инструментов для взаимодействия программных компонентов.

HTTP (HyperText Transfer Protocol) – протокол передачи гипертекста.

HTTPS (HyperText Transfer Protocol Secure) – защищенный протокол передачи гипертекста.

SQL (Structured Query Language) – язык структурированных запросов к базам данных.

REST (Representational State Transfer) — архитектурный стиль взаимодействия компонентов распределённого приложения через HTTP.

JWT (JSON Web Token) — формат токена, используемый для аутентификации и авторизации пользователей.

JSON (JavaScript Object Notation) — текстовый формат обмена данными, применяемый для передачи информации между клиентской и серверной частями системы.

CRUD (Create, Read, Update, Delete) — основные операции по работе с данными: создание, чтение, обновление и удаление.

PDF (Portable Document Format) — формат электронных документов, используемый для генерации договоров аренды.

РЕЗЮМЕ СТАРТАП-ПРОЕКТА

Название: «Онлайн-площадка для сдачи вещей в аренду». Проект представляет собой веб-приложение, предназначенное для автоматизации процессов аренды различных товаров между частными лицами или компаниями. Разработанная система должна цифровизировать такие задачи как просмотр каталога доступных вещей, бронирование товаров на определенный срок, расчет стоимости аренды, обработка заявок, а также взаимодействие между арендодателями и арендаторами. В ходе работы были проанализированы современные аналоги и сервисы совместного потребления (шеринг-платформы), и выявлены их отрицательные стороны:

1. Интерфейс. Многие существующие платформы перегружены элементами, рекламой и второстепенными функциями, что затрудняет быстрый поиск нужного товара и оформление аренды для новых пользователей.

2. Высокие комиссии и скрытые платежи за аренду товара или услуги. Ряд популярных сервисов взимают значительную комиссию как с арендодателей, так и с арендаторов, либо требуют оформления платной подписки для доступа к базовым возможностям, что снижает экономическую выгоду от использования платформы.

3. Отсутствие гибкости в настройке условий аренды вещей или услуг. Во многих системах отсутствует возможность гибко настроить условия аренды для конкретного товара или услуги: посуточная/почасовая аренда, размер залога, стоимость доставки, что ограничивает владельцев вещей.

4. Неудобная система коммуникации между арендатором и арендодателем. Встроенные чаты или формы связи часто работают с задержками, либо требуют перехода в сторонние мессенджеры, что создает неудобства и риски для безопасности сделки.

Создаваемое приложение для аренды вещей позволит избежать вышеперечисленные недостатки без негативного влияния на процесс работы и скорость выполнения поставленных задач, предлагая сбалансированное решение с понятным интерфейсом и прозрачными условиями.

Актуальность проекта связана с ростом популярности модели совместного потребления (шеринг-экономики) в России и мире. Большое количество людей предпочитают арендовать вещи для разового или краткосрочного использования вместо их покупки, однако существующие площадки не всегда отвечают потребностям пользователей в простоте, надежности и доступности.

Целью является разработка полнофункционального веб-приложения для аренды вещей, обеспечивающего удобный поиск, бронирование и сдачу товаров в аренду.

Отличительные особенности разрабатываемого приложения по сравнению с уже существующими решениями:

1. Интерфейс с продуманной структурой каталога и фильтрацией, позволяющей быстро подобрать вещь по категории, цене, сроку аренды.

2. Прозрачная система формирования стоимости с автоматическим расчетом итоговой цены в зависимости от количества дней аренды, с возможностью учета скидок и залога.

3. Реализация личного кабинета для арендодателей и арендаторов с разделением возможностей: управление объявлениями, просмотр истории бронирований, отслеживание статусов заказов.

4. Интеграция с серверным API для выполнения основных операций (получение списка товаров, отправка заявок на аренду, добавление в корзину) без перезагрузки страницы, что обеспечивает высокую скорость работы и комфорт пользователя.

5. Адаптивная верстка, обеспечивающая корректное отображение и удобство использования сайта на всех типах устройств: персональных компьютерах, планшетах и смартфонах.

В разработанном приложении для аренды вещей реализованы такие функции как: просмотр каталога товаров с фильтрацией, регистрация и авторизация пользователей, добавление новых объявлений о сдаче вещей в аренду, оформление заказа с выбором дат аренды, корзина аренды, личный каби-

нет с историей операций, а также административная панель для модерации объявлений и управления пользователями.

Идея создания проекта появилась в сентябре 2025 года, активная разработка была начата в феврале 2026 года. С апреля 2026 года производилось тестирование системы путем моделирования различных пользовательских сценариев: размещение объявлений, поиск вещей, оформление бронирований. Проект планируется закончить к маю 2026 года. Также будет осуществлена регистрация программного продукта для защиты авторских прав и обеспечения исключительности прав.

Бизнес-цели и перспективы дальнейшего развития проекта:

1. Достижение 2 500 успешных завершённых сделок (аренд) в течение первых 12 месяцев работы площадки.
2. Обеспечение доли повторных аренд (пользователь арендует второй раз и более) на уровне не менее 30% от общего числа активных клиентов к концу 1-го года.
3. Снижение доли спорных сделок и возвратов залогов до менее 5% от общего числа аренд путём внедрения системы верификации и страховки.
4. Достижение среднего чека одной аренды на уровне не менее 1 500 рублей.
5. Обеспечение времени отклика арендодателя на запрос арендатора не более 2 часов в 90% случаев (для удержания пользователей).

ВВЕДЕНИЕ

В последнее десятилетие в мировой экономике наблюдается стремительный рост модели потребления, основанной на совместном использовании товаров, получившей название «шеринг-экономика» (от англ. sharing — делить). Идея временного пользования вещью вместо её приобретения в собственность становится всё более популярной среди населения. Аренда электроинструментов, бытовой техники, спортивного инвентаря, одежды и даже транспортных средств позволяет потребителям существенно экономить ресурсы, а владельцам вещей — получать прибыль от простаивающего имущества. Данная модель потребления не только выгодна экономически, но и отвечает современным принципам устойчивого развития, снижая нагрузку на экологию за счёт уменьшения объёмов производства и утилизации товаров.

Развитие информационных технологий и повсеместное проникновение сети Интернет стали ключевыми факторами, обеспечившими рост рынка арендных услуг. Если раньше аренда ограничивалась специализированными пунктами проката, расположенными в конкретных районах города, то сегодня веб-сайты и мобильные приложения позволяют создать единое цифровое пространство, где арендодатели и арендаторы могут быстро находить друг друга независимо от времени и местоположения.

Сайт для аренды вещей выполняет функции не только витрины товаров, но и полноценного посреднического сервиса. Он автоматизирует процессы бронирования, расчёта стоимости аренды на срок, обработки платежей и коммуникации между сторонами. Для современного бизнеса в сфере проката наличие качественного веб-сайта является необходимым условием конкурентоспособности. От того, насколько удобен и функционален интерфейс, зависит, сможет ли компания привлечь и удержать клиентов в условиях высокой конкуренции с крупными маркетплейсами и специализированными агрегаторами.

Клиентская часть сайта (фронтенд) играет в этом процессе ключевую роль. Именно через интерфейс пользователь взаимодействует с сервисом:

просматривает каталог, изучает характеристики вещей, выбирает даты аренды и совершает заказ. От качества вёрстки, скорости загрузки, адаптивности под мобильные устройства и интуитивной понятности кнопок и форм напрямую зависит, превратится ли посетитель в реального клиента. Современные веб-технологии позволяют создавать интерфейсы, которые работают с серверной частью сайта в фоновом режиме (асинхронно), благодаря чему оформление аренды происходит без задержек и перезагрузки страниц, максимально приближаясь по удобству к настольным приложениям.

Цель настоящей работы – разработка клиентской части веб-сайта для сервиса аренды вещей, включая создание пользовательского интерфейса и реализацию программного взаимодействия с серверным API. Для достижения поставленной цели необходимо решить *следующие задачи*:

- провести анализ предметной области сервисов аренды и существующих решений;
- разработать структуру и дизайн пользовательского интерфейса;
- выполнить вёрстку страниц сайта с использованием современных технологий HTML и CSS;
- реализовать на языке JavaScript интерактивные элементы и функции обращения к API для выполнения операций (получение списка вещей, оформление заказа, фильтрация);
- провести тестирование разработанного интерфейса и его интеграцию с серверной частью.

Структура и объем работы. Отчет состоит из введения, 4 разделов основной части, заключения, списка использованных источников, 2 приложений. Текст выпускной квалификационной работы равен 153 страницам.

Во введении сформулирована цель работы, поставлены задачи разработки, описана структура работы, приведено краткое содержание каждого из разделов.

В первом разделе рассматриваются современное состояние рынка арендных онлайн-сервисов, основные требования к подобным сайтам и су-

ществующие аналоги. Проводится анализ предметной области и формируется техническое задание на разработку клиентской части.

Во втором разделе описываются архитектура разрабатываемого интерфейса, структура страниц, проектируются основные пользовательские сценарии (поиск вещи, оформление аренды). Разрабатываются прототипы интерфейса и схемы взаимодействия с API.

В третьем разделе подробно описывается процесс реализации клиентской части: создание HTML-разметки, написание стилей CSS, разработка логики на JavaScript, реализация вызовов к API для отправки и получения данных. Приводятся примеры кода.

В четвертом разделе описываются результаты тестирования разработанного интерфейса в различных браузерах и на различных устройствах, проверяется корректность работы с API, оценивается производительность.

В заключении излагаются основные результаты работы, полученные в ходе разработки, и делаются выводы о соответствии готового продукта поставленным задачам.

В приложении А представлены экранные формы (скриншоты) разработанного сайта. В приложении Б представлены фрагменты исходного кода (HTML, CSS, JavaScript).

1 Анализ предметной области

1.1 Понятие и принципы онлайн аренды вещей

Электронная коммерция является частью цифровой экономики и охватывает все финансовые и торговые операции, совершаемые с использованием компьютерных сетей, а также связанные с ними бизнес-процессы. Онлайн-аренда вещей представляет собой отдельное направление электронной коммерции, в рамках которого через интернет-платформы на платной основе временно передаётся право пользования материальными объектами.

Виды электронной коммерции:

- P2P (peer-to-peer) – между пользователями;
- B2B (business-to-business) – между предприятиями;
- B2C (business-to-consumer) – между предприятием и потребителем;
- C2C (consumer-to-consumer) – между потребителями;
- B2G (business-to-government) – между предприятием и государством;

Онлайн-бронирование — это бронирование через интернет в интерактивном режиме. Термин используется применительно к бронированию гостиничных номеров, билетов, мест в ресторанах и театрах и т.д.

Данный формат взаимодействия даёт существенные преимущества как арендодателям, так и арендаторам:

1. Экономическая выгода: арендаторы экономят на покупке редко используемых или дорогих товаров, а владельцы получают доход от простаивающего имущества.

2. Удобство и доступность: поиск, бронирование и оплата возможны круглосуточно из любого места с выходом в интернет, что сокращает время на поиск и сравнение предложений.

3. Прозрачность взаимодействия: платформы позволяют формировать рейтинги, оставлять отзывы и проверять репутацию участников, что повышает доверие при сделках.

4. Удобство и доступность: поиск, бронирование и оплата осуществляются круглосуточно из любой точки с доступом к интернету, что экономит время на поиск и сравнение предложений.

Вместе с тем существуют характерные риски и ограничения:

1. Контроль качества и соответствия: отсутствует возможность физического осмотра вещи до бронирования, возможны расхождения между описанием, фотографиями и реальным состоянием.

2. Логистика и сохранность: возникают дополнительные расходы и риски при передаче, повреждении или утрате имущества во время доставки или использования.

3. Логистика и сохранность: возникают дополнительные расходы и риски при передаче, повреждении или утрате имущества в процессе доставки или эксплуатации.

4. Юридические и финансовые споры: затруднено возмещение ущерба, возврат залогов и разрешение конфликтов при отсутствии встроенных механизмов страхования.

5. Кибербезопасность: платформа подвержена рискам утечки персональных данных, мошеннических действий и нарушения прав пользователей.

1.2 Классификация онлайн площадок для аренды

Рынок онлайн-площадок для аренды вещей неоднороден и включает сервисы, которые можно классифицировать по нескольким ключевым признакам: типу предлагаемых товаров, модели взаимодействия с пользователями, а также типу арендодателей. Понимание этой классификации необходимо для точного позиционирования проектируемой площадки и определения её целевой аудитории.

1.2.1 По типу предлагаемых товаров

По специализации онлайн-площадки для аренды делятся на универсальные и нишевые. Универсальные площадки предлагают аренду широкого

спектра товаров: от инструментов и электроники до одежды и спортивного инвентаря. Их главное преимущество — широкая аудитория и большое количество предложений, однако им сложнее создавать специализированные инструменты для каждой отдельной категории. Узкоспециализированные площадки, напротив, сосредоточены на одной или нескольких смежных категориях товаров. Такие сервисы выигрывают за счёт глубокого понимания потребностей своей аудитории, предоставления специализированного функционала и формирования более сплочённого сообщества пользователей с высоким уровнем экспертизы в конкретной области.

1.2.2 По модели взаимодействия с пользователями

По способу монетизации и степени вовлечённости в транзакцию выделяют три модели онлайн-площадок для аренды. Первая — модель классифайда, когда платформа выступает только доской объявлений, а арендодатель и арендатор самостоятельно договариваются об оплате и условиях сделки. Вторая — модель с полным сопровождением сделки, при которой платформа берёт на себя обработку платежей, страхование и урегулирование споров, взимая комиссию с каждой транзакции. Именно на эту модель ориентируется разрабатываемая система. Третья — подписочная модель, предполагающая ежемесячную плату за неограниченный доступ к аренде вещей из определённого набора, что особенно востребовано в сегменте детских товаров и одежды.

1.2.3 По типу арендодателей

По типу участников сделки выделяют две основные модели онлайн-площадок для аренды. Первая — C2C (Consumer-to-Consumer), где арендодателями являются обычные частные лица, сдающие свои личные вещи во временное пользование другим людям. Вторая — B2C (Business-to-Consumer), где в роли арендодателей выступают профессиональные компании проката, использующие платформу как дополнительный канал сбыта своих услуг.

1.3 Основные бизнес-процессы системы аренды

1.3.1 Регистрация и верификация пользователей

Для минимизации рисков, связанных с мошенничеством или порчей имущества, платформа должна иметь многоуровневую систему регистрации. Пользователь может выступать в двух ролях: арендодатель и арендатор. Для получения доступа к сервису пользователь проходит обязательную верификацию: подтверждение email, а в некоторых случаях — предоставление данных банковской карты и подтверждение личности через государственные сервисы (например, Госуслуги).

1.3.2 Публикация и поиск предложений

Арендодатель создаёт карточку товара (объявление), в которой указывает название вещи, её категорию, подробное описание, фотографии, стоимость аренды и доступные даты. Платформа, в свою очередь, предоставляет систему категорий, фильтров и поиска.

1.3.3 Бронирование и совершение сделки

Ключевой этап, который платформа автоматизирует с помощью встроенного календаря бронирования. Это позволяет избежать задвоения заказов. Арендатор выбирает период аренды, и система автоматически рассчитывает итоговую стоимость. Оплата всегда проходит через платформу, которая временно замораживает средства до подтверждения обеими сторонами. Это гарантирует, что арендодатель получит деньги, а арендатор — вещь в надлежащем состоянии.

1.3.4 Бронирование и совершение сделки

В C2C-сервисах важнейшим компонентом является система рейтингов и отзывов. После завершения сделки арендатор оценивает состояние вещи и оперативность владельца. Рейтинг напрямую влияет на видимость объявлений в поиске и готовность пользователей заключать сделки.

1.4 Компоненты программно-информационной системы

Программно-информационная система онлайн-площадки аренды представляет собой комплекс взаимодействующих программных модулей, обеспечивающих хранение, обработку и предоставление информации всем участникам платформы. Рассмотрим основные компоненты.

1.4.1 Модуль управления пользователями

Обеспечивает регистрацию, аутентификацию и авторизацию пользователей. Хранит профили с историей транзакций, рейтингами, отзывами и документами верификации.

1.4.2 Модуль каталога и поиска

Обеспечивает создание, редактирование и публикацию объявлений. Реализует полнотекстовый поиск с фильтрацией по категориям и дате доступности.

1.4.3 Модуль бронирования и календаря

Управляет доступностью товаров во времени, предотвращает двойное бронирование. Ведёт календарь занятости для каждого объявления. Отправляет уведомления арендодателю о поступивших запросах на бронирование и арендатору — о подтверждении или отклонении.

1.4.4 Модуль генерации договоров

Встроенный инструмент для создания договоров аренды между участниками сделки на основе их данных из учётных записей на площадке и данных из карточек объявлений. Данный модуль реализует формирование, а также последующее подписание и расторжение договора аренды двумя сторонами с использованием средств ЭЦП на определённый срок.

1.4.5 Модуль коммуникации

Встроенный мессенджер обеспечивает безопасное общение между арендатором и арендодателем без раскрытия личных контактных данных. Система push-уведомлений и email-рассылок информирует пользователей о ключевых событиях: бронировании, оплате, возврате, заявках на аренду и прочих уведомлениях в чатах.

1.4.6 Модуль аналитики и отчётности

Предоставляет арендодателям дашборд с данными о доходах, популярности объявлений. Доступна сводная статистика: оборот, количество сделок. Арендаторам предоставляется дашборд с количеством объявлений и историей их аренды.

1.5 Правовое регулирование сферы онлайн-аренды

Деятельность онлайн-площадки по сдаче вещей в аренду регулируется рядом нормативно-правовых актов Российской Федерации. Основой является Гражданский кодекс РФ, а именно глава 34 «Аренда», которая определяет общие положения о договоре аренды, права и обязанности арендодателя и арендатора, ответственность за недостатки сданного в аренду имущества.[24]

Поскольку арендатор использует вещь для личных, семейных или домашних нужд, на отношения между участниками сделки распространяются положения Закона РФ «О защите прав потребителей» от 07.02.1992 № 2300-1. Согласно данному закону, арендатор имеет право на получение полной и достоверной информации об услуге, на отказ от исполнения договора в любое время при условии оплаты арендодателю фактически понесённых расходов, а также на возмещение убытков, причинённых вследствие недостатков оказанной услуги. Арендодатель, в свою очередь, обязан предоставить вещь в состоянии, соответствующем условиям договора аренды и назначению имущества, и несёт ответственность за недостатки, обнаруженные в течение срока аренды.[23]

Платформа, являясь оператором персональных данных, обязана соблюдать требования Федерального закона № 152-ФЗ «О персональных данных» от 27.07.2006 г. Вся собираемая информация о пользователях (паспортные данные, контактная информация, платёжные реквизиты) должна обрабатываться на законном основании, а также надёжно храниться и защищаться от несанкционированного доступа.[26] Для реализации функций безопасной сделки платформа интегрируется с платёжными сервисами, сертифицированными по стандарту PCI DSS или ISO-20022, что обеспечивает безопасность финансовых транзакций.

Важным аспектом правового регулирования является налогообложение доходов арендодателей. Арендодатели — физические лица, не зарегистрированные как индивидуальные предприниматели, могут применять специальный налоговый режим «Налог на профессиональный доход» в соответствии с Федеральным законом № 422-ФЗ от 27.11.2018 г.[25]

1.6 Анализ существующих решений и аналогов

Для улучшения качества собственного продукта следует провести анализ уже существующих решений. Сфера онлайн-бронирования с каждым годом развивается и расширяется. Наиболее востребованными в данном сегменте являются платформы Rentomania, Авито и Арендую.ру. Сравнение функционала данных площадок представлено в таблице 1.1.

Таблица 1.1 – Сравнение существующих решений и аналогов

Параметр	Rentomania	Авито	Арендую.ру	Проектируемая онлайн площадка
Размещение объявлений арендодателем	+	+	+	+
Поиск и фильтрация товаров	+	+	+	+
Онлайн-бронирование	+	—	—	+
Встроенная онлайн-оплата	+	+	—	+
Система рейтинга и отзывов	+	+	—	+
Чат между арендатором и арендодателем	+	+	—	+
Адаптивная вёрстка для мобильных устройств	+	+	—	+

Продолжение таблицы 1.1

Параметр	Rentomania	Авито	Арендую.ру	Проекти- руемая онлайн площадка
Верификация пользовате- лей	+	—	—	+
Панель аналитики для арендо- дателя	—	—	—	+
Понятный интерфейс	—	+	—	+
Уведомления о статусе аренды	+	—	—	+

2 Техническое задание

2.1 Основание для разработки

Основанием для разработки является задание на выпускную квалификационную работу бакалавра «Онлайн-площадка для сдачи вещей в аренду».

2.2 Цель и назначение разработки

Основной задачей выпускной квалификационной работы является разработка и тестирование веб-приложения, предоставляющего возможность арендодателям публиковать объявления о сдаче вещей в аренду, а арендаторам - находить, бронировать и оплачивать такие вещи с обеспечением базовых гарантий безопасности сделки.

Задачами данной разработки являются:

- изучение и анализ предметной области;
- проектирование интерфейса веб-приложения;
- проектирование базы данных;
- проектирование модуля взаимодействия с пользователем;
- проектирование API веб-приложения.

2.3 Требования к веб-приложению

2.3.1 Функциональные требования к веб-приложению

На основании описанных бизнес-процессов и классификации площадок был сформирован перечень функциональных требований, представленный в таблице 2.1.

Таблица 2.1 – Функциональные требования к системе

№	Функциональное требование	Описание
1	Регистрация и аутентификация	Создание учётной записи через email, вход по логину/паролю, восстановление доступа.

Продолжение таблицы 2.1

№	Функциональное требование	Описание
2	Верификация пользователей	Вводе сведений из документа, удостоверяющего личность, присвоение статуса «Верифицирован».
3	Управление профилем	Редактирование персональных данных, смена данных, просмотр истории сделок.
4	Размещение объявления	Создание карточки товара: название, категория, описание, фотографии, цена.
5	Управление объявлениями	Редактирование, удаление объявления.
6	Фильтрация	Фильтры по категории, дате доступности.
7	Просмотр карточки товара	Отображение фото, описания, характеристик, отзывов, цены.
8	Онлайн-бронирование	Выбор дат аренды, проверка доступности, подтверждение брони арендодателем.
9	Онлайн-оплата	Оплата банковской картой через платёжный шлюз, удержание средств до подтверждения получения.
10	Чат между пользователями	Встроенный мессенджер для обсуждения деталей сделки без раскрытия личных контактов.

Продолжение таблицы 2.1

№	Функциональное требование	Описание
11	Система отзывов	Оценки после завершения сделки, накопленный рейтинг пользователей и объявлений.
12	Уведомления	Push- и email-уведомления о бронировании, оплате, сообщениях, истечении срока аренды.
13	Генератор договоров	Автоматическое создание договора при оформлении аренды; возможность продолжить сделку только после подписания обеими сторонами.
14	Панель аналитики арендодателя	Статистика бронирований, доходов объявлений за период.
15	Административная панель	Модерация объявлений и пользователей, управление категориями, просмотр транзакций, сводная аналитика.

2.3.2 Требования пользователя к интерфейсу

Веб-приложение должно включать в себя:

- регистрацию;
- авторизацию;
- профиль пользователя;
- окно с статистикой по сделкам;
- встроенный чат между участниками сделки;
- модуль бронирования с выбором дат аренды и расчётом стоимости;
- модуль договоров аренды;

- модуль создания и редактирования объявления.
- карточку товара;
- окно с входящими заявками;

Схема окна регистрации пользователя представлена на рисунке 2.1 и состоит из:

- кнопка для возвращения на предыдущую страницу (1);
- поле для ввода имени пользователя (2);
- поле для ввода электронной почты (3);
- поле для ввода пароля (4);
- поле для повторного ввода пароля (5);
- поле для выбора типа пользователя (6);
- поля для заполнения реквизитов (ИНН, КПП, зависит от типа пользователя) (7);
- кнопка для регистрации пользователя (8).

Назад → 1

Регистрация

Имя пользователя → 2

Email → 3

Пароль → 4

Подтверждение пароля → 5

Тип пользователя → 6

Поля для заполнения реквизитов → 7

Зарегистрироваться → 8

Рисунок 2.1 – Окно регистрации пользователя

Схема окна авторизации пользователя представлена на рисунке 2.2 и состоит из:

- кнопка для возвращения на предыдущую страницу (1);

- поле для ввода имени пользователя (2);
- поле для ввода пароля (3);
- кнопка для авторизации пользователя (4).

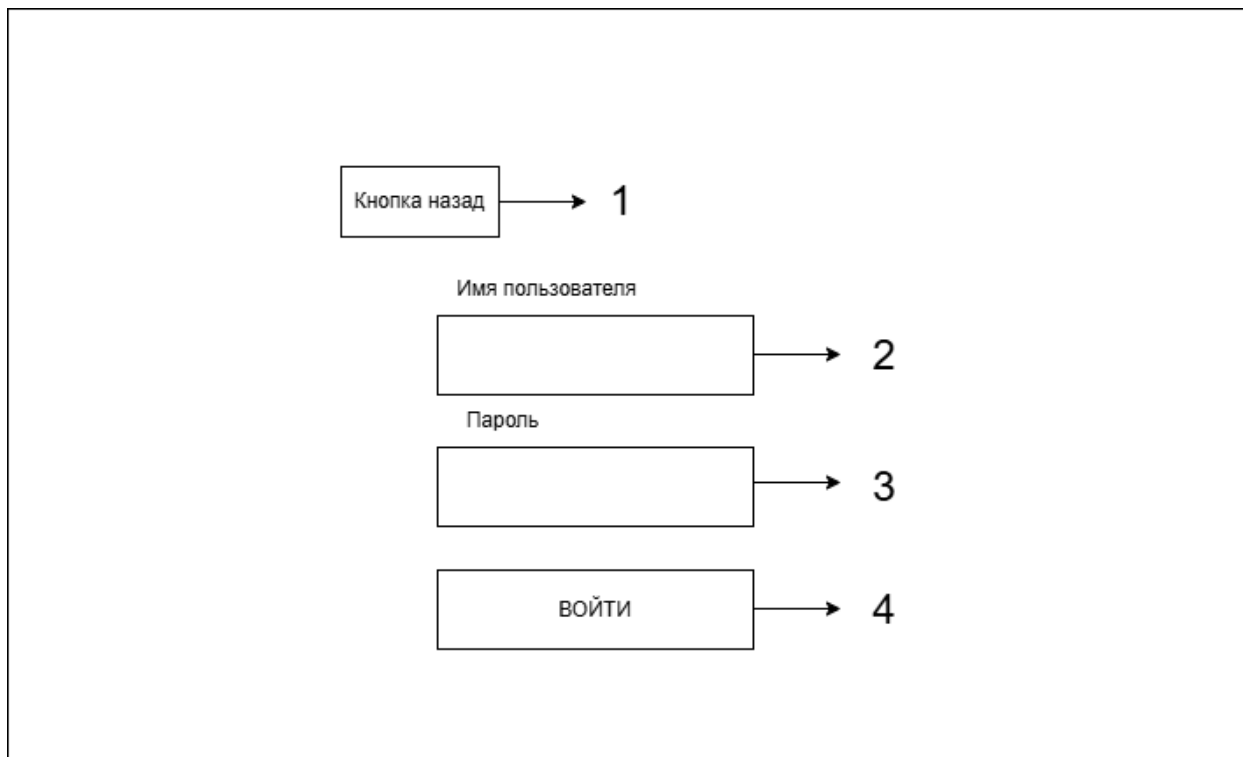


Рисунок 2.2 – Окно авторизации пользователя

Схема окна профиля пользователя представлена на рисунке 2.3 и состоит из:

- кнопка для возвращения на предыдущую страницу (1);
- поле для ввода имени пользователя (2);
- поле для ввода электронной почты (3);
- поля для заполнения реквизитов (ИНН, КПП, зависит от типа пользователя) (4);
- кнопка для сохранения изменений (5);
- поле для ввода текущего пароля (6);
- поле для ввода нового пароля (7);
- поле для повторного ввода нового пароля (8);
- кнопка для смены пароля (9).

Назад → 1

Мой профиль

Тип пользователя

Имя пользователя → 2

Email → 3

Данные

Поля для
определенного типа
реквизитов → 4

Сохранить изменения → 5

Смена пароля

Текущий пароль → 6

Новый пароль → 7

Подтверждение пароля → 8

Сменить пароль → 9

Рисунок 2.3 – Окно профиля пользователя

Схема окна с статистикой по сделкам представлена на рисунке 2.4 и состоит из:

- поле для вывода общего количества объявлений созданные пользователем (1);
- поле для вывода количества бронирований у пользователя (2);
- поле для вывода количества забронированных товаров пользователем (3);
- поле для вывода общей прибыли за аренду товаров пользователя (4);
- поле для вывода объявлений созданных пользователем (5);
- поле для вывода объявлений находящихся в аренде (6);

- поле для вывода объявлений с истекшим сроком аренды (7);
- поле для вывода объявлений забронированных пользователем (8).

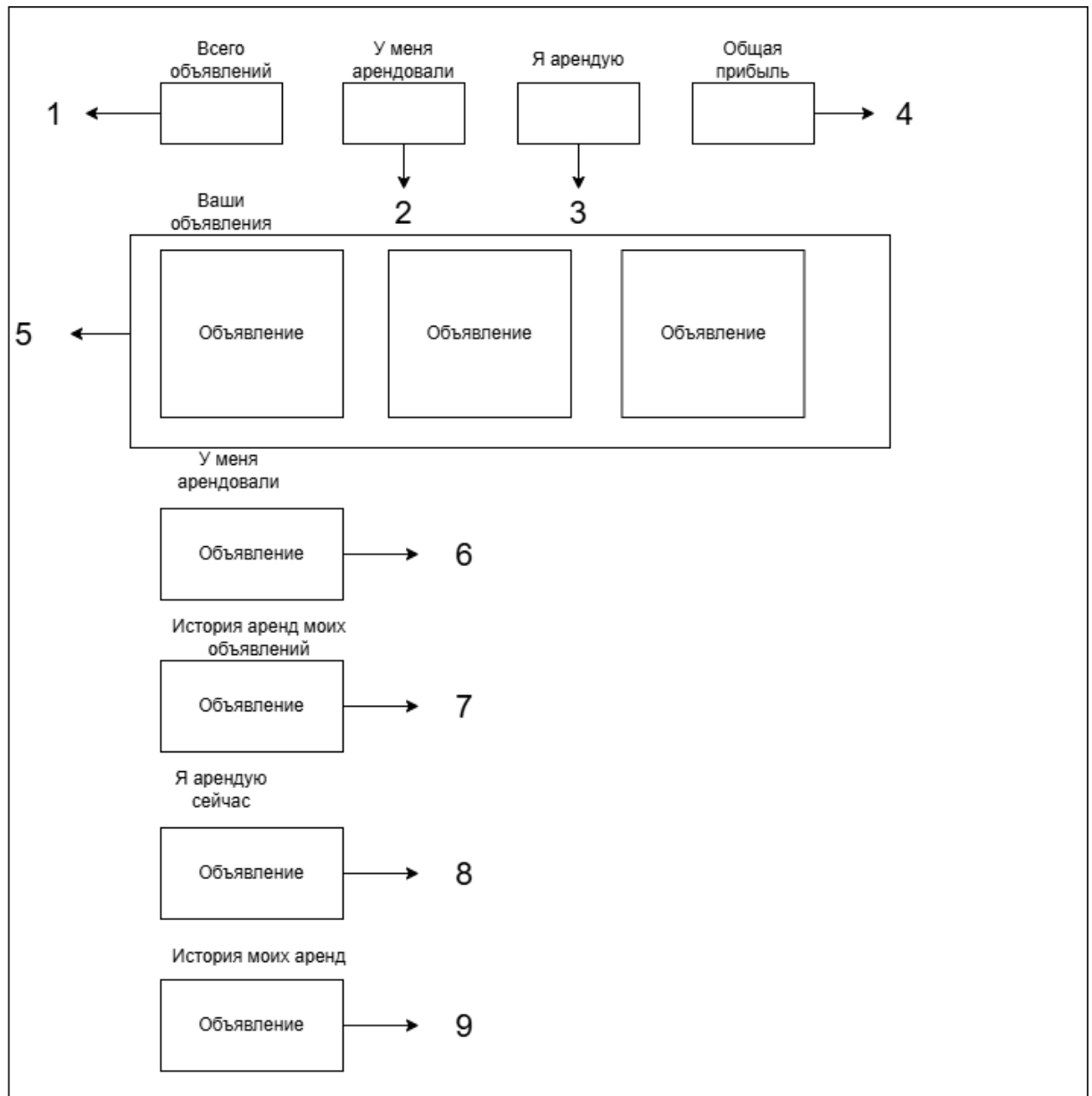


Рисунок 2.4 – Окно с статистикой по сделкам

Схема окна встроенного чат между участниками сделки представлена на рисунке 2.5 и состоит из:

- кнопка для возвращения на предыдущую страницу (1);
- кнопка для отправки сообщения (2);
- поле для ввода сообщения (3);
- кнопка для загрузки изображения в сообщение (4).

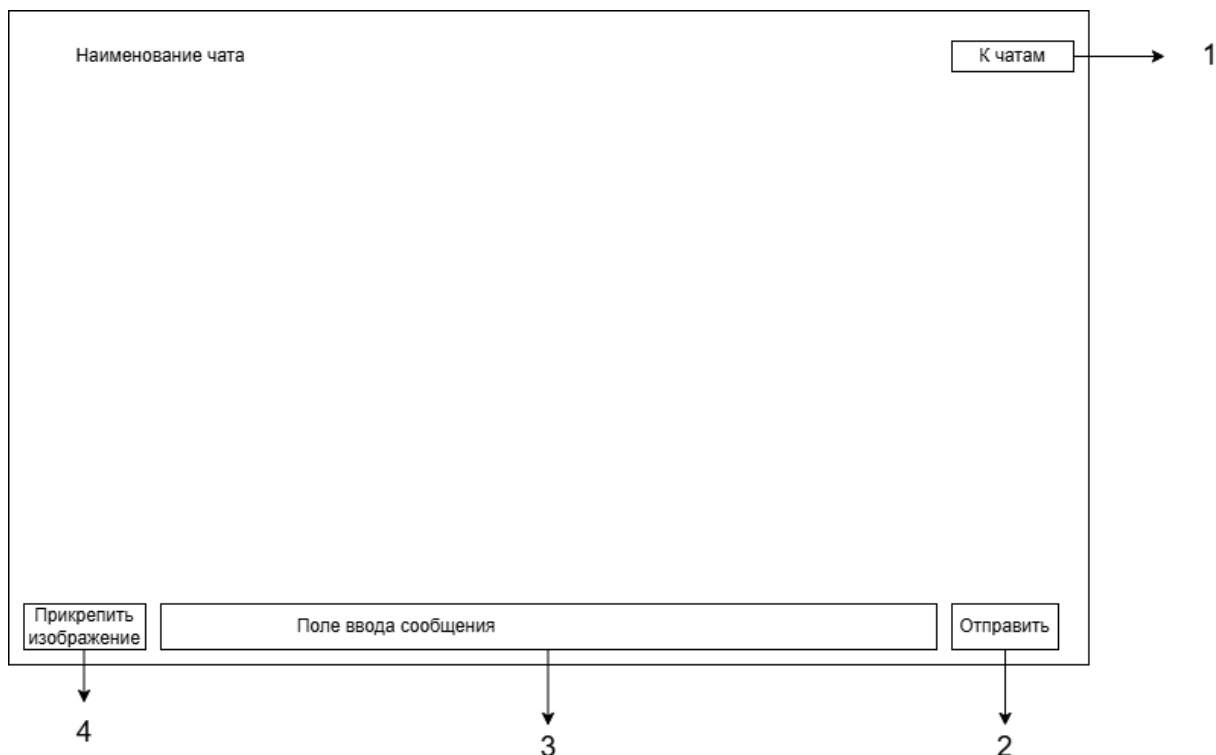


Рисунок 2.5 – Окно встроенного чат между участниками сделки

Схема интерфейса модуля бронирования с выбором дат аренды и расчётом стоимости представлена на рисунке и состоит из:

- поле для ввода даты начала аренды (1);
- поле для ввода даты окончания аренды (2);
- поле для вывода статуса брони (3);
- кнопка для подписи договора аренды через демонстрационный механизм ЭЦП (4);
- кнопка для отправки заявки на бронь (5).

Наименование

Дата начала аренды

→ 1

Дата окончания аренды

→ 2

Статус брони

→ 3

Отправить заявку

↓ 5

Подписать документ с помощью ЭЦП

↓ 4

Рисунок 2.6 – Окно встроенного чат между участниками сделки

Схема интерфейса модуля модуля договоров аренды представлена на рисунке 2.7 и состоит из:

- поле для вывода данных договора(1);
- поле для вывода сводки по договору (2);
- кнопка для установки PDF-файла договора аренды (3);
- поле для ввода ФИО подписанта(4);
- поле для ввода pin сертификата (5);
- кнопка для подписывание договора (6).

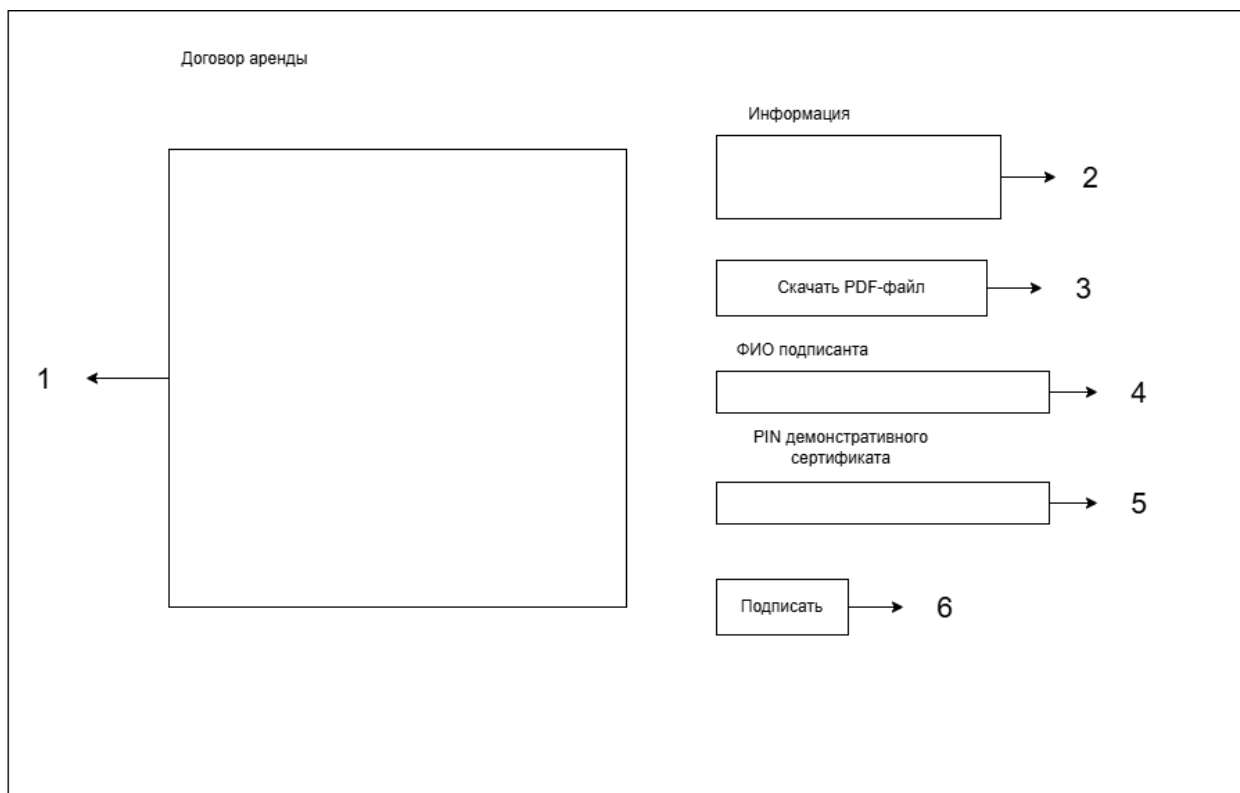


Рисунок 2.7 – Интерфейс модуля договоров аренды

Схема интерфейса модуля создания и редактирования объявления представлена на рисунке 2.8 и состоит из:

- кнопка для возвращения на предыдущую страницу (1);
- поле для ввода наименования товара (2);
- поле для ввода описания товара (3);
- поле для ввода цены товара (4);
- поле для выбора категории товара (5);
- поле для загрузки изображения (6);
- кнопка для публикации объявления (7).

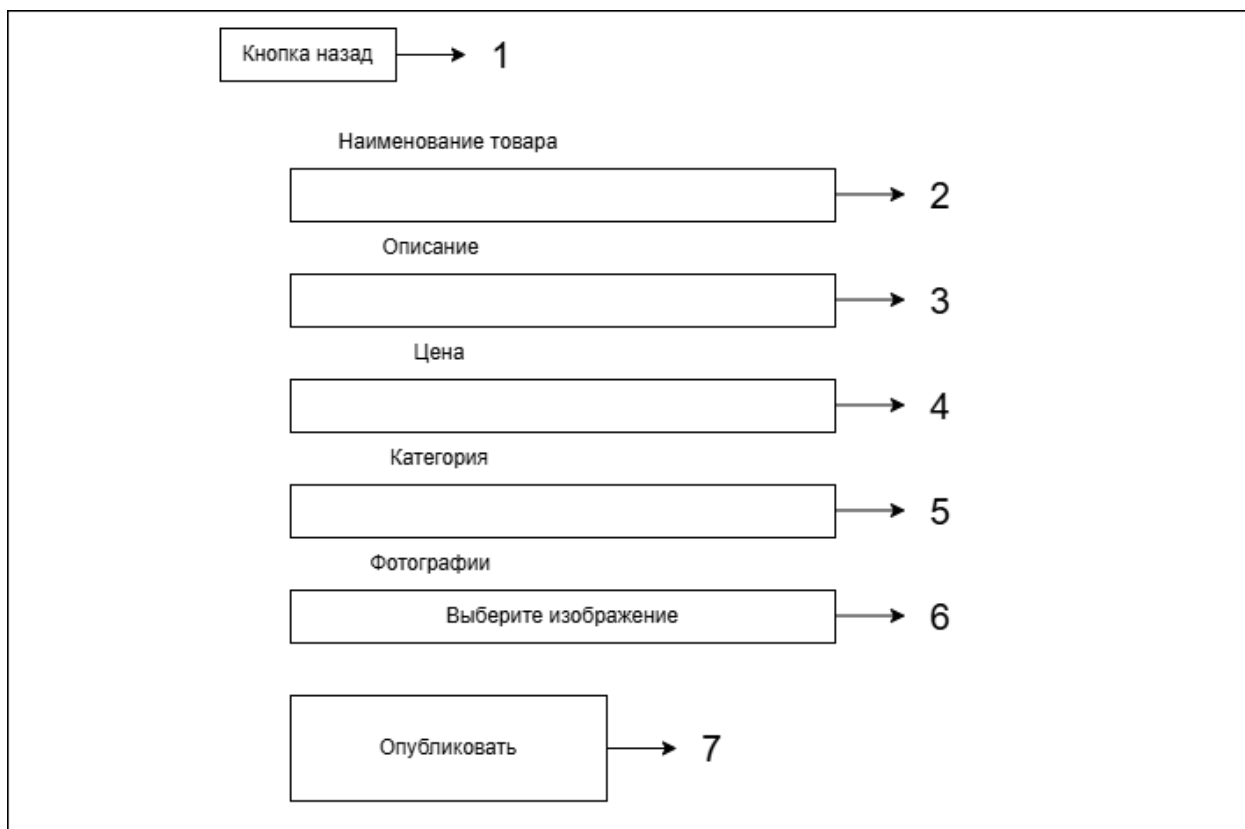


Рисунок 2.8 – Интерфейс модуля создания и редактирования объявления

Схема окна карточки товара представлена на рисунке 2.9 и состоит из:

- поле для вывода изображений (1);
- кнопка для отправки заявки на аренду товара (2);
- кнопка для перенаправления пользователя в чат с владельцем объявления (3);
- поле для вывода отзывов по объявлению (4).

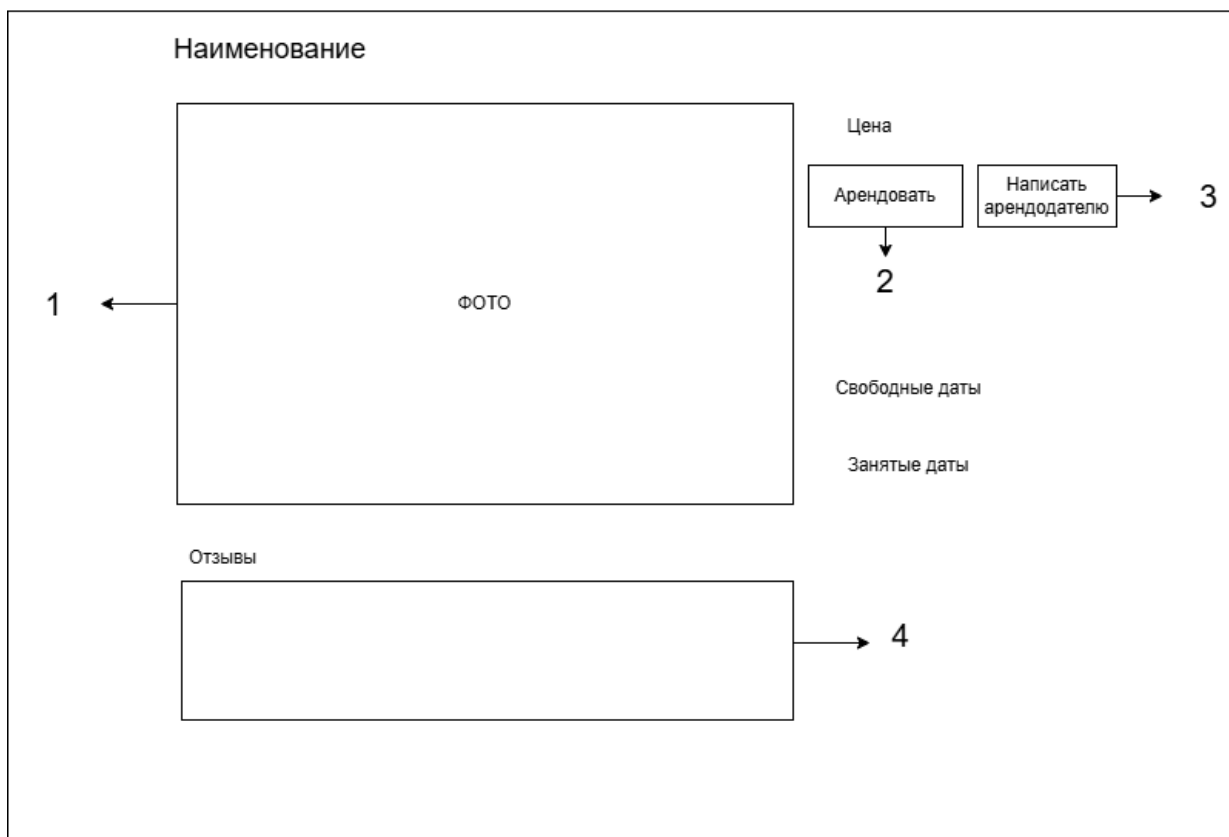


Рисунок 2.9 – Окно карточки товара

Схема окна с входящими заявками представлена на рисунке 2.10 и состоит из:

- кнопка для подтверждения заявки (1);
- кнопка для отклонения заявки (2);
- поле для вывода входящих заявок (3);
- поле для вывода заявок оставленные пользователем (4);
- кнопка для открытия объявления (5).

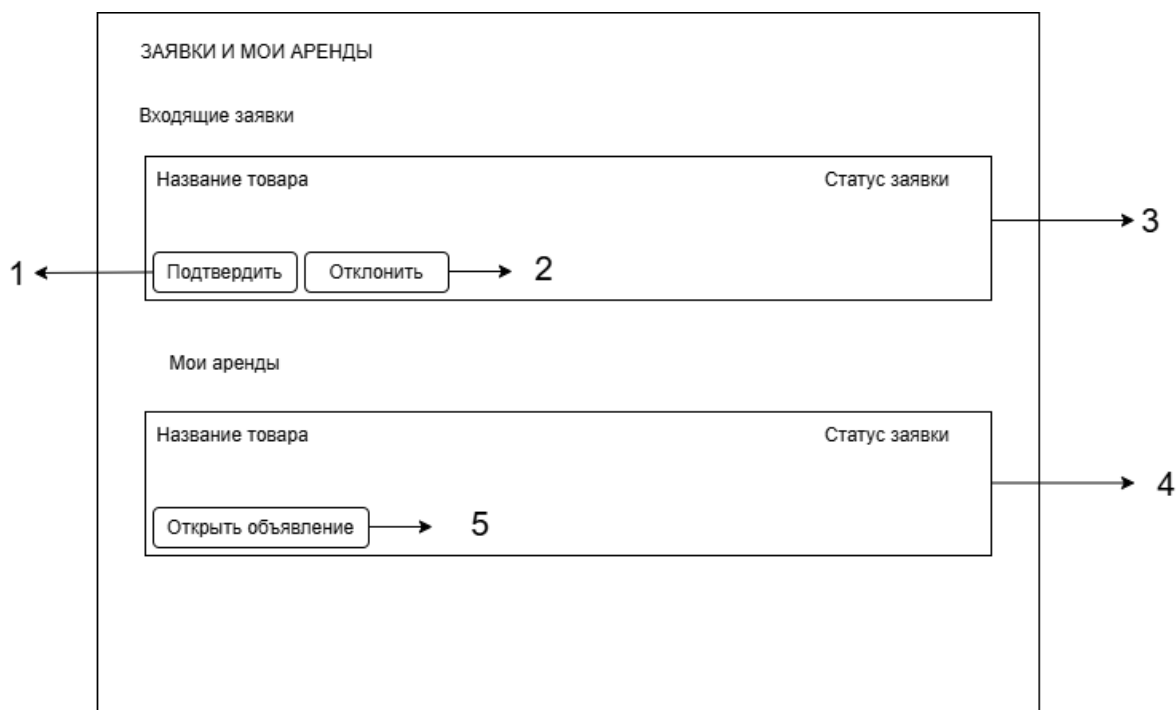


Рисунок 2.10 – Окно с входящими заявками

2.4 Моделирование вариантов использования

Для разрабатываемого веб-приложения была реализована модель, обеспечивающая наглядное представление вариантов использования программы с помощью унифицированного языка визуального моделирования UML. На основании анализа предметной области в программе должны быть реализованы следующие прецеденты:

1. Регистрация
2. Авторизация
3. Изменение пароля пользователя.
4. Просмотр профиля пользователя.
5. Редактирование профиля пользователя.
6. Просмотр каталога вещей.
7. Фильтрация объявлений по категории и дате.
8. Создание собственных объявлений.
9. Редактирование собственных объявлений.
10. Создание бронирования вещи с автоматическим расчетом стоимости.

11. Удаление собственных объявлений.
12. Загрузка изображений в объявление.
13. Удаление изображений в объявлении.
14. Просмотр карточки вещи, ее рейтинга, отзывов и занятых периодов аренды.
15. Управление статусами бронирования: подтверждение, отмена.
16. Просмотр входящих и исходящих бронирований пользователя.
17. Оплата бронирования через демонстрационный механизм СБП.
18. Формирование и подписание договора аренды участниками сделки.
19. Создание отзывов по завершенным или оплаченным бронированиям.
20. Чат между пользователями с возможностью отправить изображение.
21. Помощь на часто задаваемые вопросы.

На рисунке 2.11 изображены прецеденты для пользователя.

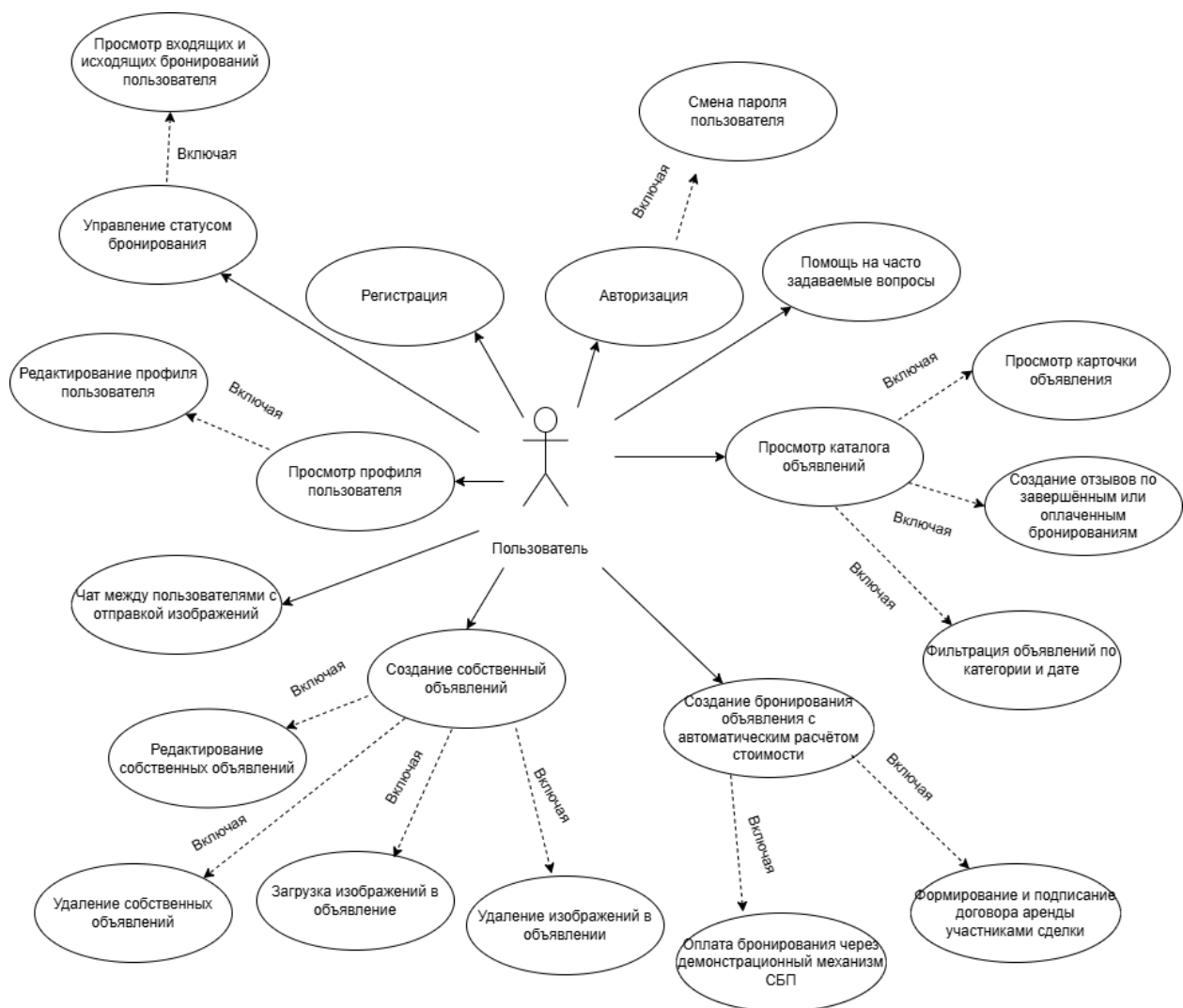


Рисунок 2.11 – Диаграмма прецедентов для пользователя

2.4.1 Регистрация

Заинтересованные лица и их требования: пользователь желает создать учетную запись для получения доступа к функциям сервиса аренды вещей. Предусловие: пользователь находится на странице регистрации. Постусловие: в системе создана новая учетная запись пользователя.

1. Пользователь открывает форму регистрации.
2. Пользователь вводит имя пользователя, email, пароль и выбирает тип аккаунта.
3. Веб-приложение передает введенные данные на сервер.
4. Сервер проверяет корректность данных, уникальность email и длину пароля.

5. Сервер создает учетную запись пользователя и сохраняет ее в базе данных.

6. Веб-приложение сообщает пользователю об успешной регистрации.

2.4.2 Авторизация

Заинтересованные лица и их требования: пользователь желает войти в систему для доступа к личному кабинету, объявлениям, бронированиям и сообщениям. Предусловие: пользователь зарегистрирован в системе и находится на странице входа. Постусловие: пользователь авторизован и получает доступ к функциям личного кабинета. Основной успешный сценарий:

1. Пользователь открывает форму авторизации.
2. Пользователь вводит email или имя пользователя и пароль.
3. Приложение передает данные авторизации на сервер.
4. Сервер проверяет соответствие введенных данных учетной записи.
5. Сервер подтверждает авторизацию пользователя.
6. Веб-приложение открывает пользователю доступ к личному кабинету.

2.4.3 Изменение пароля пользователя

Заинтересованные лица и их требования: пользователь желает изменить пароль для защиты своей учетной записи.

Предусловие: пользователь авторизован в системе.

Постусловие: пароль пользователя изменен, дальнейшая авторизация выполняется с новым паролем.

Основной успешный сценарий:

1. Пользователь восстанавливает пароль без входа в систему, он открывает форму «Забыли пароль?».
2. Пользователь вводит email, указанный при регистрации, и нажимает кнопку отправки кода.
3. Приложение передает email на сервер.

4. Сервер находит пользователя по email, генерирует шестизначный код подтверждения и сохраняет его во временном хранилище.
5. Сервер отправляет код подтверждения на email пользователя.
6. Пользователь вводит полученный код и новый пароль.
7. Приложение передает email, код подтверждения и новый пароль на сервер.
8. Сервер проверяет корректность кода и срок его действия.
9. Сервер проверяет, что новый пароль соответствует требованиям, в частности содержит не менее 6 символов.
10. Сервер сохраняет новый пароль пользователя и удаляет использованный код подтверждения.
11. Приложение сбрасывает текущую сессию пользователя и предлагает войти заново с новым паролем.

2.4.4 Просмотр профиля пользователя

Заинтересованные лица и их требования: пользователь желает просмотреть данные своего аккаунта и проверить заполненность профиля.

Предусловие: пользователь авторизован в системе.

Постусловие: пользователю отображаются данные его профиля.

Основной успешный сценарий:

1. Пользователь открывает раздел «Профиль».
2. Приложение отправляет запрос на сервер для получения данных текущего пользователя.
3. Сервер находит профиль пользователя в базе данных.
4. Сервер возвращает данные профиля: имя пользователя, email, тип аккаунта и реквизиты.
5. Приложение отображает полученные данные пользователю.

2.4.5 Редактирование профиля пользователя

Заинтересованные лица и их требования: пользователь желает изменить данные профиля с учетом типа аккаунта.

Предусловие: пользователь авторизован и открыл раздел «Профиль».

Постусловие: данные профиля обновлены, признак заполненности профиля пересчитан.

Основной успешный сценарий:

1. Пользователь нажимает кнопку редактирования профиля.
2. Пользователь изменяет необходимые поля.
3. Для физического лица указываются ФИО, серия и номер паспорта, ИНН.
4. Приложение передает обновленные данные на сервер.
5. Сервер проверяет корректность и заполненность обязательных полей.
6. Сервер сохраняет изменения в базе данных.
7. Приложение отображает обновленный профиль пользователя.

2.4.6 Просмотр каталога вещей

Заинтересованные лица и их требования: пользователь желает посмотреть список вещей, доступных для аренды.

Предусловие: пользователь открыл страницу каталога вещей.

Постусловие: на странице отображается список объявлений о вещах.

Основной успешный сценарий:

1. Пользователь переходит в раздел каталога вещей.
2. Приложение отправляет запрос на сервер для получения списка объявлений.
3. Сервер выбирает из базы данных доступные вещи.
4. Сервер возвращает сведения о вещах: название, описание, цену, категорию, рейтинг и изображения.
5. Приложение отображает полученный список объявлений.
6. Пользователь просматривает каталог и выбирает интересующую вещь.

2.4.7 Фильтрация объявлений по категории и дате

Заинтересованные лица и их требования: пользователь желает отобразить объявления по нужной категории и отсортировать их по дате создания.

Предусловие: пользователь находится в каталоге вещей.

Постусловие: каталог обновлен и содержит объявления, соответствующие выбранным параметрам.

Основной успешный сценарий:

1. Пользователь выбирает категорию вещи в интерфейсе каталога.
2. Пользователь выбирает сортировку по дате создания.
3. Приложение передает выбранные параметры фильтрации и сортировки на сервер.
4. Сервер формирует запрос к базе данных с учетом категории.
5. Сервер сортирует найденные объявления по дате создания.
6. Сервер передает результат в приложение.
7. Приложение обновляет список отображаемых объявлений.

2.4.8 Создание собственных объявлений

Заинтересованные лица и их требования: пользователь желает разместить собственную вещь для аренды.

Предусловие: пользователь авторизован, его профиль заполнен.

Постусловие: новое объявление создано и доступно в системе.

Основной успешный сценарий:

1. Пользователь открывает форму создания объявления.
2. Пользователь вводит название, описание, цену за день и выбирает категорию.
3. Пользователь указывает статус вещи.
4. Приложение передает данные объявления на сервер.
5. Сервер проверяет авторизацию пользователя и заполненность профиля.

6. Сервер сохраняет объявление и назначает пользователя владельцем вещи.

7. Приложение отображает созданное объявление в списке вещей.

2.4.9 Редактирование собственных объявлений

Заинтересованные лица и их требования: владелец вещи желает изменить данные ранее созданного объявления.

Предусловие: пользователь авторизован и является владельцем объявления.

Постусловие: данные объявления обновлены.

Основной успешный сценарий:

1. Пользователь открывает список своих объявлений.
2. Пользователь выбирает объявление и нажимает кнопку редактирования.
3. Пользователь изменяет название, описание, цену, категорию или статус вещи.
4. Приложение передает обновленные данные на сервер.
5. Сервер проверяет, что текущий пользователь является владельцем объявления.
6. Сервер сохраняет изменения в базе данных.
7. Приложение отображает обновленную карточку объявления.

2.4.10 Создание бронирования вещи с автоматическим расчетом стоимости

Заинтересованные лица и их требования: пользователь желает забронировать вещь и сразу увидеть итоговую стоимость аренды.

Предусловие: пользователь авторизован, профиль заполнен, открыта карточка чужой вещи.

Постусловие: бронирование создано, итоговая стоимость рассчитана автоматически.

Основной успешный сценарий:

1. Пользователь нажимает кнопку бронирования в карточке вещи.
2. Пользователь выбирает дату начала и дату окончания аренды.
3. Приложение передает данные бронирования на сервер.
4. Сервер проверяет, что пользователь не является владельцем вещи.
5. Сервер проверяет заполненность профиля пользователя.
6. Сервер рассчитывает количество дней аренды.
7. Сервер умножает количество дней на цену вещи за день.
8. Сервер сохраняет бронирование со статусом рассматриваемое.
9. Приложение отображает созданную заявку и рассчитанную стоимость.

2.4.11 Удаление собственных объявлений

Заинтересованные лица и их требования: владелец вещи желает удалить свое объявление из системы.

Предусловие: пользователь авторизован и является владельцем объявления.

Постусловие: объявление удалено из базы данных и больше не отображается в каталоге.

Основной успешный сценарий:

1. Пользователь открывает список своих объявлений.
2. Пользователь выбирает объявление для удаления.
3. Пользователь подтверждает удаление.
4. Приложение отправляет запрос на сервер.
5. Сервер проверяет, что пользователь является владельцем объявления.
6. Сервер удаляет объявление из базы данных.
7. Приложение обновляет список объявлений пользователя.

2.4.12 Загрузка изображений в объявление

Заинтересованные лица и их требования: владелец объявления желает добавить изображения вещи для более подробного представления объявления.

Предусловие: пользователь авторизован и является владельцем объявления.

Постусловие: изображение загружено и связано с объявлением.

Основной успешный сценарий:

1. Пользователь открывает свое объявление.
2. Пользователь выбирает изображение для загрузки.
3. Приложение передает файл изображения и идентификатор вещи на сервер.
4. Сервер проверяет, что вещь принадлежит текущему пользователю.
5. Сервер сохраняет изображение в файловом хранилище.
6. Сервер связывает изображение с объявлением.
7. Приложение отображает загруженное изображение в карточке вещи.

2.4.13 Удаление изображений в объявлении

Заинтересованные лица и их требования: владелец объявления желает удалить лишнее или устаревшее изображение вещи.

Предусловие: пользователь авторизован и является владельцем объявления, к объявлению добавлено изображение.

Постусловие: выбранное изображение удалено из объявления.

Основной успешный сценарий:

1. Пользователь открывает свое объявление.
2. Пользователь выбирает изображение для удаления.
3. Пользователь подтверждает удаление изображения.
4. Приложение отправляет запрос на сервер.
5. Сервер проверяет, что изображение относится к вещи текущего пользователя.

6. Сервер удаляет изображение.
7. Приложение обновляет список изображений объявления.

2.4.14 Просмотр карточки вещи, ее рейтинга, отзывов и занятых периодов аренды

Заинтересованные лица и их требования: пользователь желает получить подробную информацию о вещи перед арендой.

Предусловие: пользователь находится в каталоге вещей.

Постусловие: открыта карточка вещи с подробной информацией.

Основной успешный сценарий:

1. Пользователь выбирает вещь в каталоге.
2. Приложение отправляет запрос на сервер по идентификатору вещи.
3. Сервер получает данные вещи, владельца, категории и изображений.
4. Сервер получает рейтинг вещи и количество отзывов.
5. Сервер получает список отзывов о вещи.
6. Сервер определяет занятые периоды аренды по активным бронированиям.
7. Приложение отображает карточку вещи, рейтинг, отзывы и занятые даты.

2.4.15 Управление статусами бронирования: подтверждение, отмена

Заинтересованные лица и их требования: владелец и арендатор желают управлять заявкой на аренду.

Предусловие: бронирование уже создано.

Постусловие: статус бронирования изменен на принято или отменено.

Основной успешный сценарий:

1. Владелец вещи открывает входящее бронирование.
2. Владелец нажимает кнопку подтверждения.
3. Приложение отправляет запрос на сервер.

4. Сервер проверяет, что текущий пользователь является владельцем вещи.
5. Сервер изменяет статус бронирования на принято.
6. Сервер отправляет арендатору уведомление о подтверждении.
7. При отмене участник сделки открывает бронирование.
8. Пользователь нажимает кнопку отмены.
9. Сервер проверяет, что пользователь является арендатором или владельцем.
10. Сервер изменяет статус бронирования на отменено.

2.4.16 Просмотр входящих и исходящих бронирований пользователя

Заинтересованные лица и их требования: пользователь желает просматривать заявки, созданные им самим, и заявки, поступившие на его вещи.

Предусловие: пользователь авторизован в системе.

Постусловие: пользователю отображается список входящих и исходящих бронирований.

Основной успешный сценарий:

1. Пользователь открывает раздел бронирований.
2. Приложение отправляет запрос на сервер.
3. Сервер выбирает бронирования, где пользователь является арендатором или владельцем вещи.
4. При выборе входящих бронирований сервер возвращает заявки на вещи пользователя.
5. При выборе исходящих бронирований сервер возвращает заявки, созданные пользователем.
6. Приложение отображает список бронирований.
7. Пользователь видит вещь, даты аренды, стоимость, статус бронирования и статус оплаты.

2.4.17 Оплата бронирования через демонстрационный механизм СБП

Заинтересованные лица и их требования: арендатор желает оплатить подтвержденное бронирование через демонстрационный механизм СБП.

Предусловие: бронирование подтверждено владельцем и имеет статус confirmed.

Постусловие: бронирование получает статус оплаты paid.

Основной успешный сценарий:

1. Арендатор открывает подтвержденное бронирование.
2. Пользователь нажимает кнопку оплаты.
3. Приложение отправляет запрос на сервер для запуска оплаты.
4. Сервер проверяет, что бронирование подтверждено и еще не оплачено.
5. Сервер создает демонстрационную СБП-сессию.
6. Сервер возвращает сумму, получателя, банк, ссылку и QR-данные.
7. Пользователь выполняет демонстрационную оплату.
8. Пользователь подтверждает оплату на сайте.
9. Сервер изменяет статус оплаты на оплачено.
10. Владелец вещи получает уведомление об оплате.

2.4.18 Формирование и подписание договора аренды участниками сделки

Заинтересованные лица и их требования: арендатор и владелец желают сформировать и подписать договор аренды по бронированию.

Предусловие: бронирование подтверждено или завершено.

Постусловие: договор сформирован, после подписи обеих сторон отмечен как подписанный.

Основной успешный сценарий:

1. Участник сделки открывает договор по бронированию.
2. Приложение отправляет запрос на сервер.

3. Сервер проверяет, что пользователь является участником сделки.
4. Сервер проверяет статус бронирования.
5. Сервер создает договор, если он еще не был создан.
6. Сервер формирует номер договора и файл с данными аренды.
7. Пользователь вводит имя подписанта и PIN демонстрационной подписи.
8. Сервер сохраняет подпись участника.
9. Второй участник выполняет подписание аналогичным образом.
10. После подписания обеими сторонами сервер отмечает договор как подписанный.

2.4.19 Создание отзывов по завершенным или оплаченным бронированиям

Заинтересованные лица и их требования: арендатор и владелец желают обсудить детали аренды во встроенном чате.

Предусловие: пользователь авторизован в системе.

Постусловие: сообщение отправлено, сохранено и доступно участникам чата.

Основной успешный сценарий:

1. Пользователь открывает карточку вещи или раздел чатов.
2. Пользователь выбирает собеседника и начинает диалог.
3. Приложение передает на сервер идентификатор собеседника и при необходимости идентификатор вещи.
4. Сервер проверяет наличие существующего чата между пользователями.
5. Если чат отсутствует, сервер создает новый чат.
6. Пользователь вводит текст сообщения или прикрепляет изображение.
7. Приложение отправляет сообщение на сервер.
8. Сервер сохраняет сообщение в базе данных.
9. Сервер отправляет уведомление получателю.

10. Приложение отображает новое сообщение в чате.

2.4.20 Чат между пользователями с возможностью отправить изображение

Заинтересованные лица и их требования: арендатор и владелец желают обсудить детали аренды во встроенном чате.

Предусловие: пользователь авторизован в системе.

Постусловие: сообщение отправлено, сохранено и доступно участникам чата.

Основной успешный сценарий:

1. Пользователь открывает карточку вещи или раздел чатов.
2. Пользователь выбирает собеседника и начинает диалог.
3. Приложение передает на сервер идентификатор собеседника и при необходимости идентификатор вещи.
4. Сервер проверяет наличие существующего чата между пользователями.
5. Если чат отсутствует, сервер создает новый чат.
6. Пользователь вводит текст сообщения или прикрепляет изображение.
7. Сервер сохраняет сообщение в базе данных.
8. Сервер отправляет уведомление получателю.
9. Приложение отображает новое сообщение в чате.

2.4.21 Помощь на часто задаваемые вопросы

Заинтересованные лица и их требования: пользователь желает получить справочную информацию по основным действиям в системе аренды вещей: созданию объявления, заполнению профиля, оформлению аренды, оплате, переписке и просмотру чатов.

Предусловие: пользователь находится на сайте и открыл раздел помощи или окно поддержки.

Постусловие: пользователь получает список часто задаваемых вопросов и ответ с пошаговой инструкцией по выбранной теме.

Основной успешный сценарий:

1. Пользователь открывает раздел помощи или окно поддержки на сайте.
2. Приложение отправляет запрос на сервер для получения списка часто задаваемых вопросов.
3. Сервер получает подготовленный список FAQ-вопросов из системы.
4. Сервер передает в приложение список вопросов и связанных с ними ответов.
5. Приложение отображает пользователю доступные темы помощи.
6. Пользователь выбирает интересующий вопрос, например о создании объявления, заполнении профиля, бронировании, оплате или чатах.
7. Приложение отображает ответ по выбранной теме.
8. Пользователь просматривает пошаговую инструкцию.
9. При необходимости пользователь переходит в соответствующий раздел сайта и выполняет действие по инструкции.

2.5 Требования к оформлению документации

Разработка программной документации и программного изделия должна производиться согласно ГОСТ 19.102-77 и ГОСТ 34.601-90. Единая система программной документации.

3 Технический проект

3.1 Архитектура онлайн-площадки для сдачи вещей в аренду

Необходимо спроектировать и разработать веб-приложение для онлайн площадки для сдачи вещей в аренду. Проектируемая программа представляет собой веб-приложение, построенное по архитектуре «клиент – сервер». Клиентская часть представлена веб-интерфейсом на HTML/CSS/JavaScript, который открывается в браузере пользователя. Серверная часть реализована на языке Python с использованием веб-фреймворка Django и предоставляет REST API для чтения и изменения данных. Хранение структурированных данных выполняется в СУБД PostgreSQL, кэширование и управление WebSocket-соединениями — в Redis. Развёртывание всех компонентов серверной части (Django, PostgreSQL, Redis, Nginx) выполняется с использованием системы контейнеризации Docker, что обеспечивает идентичность окружений на этапах разработки, тестирования и эксплуатации.

Общая схема взаимодействия компонентов приведена на рисунке 3.1

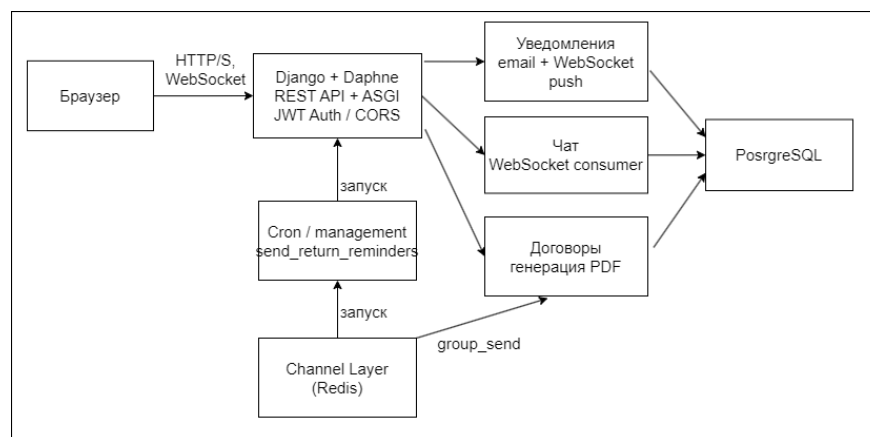


Рисунок 3.1 – Общая схема взаимодействия компонентов

Выбор такой архитектуры обоснован требованиями проекта: необходимо обеспечить одновременную работу пользовательской части (просмотр товаров, бронирование, чаты) и административной части с единой базой данных. Сервер централизует бизнес-логику, валидацию, контроль пересечения дат бронирования, а также статусную модель аренды.

3.2 Обоснование выбора технологий проектирования

Для реализации клиентской части выбраны HTML5, CSS3 и Vanilla JavaScript. Использование стандартных веб-технологий позволяет создать полноценный интерфейс без подключения крупных frontend-фреймворков. Для данного проекта такой подход является достаточным, поскольку приложение имеет понятную структуру: каталог вещей, формы, личный кабинет, бронирования, чаты и уведомления.

HTML5 используется для описания структуры интерфейса. CSS3 применяется для оформления страниц, адаптивной верстки и визуального выделения интерактивных элементов. JavaScript отвечает за динамическое изменение интерфейса, обработку действий пользователя, отправку запросов к серверу и работу с WebSocket.

Для взаимодействия с сервером используется Fetch API. Он встроен в современные браузеры и позволяет отправлять HTTP-запросы без дополнительных библиотек. В проекте все запросы централизованы через модуль `api.js`, что позволяет единообразно обрабатывать ответы сервера, ошибки и авторизацию.

Для хранения данных сессии применяется `localStorage`. В нем сохраняются JWT-токены, идентификатор пользователя, имя, email и тип учетной записи. Это позволяет сохранять состояние авторизации после перезагрузки страницы.

Для выбора периода аренды используется библиотека `flatpickr`. Она подключается к интерфейсу выбора дат и обеспечивает удобный ввод даты начала и окончания аренды. Использование готовой библиотеки снижает риск ошибок при ручном вводе дат и улучшает пользовательский опыт.

3.3 Архитектура клиентской части

Клиентская часть веб-приложения реализована как одностраничное приложение. Пользователь работает с одним HTML-документом, а смена экранов выполняется средствами JavaScript без полной перезагрузки страни-

цы. Такой подход позволяет сделать интерфейс более быстрым и удобным, так как при переходе между разделами обновляется только содержимое рабочей области приложения.

Клиентская часть состоит из основного файла `index.html`, общего файла стилей `styles.css` и набора JavaScript-модулей, расположенных в папке `js`. Такое разделение позволяет отделить структуру страницы, внешний вид интерфейса и программную логику приложения. Ниже приведена схема компонентов клиентской части 3.2

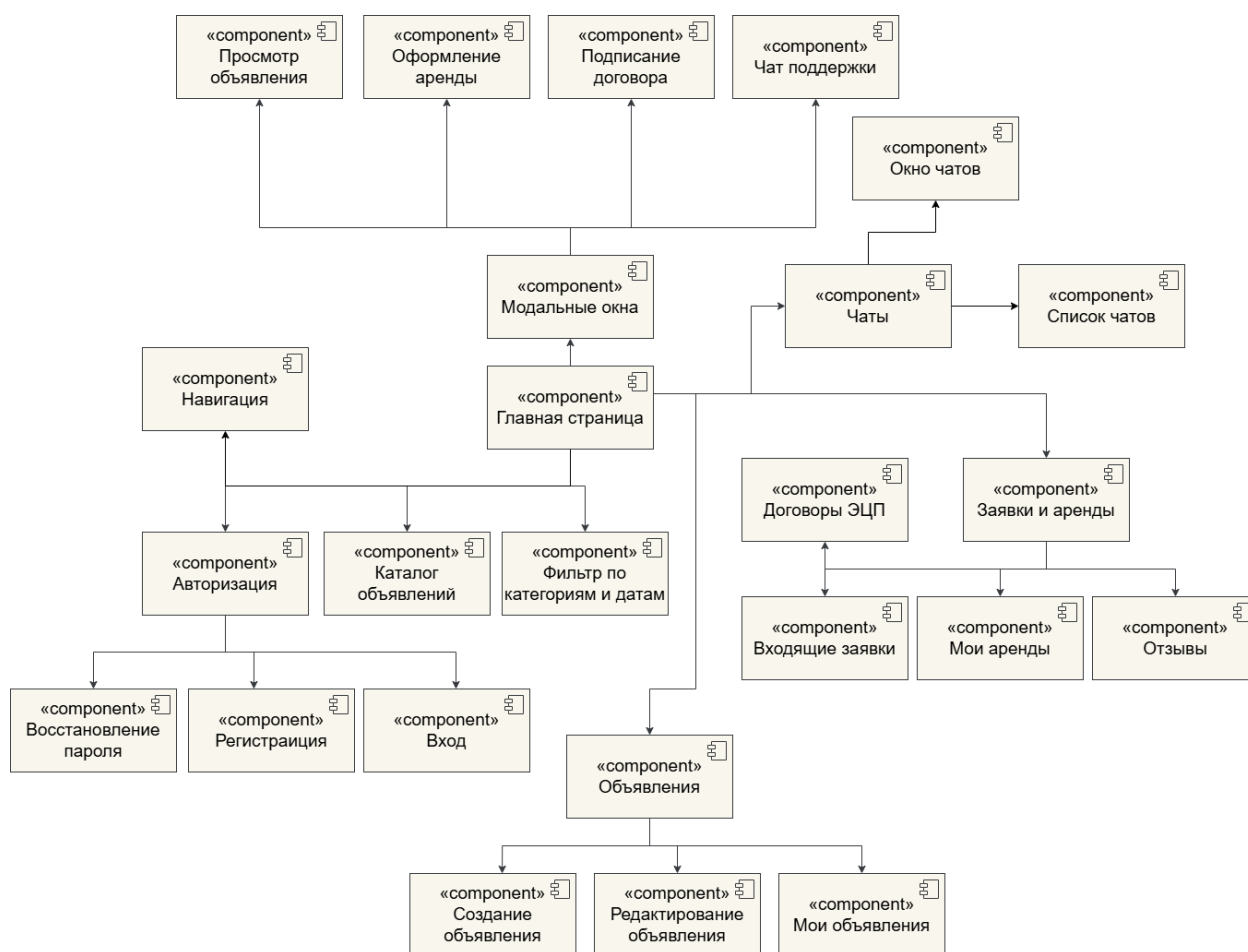


Рисунок 3.2 – Схема компонентов клиентской части

3.3.1 Файл `index.html`

Файл `index.html` выступает точкой входа в приложение. В нем размещается базовая HTML-структура страницы, подключаются таблица стилей, сторонние библиотеки и JavaScript-файлы. Также в нем находится основной

контейнер, в который с помощью JavaScript подставляется содержимое текущего раздела сайта.

3.3.2 Файл style.css

Файл styles.css содержит все стили клиентской части. В нем описывается внешний вид шапки сайта, карточек вещей, кнопок, форм, модальных окон, страниц профиля, бронирований, чатов и уведомлений. Кроме того, в этом файле задаются правила адаптивной верстки, благодаря которым интерфейс корректно отображается на компьютерах, планшетах и мобильных устройствах.

3.3.3 Файл config.js

JavaScript-код разделен на несколько файлов по функциональному назначению. Файл config.js хранит общие настройки приложения, например базовый адрес API и адрес WebSocket-соединений. Файл api.js отвечает за отправку запросов к серверу, добавление токена авторизации в заголовки, обработку ошибок и обновление access-токена.

3.3.4 Файл auth.js

Файл auth.js содержит логику регистрации, входа, выхода из аккаунта, восстановления пароля и проверки состояния авторизации. В нем также выполняется сохранение и удаление данных пользователя в localStorage.

3.3.5 Файл catalog.js

Файл catalog.js отвечает за работу каталога вещей. В нем реализована загрузка списка товаров, отображение карточек, получение категорий, фильтрация, пагинация и открытие подробной информации о выбранной вещи.

3.3.6 Файл bookings.js

Файл bookings.js содержит функции, связанные с арендой и бронированиями. Он отвечает за выбор периода аренды, создание заявки, получение

входящих и исходящих бронирований, а также подтверждение или отмену заявок.

3.3.7 Файл chats.js

Файл chats.js реализует работу с чатами. В нем находится логика получения списка диалогов, открытия переписки, отправки сообщений и изображений, а также подключения к WebSocket для получения новых сообщений в реальном времени.

3.3.8 Файл profile.js

Файл profile.js отвечает за личный кабинет пользователя. Через него выполняется загрузка данных профиля, отображение информации о пользователе, редактирование профиля и изменение пароля.

3.3.9 Файл notifications.js

Файл notifications.js содержит функции для работы с уведомлениями. Он отвечает за получение уведомлений с сервера, отображение счетчиков, обработку прочитанных уведомлений и подключение к WebSocket-каналу уведомлений.

3.3.10 Файл contracts.js

Файл contracts.js отвечает за работу с договорами аренды и демонстрационным процессом оплаты. В нем реализовано получение договора по бронированию, открытие договора, подписание и дальнейшее выполнение демонстрационной оплаты.

3.3.11 Файл main.js

Файл main.js является основным модулем запуска приложения. Он инициализирует интерфейс, подключает обработчики событий, проверяет состояние авторизации пользователя, загружает начальные данные и отображает главную страницу сайта.

3.3.12 Связь с API

Связь клиентской части с сервером выполняется через Django REST API. Для обычных операций используются HTTP-запросы к endpoint с базовым адресом `api`, а для работы в реальном времени применяются WebSocket-соединения. Такой подход позволяет совместить классическое API-взаимодействие с мгновенным получением сообщений и уведомлений.

3.4 Реализация ключевых функциональных модулей

3.4.1 Модуль каталога вещей

Модуль каталога реализован в файле `catalog.js`. Он отвечает за загрузку и отображение вещей, доступных для аренды. При открытии главной страницы модуль отправляет запрос к endpoint `/api/items/`, получает список объектов и формирует карточки вещей.

Каждая карточка содержит название, краткое описание, изображение, цену аренды за день, рейтинг и кнопки действий. Пользователь может открыть подробную информацию о вещи, перейти к аренде или начать чат с владельцем.

Также в модуле каталога реализована работа с категориями. Категории загружаются с сервера через API, после чего пользователь может выбрать нужную категорию. При выборе категории список вещей обновляется.

Дополнительно реализована фильтрация по доступности на выбранный период. Пользователь указывает даты аренды, после чего frontend передает параметры фильтрации серверу и отображает только подходящие вещи.

3.4.2 Модуль карточки вещи

Карточка вещи также относится к модулю `catalog.js`, так как является частью работы с каталогом. При выборе товара открывается модальное окно с подробной информацией. В нем отображаются изображения, описание, владелец, стоимость аренды, рейтинг и отзывы.

Из модального окна пользователь может начать процесс аренды или открыть чат с владельцем. Если текущий пользователь является владельцем вещи, интерфейс отображает дополнительные действия: редактирование или удаление объявления.

Такой подход позволяет пользователю получить всю основную информацию о вещи без перехода на отдельную страницу и без перезагрузки приложения.

3.4.3 Модуль аренды и бронирования

Модуль аренды и бронирования реализован в файле `bookings.js`. Он отвечает за выбор периода аренды, создание заявки и управление статусами бронирований.

При открытии окна аренды пользователь выбирает дату начала и дату окончания аренды. Для выбора дат используется библиотека `flatpickr`. После выбора периода `frontend` рассчитывает параметры аренды и отправляет запрос на создание бронирования через endpoint `/api/bookings/`.

В модуле также реализовано отображение входящих и исходящих бронирований. Входящие бронирования показывают заявки на вещи, принадлежащие текущему пользователю. Исходящие бронирования показывают заявки, созданные самим пользователем.

Владелец вещи может подтвердить или отменить заявку. Пользователь также может отслеживать статус бронирования: ожидает подтверждения, подтверждено, отменено или завершено.

3.4.4 Модуль авторизации и регистрации

Модуль авторизации расположен в файле `auth.js`. Он отвечает за регистрацию, вход, выход, восстановление пароля и хранение данных сессии.

При входе пользователь отправляет логин и пароль на endpoint `/api/token/`. Сервер возвращает `access` и `refresh` токены. Эти токены сохраняются в `localStorage` и используются при последующих защищенных запросах.

Регистрация выполняется через endpoint `/api/users/register/`. При регистрации пользователь указывает основные данные учетной записи и тип пользователя. В системе поддерживаются физическое лицо, индивидуальный предприниматель и юридическое лицо.

При выходе из аккаунта модуль очищает данные пользователя и токены из `localStorage`, после чего интерфейс возвращается в состояние неавторизованного пользователя.

3.4.5 Модуль личного кабинета

Модуль личного кабинета реализован в файле `profile.js`. Он отвечает за получение, отображение и изменение данных пользователя.

При открытии профиля `frontend` отправляет запрос к `/api/users/profile/` и получает данные текущего пользователя. В интерфейсе отображаются имя пользователя, `email`, тип учетной записи и дополнительные поля.

Набор дополнительных полей зависит от типа пользователя. Для физического лица используются паспортные данные и ИНН. Для индивидуально-го предпринимателя или юридического лица могут использоваться ОГРНИП, КПП, название организации и другие реквизиты.

Также в модуле профиля реализована смена пароля. Пользователь вводит старый и новый пароль, после чего `frontend` отправляет соответствующий запрос на сервер.

3.4.6 Модуль управления объявлениями

Функции управления объявлениями связаны с модулями `catalog.js` и `profile.js`. Пользователь может создать новое объявление, отредактировать существующее или удалить свою вещь.

При создании объявления пользователь указывает название, описание, цену за день, категорию и изображения. Основные данные вещи отправляются на сервер через `/api/items/`, а изображения загружаются отдельно через endpoint `/api/items/upload-image/`.

При редактировании объявления frontend сначала получает текущие данные вещи, затем отображает их в форме. После изменения данных пользователь сохраняет результат, и обновленная информация отправляется на сервер.

Удаление объявления доступно только владельцу вещи. Перед удалением интерфейс отображает подтверждение, чтобы избежать случайного удаления.

3.4.7 Модуль чатов

Модуль чатов расположен в файле `chats.js`. Он обеспечивает переписку между пользователями сервиса. Пользователь может открыть список диалогов, перейти к конкретному чату и отправлять сообщения.

Для получения списка чатов используется endpoint `/api/chats/conversations/`. Для отправки сообщения используется `/api/chats/conversations/id/messages/`. Сообщения могут содержать как текст, так и изображение.

Для получения новых сообщений без перезагрузки страницы используется WebSocket-соединение по адресу `/ws/chat/chat_id/`. Когда сервер отправляет новое сообщение, frontend добавляет его в окно переписки и обновляет состояние чата.

3.4.8 Модуль уведомлений

Модуль уведомлений реализован в файле `notifications.js`. Он отвечает за получение уведомлений, отображение счетчиков и обработку событий в реальном времени.

Уведомления связаны с основными действиями в системе: созданием бронирования, подтверждением или отменой заявки, оплатой, новым сообщением и новым отзывом. Для получения списка уведомлений используется endpoint `/api/notifications/`.

Для мгновенного получения новых уведомлений используется WebSocket-канал `/ws/notifications/`. При получении уведомления frontend

может показать всплывающее сообщение, обновить счетчик в навигации и при необходимости перенаправить пользователя к нужному разделу.

3.4.9 Модуль договоров и оплаты

Модуль договоров реализован в файле `contracts.js`. Он связан с процессом подтвержденной аренды. После подтверждения бронирования пользователь может открыть договор, просмотреть его данные и выполнить демонстрационное подписание.

Договор привязан к бронированию и содержит информацию о вещи, владельце, арендаторе, сроке аренды и стоимости. В интерфейсе предусмотрена возможность открыть договор и выполнить подписание от имени участника аренды.

Также в модуле реализована демонстрационная оплата через СБП. После подписания договора создается платежная сессия, пользователю отображаются данные оплаты, после чего он может подтвердить демонстрационный платеж. После успешной оплаты статус бронирования обновляется.

3.5 Управление состоянием на клиенте

Управление состоянием в проекте реализовано без отдельной библиотеки. Состояние приложения хранится средствами JavaScript и браузера.

Данные авторизации сохраняются в `localStorage`. К ним относятся access-токен, refresh-токен, идентификатор пользователя, имя пользователя, email и тип учетной записи. Это позволяет приложению восстанавливать состояние пользователя после перезагрузки страницы.

Локальное состояние интерфейса хранится в переменных соответствующих модулей. Например, модуль каталога хранит текущую категорию и выбранные даты фильтрации, модуль чатов - активный диалог и WebSocket-соединение, модуль бронирований - выбранное бронирование и период аренды.

Обработка ошибок централизована в `api.js`. Если сервер возвращает ошибку авторизации, модуль пытается обновить access-токен через refresh-

токен. Если обновить токен невозможно, пользователь выходит из системы. Остальные ошибки отображаются в интерфейсе через сообщения, пустые состояния или toast-уведомления.

3.6 Пользовательский интерфейс и UX

Интерфейс приложения выполнен в едином визуальном стиле. В `styles.css` описаны общие цвета, оформление карточек, кнопок, форм, модальных окон и навигации. За счет единого файла стилей все разделы приложения выглядят согласованно.

Главная страница ориентирована на быстрый просмотр доступных вещей. Пользователь видит категории, фильтры и карточки товаров. Карточка содержит ключевую информацию: изображение, название, цену, рейтинг и основные действия.

Формы авторизации, регистрации, профиля и создания объявления имеют единый стиль. Это упрощает взаимодействие пользователя с системой и делает интерфейс предсказуемым.

Модальные окна используются для действий, которые не требуют перехода на отдельную страницу: просмотр вещи, выбор периода аренды, подтверждение удаления, просмотр договора и демонстрационная оплата.

В интерфейсе используются состояния загрузки, сообщения об ошибках, пустые состояния, счетчики уведомлений и всплывающие toast-сообщения. Это помогает пользователю понимать, что происходит после каждого действия.

Адаптивная верстка позволяет использовать сайт на разных устройствах. Элементы интерфейса перестраиваются в зависимости от ширины экрана, карточки переносятся на новые строки, а формы и модальные окна сохраняют удобный размер.

3.7 Взаимодействие с backend API

Frontend взаимодействует с backend-частью через REST API. Базовый путь API задается в `config.js` и равен `/api`. Все запросы выполняются через

функции модуля `api.js`, что обеспечивает единообразную обработку заголовков, ошибок и авторизации.

Основные endpoint, используемые frontend-частью:

GET `/api/items/` - получение списка вещей.

POST `/api/items/` - создание объявления.

GET `/api/items/id/` - получение подробной информации о вещи.

POST `/api/items/upload-image/` - загрузка изображения вещи.

DELETE `/api/items/images/id/` - удаление изображения.

GET `/api/items/categories/` - получение категорий.

POST `/api/bookings/` - создание бронирования.

GET `/api/bookings/` - получение списка бронирований.

POST `/api/bookings/id/confirm/` - подтверждение бронирования.

POST `/api/bookings/id/cancel/` - отмена бронирования.

POST `/api/token/` - авторизация пользователя.

POST `/api/token/refresh/` - обновление access-токена.

POST `/api/users/register/` - регистрация пользователя.

GET `/api/users/profile/` - получение профиля.

PUT `/api/users/profile/` - обновление профиля.

GET `/api/chats/conversations/` - получение списка диалогов.

POST `/api/chats/conversations/` - создание диалога.

POST `/api/chats/conversations/id/messages/` - отправка сообщения.

GET `/api/notifications/` - получение уведомлений.

GET `/api/contracts/` - получение договоров.

Для обмена событиями в реальном времени используются WebSocket-соединения. Чат подключается к адресу `/ws/chat/{chat_id}/`, а уведомления - к `/ws/notifications/`. Благодаря этому новые сообщения и уведомления появляются в интерфейсе без ручного обновления страницы.

3.8 Требования к запуску клиентской части

Клиентская часть не требует отдельной сборки через Vite, Webpack или npm. Приложение состоит из HTML-файла, CSS-файла и набора JavaScript-модулей, которые подключаются в браузере.

Для корректной работы frontend-части необходимо, чтобы backend-приложение Django было запущено и предоставляло доступ к REST API, WebSocket-маршрутам и медиафайлам. Также необходимо, чтобы сервер корректно отдавал статические файлы клиентской части.

Для работы приложения требуется современный браузер с поддержкой HTML5, CSS3, JavaScript ES6+, Fetch API, LocalStorage и WebSocket. К таким браузерам относятся актуальные версии Google Chrome, Mozilla Firefox, Microsoft Edge и других современных браузеров.

Таким образом, клиентская часть проекта представляет собой модульное одностраничное приложение на HTML, CSS и JavaScript. Разделение кода на отдельные файлы повышает читаемость и сопровождаемость проекта, а взаимодействие с backend через REST API и WebSocket обеспечивает выполнение основных пользовательских сценариев сервиса аренды вещей: просмотр каталога, регистрацию и авторизацию, создание объявлений, бронирование, общение между пользователями, получение уведомлений, работу с договорами и демонстрационную оплату.

4 Рабочий проект

4.1 Спецификация функций и классов клиентской части

Клиентская часть веб-приложения реализована в виде единого HTML-документа (index.html) объёмом 7 626 строк, из которых 1 703 строки составляют описание стилей CSS, 5 610 строк — программный код на JavaScript, а оставшиеся строки — HTML-разметка. Код JavaScript размещён в одном теге <script> и содержит 207 функций, из которых 79 являются асинхронными (async function), что соответствует модульному принципу, описанному в техническом проекте.

Все функции сгруппированы по функциональной принадлежности согласно архитектуре, описанной в разделе 3. Таблица 4.1 содержит спецификацию основных функций клиентской части с указанием их назначения.

Таблица 4.1 – Спецификация основных функций клиентской части

№	Название функции	Описание
1	apiRequest(endpoint, options)	Централизованная отправка HTTP-запросов через Fetch API с автоматическим добавлением JWT-токена в заголовок Authorization и автоматическим обновлением access-токена при получении ошибки 401.
2	refreshAccessToken()	Обновление access-токена с помощью refresh-токена через endpoint /api/token/refresh/. При неудаче выполняет выход из аккаунта.

Продолжение таблицы 4.1

№	Название функции	Описание
3	login(username, password)	Аутентификация пользователя: отправка запроса к /api/token/, декодирование JWT-payload для извлечения user_id, сохранение токенов и данных пользователя в localStorage.
4	register(...)	Регистрация нового пользователя через /api/users/register/ с последующей автоматической авторизацией и поддержкой трёх типов аккаунта: физическое лицо, ИП, юридическое лицо.
5	setTokens(...) / clearTokens()	Сохранение и очистка данных пользователя (токены, user_id, email, user_type, данные профиля) в localStorage.
6	loadProducts(page)	Загрузка постраничного списка вещей с сервера (/api/items/), применение фильтра категории и поискового запроса, отображение карточек товаров.
7	renderProducts(products)	Формирование HTML-разметки карточек вещей с учётом фильтрации по категории и доступности на выбранный период аренды.

Продолжение таблицы 4.1

№	Название функции	Описание
8	<code>openItemPreview(item)</code>	Открытие модального окна с подробной информацией о вещи, её рейтингом, отзывами и занятыми датами аренды.
9	<code>createBookingRequest()</code>	Создание заявки на аренду через <code>/api/bookings/</code> с проверкой дат, расчётом количества дней и обновлением сводки стоимости.
10	<code>updateRentalSummary()</code>	Автоматический расчёт итоговой стоимости аренды: базовая цена, наценка платформы 20%, комиссия бронирования 5%, залог.
11	<code>startBookingPayment()</code>	Инициализация демонстрационного платежа через СБП: обращение к <code>/api/bookings/{id}/start_payment/</code> , отображение QR-данных и суммы в модальном окне.
12	<code>confirmBookingPayment()</code>	Подтверждение демонстрационной оплаты, обновление статуса бронирования на <code>paid</code> .
13	<code>signContractWithDemoEds(...)</code>	Подписание договора аренды демонстрационной ЭЦП через <code>/api/contracts/{id}/sign/</code> с передачей имени подписанта и PIN-кода сертификата.

Продолжение таблицы 4.1

№	Название функции	Описание
14	<code>renderContractModal(contract)</code>	Отображение модального окна договора: номер договора, сводка по сделке, текст договора, кнопка скачивания PDF, состояние подписей обеих сторон.
15	<code>showChatPage(conversationId)</code>	Открытие страницы переписки с загрузкой истории сообщений и установкой WebSocket-соединения по адресу <code>/ws/chat/{id}/?token=...</code>
16	<code>sendChatMessageWithOptionalImage(optionalImage)</code>	Отправка сообщения в чат через REST API с поддержкой прикреплённого изображения (multipart/form-data). При активном WebSocket-соединении сообщение отображается через сокет.
17	<code>connectNotificationsSocket()</code>	Подключение к WebSocket-каналу <code>/ws/notifications/</code> для получения системных уведомлений в реальном времени с автоматическим переподключением при разрыве.

Продолжение таблицы 4.1

№	Название функции	Описание
18	<code>showMyItemsPage()</code>	Отображение страницы объявлений и статистики пользователя: количество объявлений, количество аренд, чистая прибыль, списки входящих и исходящих бронирований.
19	<code>createItemWithImages(...)</code>	Создание объявления через <code>/api/items/</code> с последующей загрузкой изображений через <code>/api/items/upload-image/</code> .
20	<code>validateInnClient(val, type)</code>	Клиентская валидация ИНН с проверкой контрольных цифр для физических лиц (12 знаков) и юридических лиц (10 знаков) без обращения к серверу.
21	<code>submitReview(...)</code>	Отправка отзыва по завершённомому или оплаченному бронированию через <code>/api/bookings/{id}/review/</code> с оценкой и текстовым комментарием.

4.2 Реализация ключевых фрагментов клиентской части

4.2.1 Централизованный модуль взаимодействия с API

Все HTTP-запросы к серверной части выполняются через единую асинхронную функцию `apiRequest`, расположенную в блоке `<script>` файла

index.html. Функция автоматически добавляет JWT-токен авторизации в заголовки каждого запроса и обеспечивает прозрачное обновление access-токена при получении ошибки 401 (Unauthorized). Фрагмент реализации представлен на рисунке 4.1.

```
1  async function apiRequest(endpoint, options = {}) {
2    const url = `${API_BASE}${endpoint}`;
3    const headers = { ...options.headers };
4    const token = getAccessToken();
5    if (token) headers['Authorization'] = `Bearer ${token}`;
6    if (!(options.body instanceof FormData)) {
7      headers['Content-Type'] = 'application/json';
8    }
9    let response = await fetch(url, { ...options, headers });
10   if (response.status === 401 && !options._retry) {
11     const refreshed = await refreshAccessToken();
12     if (refreshed) {
13       options._retry = true;
14       headers['Authorization'] = `Bearer ${getAccessToken()}`;
15       response = await fetch(url, { ...options, headers });
16     }
17   }
18   return response;
19 }
```

Рисунок 4.1 – Функция apiRequest

Данный подход позволяет избежать дублирования кода авторизации в каждом модуле и обеспечивает единообразную обработку ошибок. Если сервер возвращает код 401 и запрос выполняется впервые (флаг `_retry` не установлен), функция обращается к endpoint `/api/token/refresh/` для получения нового access-токена и повторяет исходный запрос.

4.2.2 Модуль бронирования

Модуль бронирования реализован через функции `updateRentalSummary` и `createBookingRequest`. Функция `updateRentalSummary` вычисляет итоговую стоимость аренды на стороне клиента по следующей формуле: к базовой цене арендодателя добавляется наценка платформы в размере 20%, затем к полученной сумме прибавляется комиссия бронирования 5% и залог. Результат

отображается в модальном окне в реальном времени по мере выбора дат. Фрагмент расчёта стоимости представлен на рисунке 4.2.

```
1  const basePricePerDay = Number(currentRentalItem?.price_per_day || 0);
2  const platformMarkupRate = 0.20;
3  const pricePerDayWithMarkup =
4  Math.round(basePricePerDay * (1 + platformMarkupRate) * 100) / 100;
5  const rentAmountWithMarkup = pricePerDayWithMarkup * days;
6  const bookingFeeRate = 0.05;
7  const bookingFee =
8  Math.round(rentAmountWithMarkup * bookingFeeRate * 100) / 100;
9  const deposit = Number(currentRentalItem?.deposit ?? 0);
10 const total = rentAmountWithMarkup + bookingFee + deposit;
```

Рисунок 4.2 – Функция updateRentalSummary

После выбора дат пользователь нажимает кнопку отправки заявки, и функция createBookingRequest передаёт данные бронирования на сервер. В запросе содержатся идентификатор объявления, даты начала и окончания аренды, а также выбранный пункт самовывоза. При успешном создании бронирования интерфейс обновляется без перезагрузки страницы.

4.2.3 Модуль договоров и демонстрационного подписания ЭЦП

Модуль договоров обеспечивает формирование, просмотр и подписание договора аренды. Функция renderContractModal отображает модальное окно с текстом договора, сводкой по сделке и полями для ввода данных подписанта. Подписание выполняется через функцию signContractWithDemoEds, которая передаёт на сервер имя подписанта и PIN демонстрационного сертификата. Фрагмент функции подписания представлен на рисунке 4.3.

```

1  async function signContractWithDemoEds(contractId, signerName,
2  certificatePin) {
3      try {
4          const response = await apiRequest(
5              `/contracts/${contractId}/sign/`,
6              {
7                  method: 'POST',
8                  body: JSON.stringify({
9                      signer_name: signerName,
10                     certificate_pin: certificatePin,
11                 })
12             }
13         );
14         if (!response.ok) {
15             const message = await readApiError(response,
16                 'Не удалось подписать договор. ');
17             return { success: false, data: { error: message } };
18         }
19         const data = await response.json().catch(() => ({}));
20         return { success: response.ok, data };
21     } catch (error) {
22         return { success: false,
23             data: { error: 'Ошибка подключения.' } };
24     }
25 }

```

Рисунок 4.3 – Функция renderContractModal

После того как обе стороны выполнили подписание, сервер отмечает договор как полностью подписанный и открывает возможность оплаты. Интерфейс отображает состояние подписи каждой из сторон, включая дату и имя подписанта.

4.2.4 Модуль чатов

Для обеспечения обмена сообщениями в реальном времени в модуле чатов используется WebSocket-соединение. При открытии страницы чата функция showChatPage устанавливает подключение к серверу по адресу /ws/chat/conversationId/?token=..., где в качестве параметра передаётся JWT access-токен. Фрагмент кода установки соединения представлен на рисунке 4.4.

```

1  const token = getAccessToken();
2  const wsUrl = `${WS_BASE}/chat/${conversationId}?token=${token}`;
3  if (wsConnection) wsConnection.close();
4  wsConnection = new WebSocket(wsUrl);
5
6  wsConnection.onmessage = (event) => {
7    const data = JSON.parse(event.data);
8    if (data.event === 'new_message' && data.message) {
9      const message = data.message;
10     messagesContainer.insertAdjacentHTML(
11       'beforeend',
12       renderSingleMessage(
13         message,
14         message.sender_id == localStorage.getItem('user_id')
15       )
16     );
17     messagesContainer.scrollTop =
18       messagesContainer.scrollHeight;
19     refreshNavCounters();
20   }
21 };

```

Рисунок 4.4 – Функция showChatPage

При получении сервером нового события `new_message` фронтенд добавляет сообщение в окно переписки и прокручивает его к последнему сообщению без перезагрузки страницы. При закрытии чата соединение разрывается корректно с помощью функции `teardownActiveChatState`.

4.2.5 Модуль уведомлений

Системные уведомления реализованы с применением двух механизмов. Основным является WebSocket-канал `/ws/notifications/`, который обеспечивает мгновенную доставку уведомлений об операциях в системе. В качестве резервного механизма предусмотрен периодический опрос REST-endpoint `/api/notifications/` с интервалом 10 секунд при отсутствии активного WebSocket-соединения. При разрыве WebSocket-соединения функция `scheduleNotificationSocketReconnect` автоматически планирует повторное подключение. Уведомления отображаются в виде всплывающих toast-сообщений в правом нижнем углу экрана, счётчик непрочитанных обновляется в навигационной панели.

4.2.6 Модуль валидации форм на стороне клиента

В форме регистрации реализована клиентская валидация реквизитов пользователя, зеркалирующая логику серверных валидаторов. Для ИНН выполняется проверка длины (10 цифр — юридическое лицо, 12 цифр — физическое лицо или ИП) и проверка контрольных цифр по установленному алгоритму. Для ОГРНИП проверяется соответствие формату из 15 цифр и корректность контрольной цифры по остатку от деления на 13. Для КПП проверяется формат из 9 символов. Такой подход позволяет выявлять ошибки ввода до отправки запроса к серверу, снижая нагрузку на API и обеспечивая мгновенную обратную связь пользователю.

Дополнительно в форме регистрации реализовано подтверждение адреса электронной почты: до завершения регистрации пользователь обязан ввести шестизначный код, отправленный на указанный email. Кнопка завершения регистрации остаётся заблокированной до подтверждения email и принятия условий использования площадки.

4.3 Тестирование клиентской части

4.3.1 Модульное тестирование

Для проверки корректности серверной логики, с которой взаимодействует клиентская часть, были разработаны модульные тесты на языке Python с использованием встроенного фреймворка Django TestCase. Тесты охватывают модели данных, сериализаторы и бизнес-логику, обеспечивая корректность ответов API, получаемых фронтендом.

Класс `BookingModelTest` проверяет корректность расчёта стоимости бронирования, невозможность самоаренды (когда арендатор совпадает с владельцем вещи), обнаружение пересекающихся периодов бронирования, неизменность суммы при изменении цены объявления после создания брони, а также корректность работы с отзывами. Фрагмент теста представлен на рисунке 4.5.

```

1  class BookingModelTest(TestCase):
2  def setUp(self):
3      self.owner = User.objects.create_user(
4      username='owner', password='12345',
5      email='owner@test.com', profile_completed=True)
6      self.renter = User.objects.create_user(
7      username='renter', password='12345',
8      email='renter@test.com', profile_completed=True)
9      self.item = Item.objects.create(
10     title='Drill', price_per_day=1000,
11     owner=self.owner, status='available')
12
13     def test_booking_total_price_calculation(self):
14         booking = Booking.objects.create(
15         item=self.item, renter=self.renter,
16         start_date=date(2025, 6, 1),
17         end_date=date(2025, 6, 4))
18         self.assertEqual(booking.total_price, 3000)
19
20     def test_cannot_book_own_item(self):
21         with self.assertRaises(Exception):
22             Booking.objects.create(
23             item=self.item, renter=self.owner,
24             start_date=date(2025, 6, 1),
25             end_date=date(2025, 6, 3))
26
27     def test_booking_date_overlap(self):
28         Booking.objects.create(
29         item=self.item, renter=self.renter,
30         start_date=date(2025, 6, 1),
31         end_date=date(2025, 6, 5))
32         with self.assertRaises(Exception):
33             Booking.objects.create(
34             item=self.item, renter=self.renter,
35             start_date=date(2025, 6, 3),
36             end_date=date(2025, 6, 7))

```

Рисунок 4.5 – Тестирование

Результаты тестов выше представлены на рисунке 4.6.

```

1  Found 5 test(s).
2  Creating test database for alias "default...
3  System check identified no issues (0 silenced).
4  .....
5  -----
6  Ran 5 tests in 0.312s
7
8  OK

```

Рисунок 4.6 – Результаты тестирования

4.3.2 Системное тестирование

Системное тестирование клиентской части проводилось путём ручной проверки всех пользовательских сценариев, определённых в разделе 2.4. Тестирование выполнялось в браузерах Google Chrome 124 и Mozilla Firefox 125 на операционных системах Windows 10 и Windows 11. Результаты тестирования представлены в таблице 4.2.

Таблица 4.2 – Результаты системного тестирования

№	Проверяемый сценарий	Ожидаемый результат	Фактический результат
1	Регистрация с подтверждением email и выбором типа пользователя	Создание учётной записи, автоматический вход	Соответствует ожидаемому
2	Авторизация с последующим сохранением сессии при перезагрузке страницы	Пользователь остаётся авторизованным	Соответствует ожидаемому
3	Восстановление пароля через код на email	Новый пароль принят, сессия сброшена	Соответствует ожидаемому
4	Фильтрация каталога по категории и доступности дат	Отображаются только подходящие объявления	Соответствует ожидаемому
5	Создание объявления с загрузкой изображений	Объявление создано, изображения привязаны	Соответствует ожидаемому

Продолжение таблицы 4.2

№	Проверяемый сценарий	Ожидаемый результат	Фактический результат
6	Создание бронирования с автоматическим расчётом стоимости	Заявка создана, стоимость рассчитана корректно	Соответствует ожидаемому
7	Подтверждение и отклонение входящей заявки на аренду	Статус бронирования изменён, уведомление отправлено	Соответствует ожидаемому
8	Подписание договора аренды обеими сторонами	Договор отмечен как подписанный	Соответствует ожидаемому
9	Демонстрационная оплата через механизм СБП	Статус оплаты изменён на paid	Соответствует ожидаемому
10	Обмен сообщениями в чате с отправкой изображения	Сообщение доставлено в реальном времени через WebSocket	Соответствует ожидаемому
11	Создание отзыва по завершённому бронированию	Отзыв сохранён, рейтинг пересчитан	Соответствует ожидаемому
12	Просмотр статистики в личном кабинете	Отображены корректные данные о доходах и бронированиях	Соответствует ожидаемому

Продолжение таблицы 4.2

№	Проверяемый сценарий	Ожидаемый результат	Фактический результат
13	Отображение интерфейса на мобильном устройстве (ширина 375 px)	Все элементы корректно перестраиваются под узкий экран	Соответствует ожидаемому
14	Валидация ИНН и ОГРНИП на стороне клиента	Некорректные значения отклоняются до отправки запроса	Соответствует ожидаемому

По результатам системного тестирования все проверяемые сценарии дали результат, соответствующий ожидаемому. Ошибок в работе пользовательского интерфейса, некорректного отображения данных и сбоев при взаимодействии с серверным API выявлено не было.

4.4 Сборка и развёртывание клиентской части

Клиентская часть не требует этапа сборки и не использует систем управления пакетами (npm, Webpack, Vite). Приложение представляет собой один HTML-файл index.html, включающий встроенные стили CSS и блок JavaScript, что обеспечивает простоту развёртывания и отсутствие зависимостей от инструментов сборки.

Для корректной работы клиентской части необходимо выполнение следующих условий:

1. Серверная часть на Django запущена и предоставляет доступ к REST API и WebSocket-маршрутам.
2. Веб-сервер Nginx настроен для раздачи статических файлов клиентской части и проксирования запросов к Django.

3. Браузер пользователя поддерживает стандарты HTML5, CSS3, JavaScript ES6+, Fetch API, LocalStorage и WebSocket (Google Chrome версии 90+, Mozilla Firefox версии 88+, Microsoft Edge версии 90+).

Развёртывание всех компонентов серверной части (Django, PostgreSQL, Redis, Nginx) выполняется с использованием Docker и Docker Compose, что обеспечивает идентичность окружений на этапах разработки и эксплуатации. Файл index.html помещается в директорию статических файлов, откуда Nginx отдаёт его напрямую. Базовый адрес API задаётся в константе API_BASE внутри JavaScript-блока файла index.html и при необходимости меняется перед развёртыванием на конкретный домен.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была разработана и протестирована клиентская часть онлайн-площадки для аренды вещей, реализующая полный цикл взаимодействия пользователя с сервисом: от регистрации и просмотра каталога до подписания договора аренды и демонстрационной оплаты. В процессе работы были решены все задачи, поставленные в техническом задании:

1. Проведён анализ предметной области онлайн-аренды и существующих решений. Рассмотрены классификация площадок по типу товаров, модели взаимодействия и типу арендодателей. Выявлены функциональные преимущества разрабатываемой системы по сравнению с аналогами Rentomania, Авито и Арендую.ру.

2. Разработана концептуальная модель клиентской части. Определены основные экраны, компоненты и сценарии взаимодействия пользователя с системой. Спроектированы варианты использования, охватывающий все ключевые операции сервиса.

3. Спроектирован пользовательский интерфейс, обеспечивающий взаимодействие с серверной частью посредством REST API. Определены все endpoint, используемые клиентской частью, и реализована централизованная функция apiRequest с автоматическим обновлением JWT-токена.

4. Разработана и протестирована клиентская часть веб-приложения. Реализованы модули авторизации с подтверждением email, каталога вещей с фильтрацией, бронирования с автоматическим расчётом стоимости с учётом наценки платформы и комиссии, встроенного чата с WebSocket, договоров аренды с демонстрационной ЭЦП, системы отзывов и уведомлений в реальном времени.

Клиентская часть реализована в виде одностраничного приложения (SPA) на стандартных веб-технологиях HTML5, CSS3 и Vanilla JavaScript без использования фронтенд-фреймворков. Файл index.html содержит 7 626 строк кода, из которых 5 610 строк составляет программная логика JavaScript

(207 функций). Для выбора дат аренды использована библиотека flatpickr, для хранения сессионных данных — localStorage.

Взаимодействие с серверной частью организовано через Django REST API для стандартных операций и через WebSocket-соединения (Django Channels, Redis) для чата и уведомлений в реальном времени. Реализован механизм автоматического переподключения WebSocket при разрыве связи и резервный опрос REST-endpoint уведомлений.

Проведено системное тестирование по 14 пользовательским сценариям в браузерах Google Chrome и Mozilla Firefox на настольных компьютерах и мобильных устройствах. Все сценарии дали результат, соответствующий ожидаемому.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Бретт, М. PHP и MySQL. Исчерпывающее руководство / М. Бретт. – Санкт-Петербург : Питер, 2016. – 544 с. – ISBN 978-5-496-01049-8. – Текст : непосредственный.
2. Веру, Л. Секреты CSS. Идеальные решения ежедневных задач / Л. Веру. – Санкт-Петербург : Питер, 2016. – 336 с. – ISBN 978-5-496-02082-4. – Текст : непосредственный.
3. Титтел, Э. HTML5 и CSS3 для чайников / Э. Титтел, К. Минник. – Москва : Вильямс, 2016 – 400 с. – ISBN 978-1-118-65720-1. – Текст : непосредственный.
4. Макнейл П. Веб-дизайн. Книга идей веб-разработчика / П. Макнейл. — СПб.: Питер, 2017. — 480 с. — ISBN 978-5-496-00705-4. — Текст : непосредственный.
5. Леонова В. Н. Основы продвижения сайтов : методические указания / В. Н. Леонова. — Гомель : Гомельский государственный технический университет имени П. О. Сухого, 2026. — Текст : непосредственный.
6. Макфедрис П. Веб-дизайн с нуля: HTML + CSS на практике / П. Макфедрис ; пер. с англ. — 2-е изд. — Санкт-Петербург : Питер, 2025. — 416 с. — ISBN 978-5-4461-4128-9. — Текст : непосредственный.
7. Бачурина Т. В. Веб-дизайн и реклама / Т. В. Бачурина, О. М. Кунцевич. — Минск : Белорусский государственный университет культуры и искусств, 2025. — Текст : непосредственный.
8. Нильсен Я. Веб-дизайн: книга Якоба Нильсена / Я. Нильсен. — М.: Символ, 2015. — 512 с. — ISBN 978-5-93286-004-5. — Текст : непосредственный.
9. Петин В. А. Веб-дизайн и юзабилити. Практическое руководство / В. А. Петин. — 2-е изд., перераб. и доп. — Москва : ДМК Пресс, 2025. — 352 с. — ISBN 978-5-93700-412-1. — Текст : непосредственный.

10. Морозов Д. А. Frontend-разработка. Реактивный подход и современные фреймворки / Д. А. Морозов. — Москва : Солон-Пресс, 2025. — 304 с. — ISBN 978-5-91359-563-8. — Текст : непосредственный.
11. Гурли, Д. HTTP. Подробное руководство / Д. Гурли, Б. Тотти. — Санкт-Петербург : Символ-Плюс, 2013. — 656 с. — ISBN 978-5-93286-209-7. — Текст : непосредственный.
12. Макфарланд, Д. Большая книга CSS / Д. Макфарланд. — Санкт-Петербург : Питер, 2012. — 560 с. — ISBN 978-5-496-02080-0. — Текст : непосредственный.
13. Немцова Т. И., Казанкова Т. В., Шнякин А. В. Компьютерная графика и web-дизайн. Учебное пособие. — М.: Форум, 2023. — 400 с. — ISBN 978-5-16-021098-8. Текст : непосредственный.
14. Голдстейн, А. HTML5 и CSS3 для всех / А. Голдстейн, Л. Лазарис, Э. Уэйл. — Москва : Вильямс, 2012. — 368 с. — ISBN 978-5-699-57580-0. — Текст : непосредственный.
15. Дэкетт, Д. HTML и CSS. Разработка и создание веб-сайтов / Д. Дэкетт. — Москва : Эксмо, 2014. — 480 с. — ISBN 978-5-699-64193-2. — Текст : непосредственный.
16. Хавербеке, М. Выразительный JavaScript / М. Хавербеке. — Санкт-Петербург : Питер, 2019. — 472 с. — ISBN 978-5-4461-1211-1. — Текст : непосредственный.
17. Крокфорд, Д. JavaScript: сильные стороны / Д. Крокфорд. — Санкт-Петербург : Питер, 2012. — 176 с. — ISBN 978-5-459-00788-6. — Текст : непосредственный.
18. Закас Н.С. Основы JavaScript для профессиональных веб-разработчиков / Н. Закас; пер. с англ. — 4-е изд. — Санкт-Петербург: Вильямс, 2020. — 1216 с. — ISBN 978-5-8459-2394-9.
19. Флэнаган Д. JavaScript. Подробное руководство / Д. Флэнаган; пер. с англ. — 7-е изд. — Санкт-Петербург: БХВ-Петербург, 2021. — 1168 с. — ISBN 978-5-9775-4825-9.

20. Прохоренок Н.А., Дронов В.А. JavaScript и Node.js для веб-разработчиков / Н.А. Прохоренок, В.А. Дронов. — Санкт-Петербург: БХВ Петербург, 2022. — 592 с. — ISBN 978-5-9775-6591-9.

21. Макфарланд Д. Изучаем HTML, XHTML и CSS / Д. Макфарланд; пер. с англ. — 2-е изд. — Санкт-Петербург: Питер, 2020. — 656 с. — ISBN 978-5-4461-1244-8.

22. Выдрин Д.К., Грибов А.Ю., Анохин А.Ю. HTML5 и CSS3. Веб-разработка по стандартам нового поколения / Д.К. Выдрин, А.Ю. Грибов, А.Ю. Анохин. — 2-е изд., перераб. и доп. — Москва: ДМК Пресс, 2021. — 416 с. — ISBN 978-5-97060-957-9.

23. Российская Федерация. Законы. О защите прав потребителей: Закон РФ от 7 февраля 1992г N 2300-1 — Текст : электронный // КонсультантПлюс: справочно-правовая система. URL:https://www.consultant.ru/document/cons_doc_LAW_305/ (дата обращения: 04.05.2026).

24. Российская Федерация. Законы. Глава 34 «Аренда». Гражданский кодекс РФ — Текст : электронный // КонсультантПлюс: справочно-правовая система. — URL: https://www.consultant.ru/document/cons_doc_LAW_9027/cac65a05697011caaa0eff7d476d867137f1f92a/ (дата обращения: 04.05.2026).

25. Российская Федерация. Законы. О проведении эксперимента по установлению специального налогового режима ”Налог на профессиональный доход”: Федеральный закон от 27.11.2018 № 422-ФЗ — Текст : электронный // КонсультантПлюс: справочно-правовая система. URL:https://www.consultant.ru/document/cons_doc_LAW_311977/ (дата обращения: 04.05.2026).

26. Российская Федерация. Законы. О персональных данных: Федеральный закон от 27.07.2006 № 152-ФЗ — Текст : электронный // КонсультантПлюс: справочно-правовая система. URL:https://www.consultant.ru/document/cons_doc_LAW_61801/ (дата обращения: 05.05.2026).

27. ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем.— Москва : Стандартинформ, 2010. — Текст : непосредственный.

28. ГОСТ 34.602-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы.— Москва : Стандартинформ, 2020. — Текст : непосредственный.

29. ГОСТ 34.201-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Виды, комплектность и обозначение документов при создании автоматизированных систем.— Москва : Стандартинформ, 2020. — Текст : непосредственный.

30. Алексеев А. П. Введение в Web-дизайн. Учебное пособие. — М.: Солон-Пресс, 2021. — 184 с. — ISBN 978-5-91359-355-9. Текст : непосредственный.

31. Полуэктова Н. Р. Разработка веб-приложений. — М.: Юрайт, 2024. — 205 с. — ISBN 978-5-534-18644-4. Текст : непосредственный.

32. Петроченков А., Новиков Е. Идеальный Landing Page. Создаем продающие веб-страницы. — СПб.: Питер, 2017. — 320 с. — ISBN 978-5-4461-0292-1. Текст : непосредственный.

33. Кириченко А. В. Справочник HTML. Кратко, быстро, под рукой. — М.: Наука и техника, 2023. — 288 с. — ISBN 978-5-94387-275-4. Текст : непосредственный.

34. Хрусталеv А. А. Дубовик Е. В. Справочник CSS3. Кратко, быстро, под рукой. — М.: Наука и техника, 2021. — 304 с. — ISBN 978-5-94387-279-2. Текст : непосредственный.

35. Кангин В. В. Интернет. Языки HTML и JavaScript. — М.: ТНТ, 2021. — 488 с. — ISBN 978-5-94178-140-9. Текст : непосредственный.

36. Стефанов С. React. Быстрый старт, 2-е изд. / С. Стефанов. — Санкт-Петербург : Питер, 2023. — 304 с. — ISBN 978-5-4461-2115-1. — Текст : непосредственный.

37. Заяц А. М. Проектирование и разработка WEB-приложений. Введение в frontend и backend разработку на JavaScript и node.js : учебное пособие для СПО / А. М. Заяц, Н. П. Васильев. — 3-е изд., стер. — Санкт-Петербург : Лань, 2023. — 120 с. — ISBN 978-5-507-46023-4. — Текст : непосредственный.

38. Сысолетин Е. Г. Разработка интернет-приложений : учебное пособие для СПО / Е. Г. Сысолетин, С. Д. Ростунцев, Л. Г. Доросинский. — Москва : Юрайт, 2024. — 80 с. — ISBN 978-5-534-19603-0. — Текст : непосредственный.

39. Коллер Ф. М. Полный стек. Веб-разработка / Ф. М. Коллер ; пер. с англ. — Москва : Эксмо, 2024. — 432 с. — ISBN 978-5-04-192845-6. — Текст : непосредственный.

40. Маллах Э. Введение в веб-разработку с HTML, CSS и JavaScript / Э. Маллах ; пер. с англ. — Москва : ДМК Пресс, 2025. — 414 с. — ISBN 978-5-93700-312-4. — Текст : непосредственный.

41. Никсон Р. HTML5 и CSS3. Мастер-класс / Р. Никсон ; пер. с англ. — Санкт-Петербург : БХВ-Петербург, 2024. — 464 с. — ISBN 978-5-9775-1929-8. — Текст : непосредственный.

42. Янцев В. В. Разработка web-страниц на HTML, CSS и JavaScript : учебное пособие для вузов / В. В. Янцев. — Санкт-Петербург : Лань, 2024. — 148 с. — ISBN 978-5-507-49407-1. — Текст : непосредственный.

43. Тузовский А. Ф. Проектирование и разработка web-приложений : учебник для вузов / А. Ф. Тузовский. — Москва : Юрайт, 2025. — 219 с. — ISBN 978-5-534-16300-1. — Текст : непосредственный.

44. Баркович А. А. Веб-проектирование : учебное пособие / А. А. Баркович, Т. А. Филимонова. — Москва : ИНФРА-М, 2025. — 231 с. — ISBN 978-5-16-019399-1. — Текст : непосредственный.

45. Фролов А. Б. Основы web-дизайна. Разработка, создание и сопровождение web-сайтов : учебное пособие для СПО / А. Б. Фролов, И. А. Нагаева, И. А. Кузнецов. — Саратов : Профобразование, 2024. — 144 с. — ISBN 978-5-4488-2231-5. — Текст : непосредственный.

46. Жемчужников Д. Г. Веб-дизайн. Уровень 2 : учебное пособие / Д. Г. Жемчужников. — Москва : Просвещение, 2025. — 112 с. — ISBN 978-5-09-087118-1. — Текст : непосредственный.

47. Кузнецова Л. В. Современные веб-технологии : учебное пособие / Л. В. Кузнецова. — Москва : Интернет-Университет Информационных Технологий, 2024. — 187 с. — ISBN 978-5-4497-2457-1. — Текст : непосредственный.

48. Булгакова И. А. Разработка и прототипирование веб-сайтов и интерфейсов онлайн : учебное пособие для вузов / И. А. Булгакова. — Москва : Дашков и К°, 2024. — 215 с. — ISBN 978-5-394-06214-8. — Текст : непосредственный.

49. Илларионова М. В. WEB-дизайн : учебное пособие / М. В. Илларионова. — Санкт-Петербург : Санкт-Петербургский политехнический университет Петра Великого, 2024. — 140 с. — DOI 10.18720/SPBPU/5/tr24-98. — Текст : непосредственный.

50. Медведев М. А. Web технологии в бизнесе : учебное пособие / М. А. Медведев, М. А. Медведева ; под ред. D. B. Berg. — Екатеринбург : Издательство Уральского университета, 2024. — 160 с. — ISBN 978-5-7996-3906-8. — Текст : непосредственный.

ПРИЛОЖЕНИЕ А

Представление графического материала

Графический материал, выполненный на отдельных листах, изображен на рисунках А.1–А.10.

ВКРБ 2206230.09.03.04.26.002		1													
Сведения о ВКРБ Минобрнауки России Юго-Западный государственный университет Кафедра программной инженерии ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА ПО ПРОГРАММЕ БАКАЛАВРИАТА «Бизнес-проект «Онлайн-площадка для сдачи вещей в аренду». Разработка клиентской части» Руководитель ВКРБ к.т.н, доцент Петрик Елена Анатольевна Автор ВКРБ студент группы ПО-216 Баранов Никита Александрович															
ВКРБ 2206230.09.03.04.26.002		Лит. Мисс. Золото													
<table border="1" style="width: 100%; border-collapse: collapse;"> <tr> <td style="font-size: x-small;">Фамилия И. О. Имя</td> <td style="font-size: x-small;">Дата</td> </tr> <tr> <td style="font-size: x-small;">Автор работы</td> <td style="font-size: x-small;">Баранов Н.А.</td> </tr> <tr> <td style="font-size: x-small;">Руководитель</td> <td style="font-size: x-small;">Петрик Е.А.</td> </tr> <tr> <td style="font-size: x-small;">Нормоконтроль</td> <td style="font-size: x-small;">Чаплыгин А.А.</td> </tr> <tr> <td style="font-size: x-small;"> </td> <td style="font-size: x-small;"> </td> </tr> <tr> <td style="font-size: x-small;"> </td> <td style="font-size: x-small;"> </td> </tr> </table>	Фамилия И. О. Имя	Дата	Автор работы	Баранов Н.А.	Руководитель	Петрик Е.А.	Нормоконтроль	Чаплыгин А.А.					Сведения о ВКРБ Выпускная квалификационная работа бакалавра		Лист 1 Из листов 9
Фамилия И. О. Имя	Дата														
Автор работы	Баранов Н.А.														
Руководитель	Петрик Е.А.														
Нормоконтроль	Чаплыгин А.А.														
ЮЗГУ ПО-216															

Рисунок А.1 – Сведения о ВКРБ

Цель и задачи разработки

Основной задачей выпускной квалификационной работы является разработка и тестирование веб-приложения, предоставляющего возможность арендодателям публиковать объявления о сдаче вещей в аренду, а арендаторам - находить, бронировать и оплачивать такие вещи с обеспечением базовых гарантий безопасности сделки.

Задачами данной разработки являются:

- изучение и анализ предметной области;
- проектирование интерфейса веб-приложения;
- проектирование базы данных;
- проектирование модуля взаимодействия с пользователем;
- проектирование API веб-приложения.

ВКРБ 2206230.09.03.04.26.002			
Автор работы	Фамилия И. О.	Имя	Фамилия
Руководитель	Баданов И. А.	Цель и задачи	Лит.
Рецензент	Петрик Е. А.	разработки	Место
Методический	Чайкин А. А.		Листов
		Выпускная квалификационная	10
		работа бакалавра	ЮЗГУ ПО-216

Рисунок А.2 – Цель и задачи разработки

ВКРБ 2206230.09.03.04.26.002

3

Сравнение существующих онлайн-площадок
для аренды вещей

Параметр	Rentomania	Авито	Арендую.ру	Проектируемая онлайн площадка
Размещение объявлений арендодателем	+	+	+	+
Поиск и фильтрация товаров	+	+	+	+
Онлайн- бронирование	+	—	—	+
Встроенная онлайн- оплата	+	+	—	+
Система рейтинга и отзывов	+	+	—	+
Чат между арендатором и арендодателем	+	+	—	+
Адаптивная вёрстка для мобильных устройств	+	+	—	+
Верификация пользователей	+	—	—	+
Панель аналитики для арендодателя	—	—	—	+
Понятный интерфейс	—	+	—	+
Уведомления о статусе аренды	+	—	—	+

				ВКРБ 2206230.09.03.04.26.002			
				Лит.МестоВсего			
				Сравнение существующих онлайн-площадок для аренды вещей			
				Лист 3 Листов 10			
				Выпускная квалификационная работа бакалавра			
				ЮЗГУ ПО-2023			

Рисунок А.3 – Сравнение конкурентов

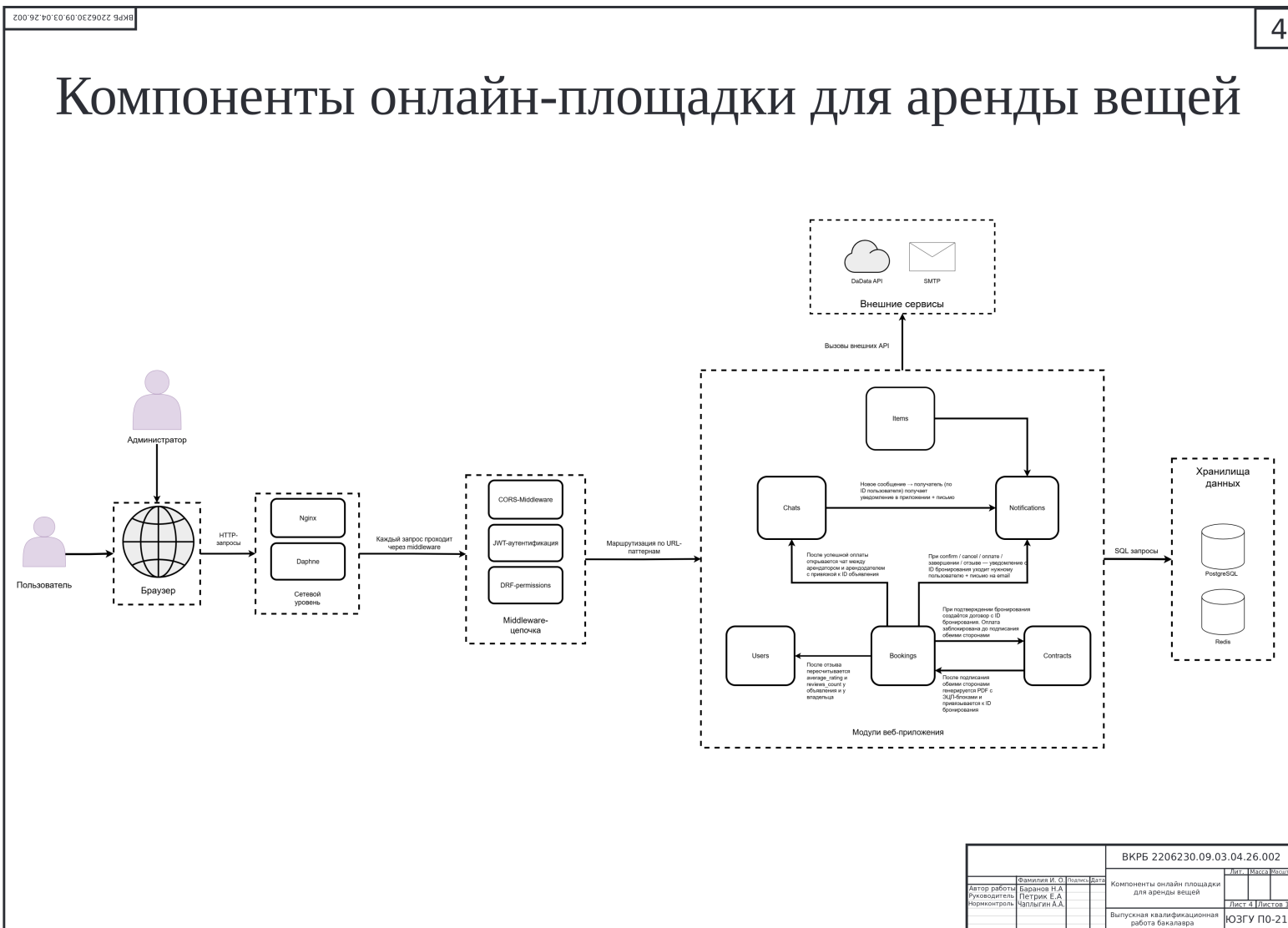


Рисунок А.4 – Компоненты онлайн-площадки для аренды вещей



Рисунок А.5 – Диаграмма прецедентов

ВКРБ 2206230.09.03.04.26.002

6

Макет страницы входа в профиль

Кнопка назад

Имя пользователя

Пароль

ВОЙТИ

ВКРБ 2206230.09.03.04.26.002		
Фамилия И. О.	Фамилия	Имя
Автор работы	Баранов Н.А.	
Руководитель	Петрик Е.А.	
Нормоконтроль	Чеплыгин А.А.	
Макет страницы входа в профиль		Лист 1 из 10
Выпускная квалификационная работа бакалавра		ЮЗГУ ПО-216

Рисунок А.6 – Макет страницы входа в профиль

ВКРБ 2206230.09.03.04.26.002

7

Макет страницы чата с пользователем

Наименование чата

К чатам

Прикрепить изображение

Поле ввода сообщения

Отправить

				ВКРБ 2206230.09.03.04.26.002
Автор работы	Фамилия И. О.	Имя	Фамилия	Макет страницы чата с пользователем
Руководитель	Баданов В.А.			
Нормоконтроль	Петрик Е.А.			Лист 5 из 10
	Чаплыгин А.А.			Выпускная квалификационная работа бакалавра
				ЮЗГУ ПО-216

Рисунок А.7 – Макет страницы чата с пользователем



Рисунок А.8 – Макет страницы просмотра объявлений

ВКРБ 2206230.09.03.04.26.002
9

Макет страницы с информацией об объявлении

Наименование

ФОТО

Цена

Арендовать

Написать арендодателю

Свободные даты

Занятые даты

Отзывы

			ВКРБ 2206230.09.03.04.26.002	
			Макет страницы с информацией об объявлении	
Автор работы	Баранов В.А.	Фамилия И.О.	Имя	Место
Рецензент	Петрик Е.А.	Фамилия И.О.	Имя	Место
Нормоконтроль	Чайкинг А.А.	Фамилия И.О.	Имя	Место
			Выпускная квалификационная работа бакалавра	
			ЮЗГУ ПО-216	

Рисунок А.9 – Макет страницы информации об объявлении

Заключение

В процессе работы были решены все задачи, поставленные в техническом задании:

1. Проведён анализ предметной области онлайн-аренды и существующих решений.

Рассмотрены классификация площадок по типу товаров, модели взаимодействия и типу арендодателей. Выявлены функциональные преимущества разрабатываемой системы по сравнению с аналогами Rentomania, Авито и Арендую.ру.

2. Разработана концептуальная модель клиентской части. Определены основные экраны, компоненты и сценарии взаимодействия пользователя с системой.

Спроектированы варианты использования, охватывающий все ключевые операции сервиса.

3. Спроектирован пользовательский интерфейс, обеспечивающий взаимодействие с серверной частью посредством REST API. Определены все endpoint, используемые клиентской частью, и реализована централизованная функция apiRequest с автоматическим обновлением JWT-токена.

4. Разработана и протестирована клиентская часть веб-приложения.

Реализованы модули авторизации с подтверждением email, каталога вещей с фильтрацией, бронирования с автоматическим расчётом стоимости с учётом наценки платформы и комиссии, встроенного чата с WebSocket, договоров аренды с демонстрационной ЭЦП, системы отзывов и уведомлений в реальном времени.

ВКРБ 2206230.09.03.04.26.002			
Фамилия И. О.	Фамилия И. О.	Дата	Лит. Мисс. Золото
Автор работы	Баданов И. А.		
Руководитель	Петрик Е. А.		
Корректор	Чайкин А. А.		
Заключение			Лист 10 из 10
Выпускная квалификационная работа бакалавра			ЮЗГУ ПО-216

Рисунок А.10 – Заключение

ПРИЛОЖЕНИЕ Б

Фрагменты исходного кода программы

index.html

```
1 <!DOCTYPE html>
2 <html lang="ru">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Аренда вещей</title>
7   <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/flatpickr/dist/
      flatpickr.min.css">
8   <link rel="stylesheet" href="styles.css">
9 </head>
10 <body>
11   <div id="app"></div>
12   <div class="site-toast-stack" id="siteToastStack"></div>
13
14   <div class="item-preview-overlay" id="itemPreviewOverlay">
15     <div class="item-preview-modal">
16       <div class="item-preview-header">
17         <div>
18           <div class="item-preview-title" id="itemPreviewTitle">
19             Объявление</div>
20           <div class="item-preview-subtitle" id="
21             itemPreviewSubtitle">Подробная информация о товаре</
22             div>
23           </div>
24           <button class="rent-close-btn" id="itemPreviewCloseBtn" type="
25             button">x</button>
26         </div>
27         <div id="itemPreviewContent"></div>
28       </div>
29     </div>
30
31   <div class="rent-modal-overlay" id="rentModalOverlay">
32     <div class="rent-modal">
33       <div class="rent-modal-header">
34         <div>
35           <div class="rent-modal-title" id="rentModalTitle">
36             Оформление аренды</div>
37           <div class="rent-modal-subtitle">Выберите даты аренды,
38             подпишите договор и только после этого переходите к
39             оплате</div>
40         </div>
41         <button class="rent-close-btn" id="rentModalCloseBtn" type="
42           button">x</button>
43       </div>
44
45       <div id="rentModalMessage"></div>
46
47       <div class="form-group">
48         <label for="rentStartDate">Дата начала аренды</label>
```

```

41         <input type="text" id="rentStartDate" placeholder="Выберите
42             дату начала">
43     </div>
44     <div class="form-group">
45         <label for="rentEndDate">Дата окончания аренды</label>
46         <input type="text" id="rentEndDate" placeholder="Выберите
47             дату окончания">
48     </div>
49     <div class="pickup-section">
50         <label>☐ Место самовывоза — Почта России (г. Курск)</label>
51         <div class="pickup-search-wrap">
52             <svg width="16" height="16" viewBox="0 0 24 24" fill="
53                 none" stroke="currentColor" stroke-width="2.2" stroke-
54                 linecap="round" stroke-linejoin="round"><circle cx="11
55                 " cy="11" r="8"/><line x1="21" y1="21" x2="16.65" y2="
56                 16.65"/></svg>
57             <input type="text" id="pickupSearchInput" class="pickup-
58                 search" placeholder="Поиск по адресу или индексу..."
59                 autocomplete="off">
60         </div>
61         <div class="pickup-list" id="pickupList"></div>
62         <div class="pickup-selected-info" id="pickupSelectedInfo"></
63             div>
64     </div>
65     <div class="rent-summary">
66         <div class="rent-summary-row">
67             <span>Товар</span>
68             <span id="rentSummaryItem">—</span>
69         </div>
70         <div class="rent-summary-row">
71             <span>Цена за день</span>
72             <span id="rentSummaryPrice">—</span>
73         </div>
74         <div class="rent-summary-row">
75             <span>Количество дней</span>
76             <span id="rentSummaryDays">0</span>
77         </div>
78         <div class="rent-summary-row" style="color:#6b8b9c;font-size
79             :0.88rem;">
80             <span>Комиссия платформы (20%)</span>
81             <span id="rentSummaryMarkup">—</span>
82         </div>
83         <div class="rent-summary-row">
84             <span>Сумма аренды</span>
85             <span id="rentSummaryTotal">0 ₺</span>
86         </div>
87         <div class="rent-summary-row" style="color:#6b8b9c;font-size
88             :0.88rem;">
89             <span>Сервисный сбор (5%)</span>
90             <span id="rentSummaryFee">—</span>
91         </div>

```

```

84     <div class="rent-summary-row">
85         <span>Залог (возвращается)</span>
86         <span id="rentSummaryDeposit">—</span>
87     </div>
88     <div class="rent-summary-row total">
89         <span>Итого к оплате</span>
90         <span id="rentSummaryGrandTotal">0 ₺</span>
91     </div>
92     <div class="rent-summary-row">
93         <span>Статус брони</span>
94         <span id="rentSummaryStatus">Не создана</span>
95     </div>
96     <div class="rent-summary-row">
97         <span>Статус залога</span>
98         <span id="rentSummaryDepositStatus">—</span>
99     </div>
100 </div>
101
102 <div class="rent-payment-actions">
103     <button class="btn-outline" id="createBookingBtn" type="
104         button">Отправить заявку</button>
105     <button class="btn-primary" id="openContractBtn" type="button
106         ">Подписать документ с помощью ЭЦГ</button>
107     <button class="btn-primary is-hidden" id="startPaymentBtn"
108         type="button">Перейти к оплате по СБГ</button>
109 </div>
110
111 <div class="payment-box" id="paymentBox">
112     <div class="payment-row">
113         <strong>Банк</strong>
114         <span id="paymentBank">—</span>
115     </div>
116     <div class="payment-row">
117         <strong>Получатель</strong>
118         <span id="paymentRecipient">—</span>
119     </div>
120     <div class="payment-row">
121         <strong>Телефон</strong>
122         <span id="paymentPhone">—</span>
123     </div>
124     <div class="payment-row">
125         <strong>Сумма</strong>
126         <span id="paymentAmount">—</span>
127     </div>
128     <div class="payment-row">
129         <strong>Статус оплаты</strong>
130         <span id="paymentStatus">—</span>
131     </div>
132     <div class="payment-row">
133         <strong>Истекает</strong>
134         <span id="paymentExpires">—</span>
135     </div>

```

```

134         <button class="btn-primary payment-link" id="paymentLinkBtn"
135             type="button" disabled>Открыть демо-оплату СБП</button>
136
137         <div class="payment-code" id="paymentQrPayload"></div>
138     </div>
139
140     <div class="rent-note">
141         Это учебный демо-сценарий: после подтверждения брони
142         арендатор сначала подписывает договор демо-ЭЦП, а затем
143         открывает муляж оплаты по СБП.
144     </div>
145
146     <div class="sbp-demo-overlay" id="sbpDemoOverlay">
147         <div class="sbp-demo-modal">
148             <div class="sbp-demo-header">
149                 <div class="sbp-demo-title">Оплата через СБП</div>
150                 <div class="sbp-demo-subtitle">Демо-экран оплаты.</div>
151             </div>
152             <div class="sbp-demo-body">
153                 <div class="sbp-demo-badge">СБП Demo</div>
154                 <div class="sbp-demo-grid">
155                     <div class="sbp-demo-row">
156                         <strong>Банк</strong>
157                         <span id="sbpDemoBank">—</span>
158                     </div>
159                     <div class="sbp-demo-row">
160                         <strong>Получатель</strong>
161                         <span id="sbpDemoRecipient">—</span>
162                     </div>
163                     <div class="sbp-demo-row">
164                         <strong>Телефон</strong>
165                         <span id="sbpDemoPhone">—</span>
166                     </div>
167                     <div class="sbp-demo-row">
168                         <strong>Сумма</strong>
169                         <span id="sbpDemoAmount">—</span>
170                     </div>
171                 </div>
172                 <div class="sbp-demo-qr" id="sbpDemoQrPayload"></div>
173                 <div class="sbp-demo-message" id="sbpDemoMessage"></div>
174                 <div class="sbp-demo-actions">
175                     <button class="btn-primary" id="sbpDemoPayBtn" type="
176                         button">Оплатить</button>
177                     <button class="btn-outline" id="sbpDemoCancelBtn" type="
178                         button">Отмена</button>
179                 </div>
180             </div>
181         </div>
182     </div>
183
184 <!-- ===== Модальные окна правовых документов ===== -->

```

```

183 <div class="legal-modal-overlay" id="termsModalOverlay">
184   <div class="legal-modal">
185     <div class="legal-modal-header">
186       <div class="legal-modal-title">Условия использования площадки
187       </div>
188       <button class="legal-modal-close" id="termsModalClose" type="
189       button">x</button>
190     </div>
191     <div class="legal-modal-body">
192       <h3>1. Общие положения</h3>
193       <p>Настоящие Условия использования (далее — Условия)
194       регулируют отношения между онлайн-площадкой Arenda (далее
195       — Платформа) и пользователями, осуществляющими аренду
196       движимого имущества через Платформу.</p>
197       <p>Использование Платформы означает полное и безоговорочное
198       принятие настоящих Условий.</p>
199       <h3>2. Предмет услуги</h3>
200       <p>Платформа предоставляет возможность физическим лицам,
201       индивидуальным предпринимателям и юридическим лицам
202       размещать объявления об аренде движимого имущества и
203       заключать договоры проката.</p>
204       <h3>3. Комиссия и расчёты</h3>
205       <ol>
206         <li>Платформа взимает комиссию 20% от цены аренды за день
207         , устанавливаемой арендодателем. Комиссия включается в
208         цену, отображаемую арендатору.</li>
209         <li>При оформлении бронирования взимается дополнительный
210         сервисный сбор 5% от итоговой суммы аренды.</li>
211         <li>Залог устанавливается арендодателем и возвращается
212         арендатору после подтверждения возврата имущества в
213         надлежащем состоянии.</li>
214       </ol>
215       <h3>4. Права и обязанности пользователей</h3>
216       <ol>
217         <li>Арендодатель обязуется предоставлять достоверную
218         информацию об имуществе и обеспечивать его передачу в
219         исправном состоянии.</li>
220         <li>Арендатор обязуется использовать имущество по
221         назначению и возвращать его в состоянии не хуже
222         исходного.</li>
223         <li>Обе стороны обязуются подписывать договор аренды с
224         использованием демонстрационной ЭЦП до начала оплаты.<
225         /li>
226       </ol>
227       <h3>5. Ответственность</h3>
228       <p>Платформа выступает посредником и не несёт ответственности
229       за ненадлежащее исполнение договора аренды его сторонами.
230       Споры между сторонами разрешаются в соответствии с
231       действующим законодательством РФ.</p>
232       <h3>6. Заключительные положения</h3>
233       <p>Платформа оставляет за собой право изменять настоящие
234       Условия с уведомлением пользователей. Продолжение
235       использования Платформы после изменений означает согласие
236       с обновлёнными Условиями.</p>

```

```

211     </div>
212     <div class="legal-modal-footer">
213         <button class="btn-primary" id="termsModalAccept" type="
                button">Понятно</button>
214     </div>
215 </div>
216 </div>
217
218 <div class="legal-modal-overlay" id="privacyModalOverlay">
219     <div class="legal-modal">
220         <div class="legal-modal-header">
221             <div class="legal-modal-title">Соглашение на обработку
                персональных данных</div>
222             <button class="legal-modal-close" id="privacyModalClose" type
                ="button">×</button>
223         </div>
224         <div class="legal-modal-body">
225             <h3>1. Оператор персональных данных</h3>
226             <p>Оператором персональных данных является онлайн-площадка
                Arenda, действующая в соответствии с Федеральным законом
                № 152-ФЗ «О персональных данных».</p>
227             <h3>2. Состав персональных данных</h3>
228             <p>В ходе регистрации и использования Платформы могут
                обрабатываться следующие персональные данные:</p>
229             <ul>
230                 <li>Фамилия, имя, отчество</li>
231                 <li>Адрес электронной почты и номер телефона</li>
232                 <li>Серия и номер паспорта (для физических лиц)</li>
233                 <li>ИНН, КПП, ОГРН/ОГРНИП (для ИП и юридических лиц)</li>
234                 <li>Сведения о заключённых договорах аренды</li>
235             </ul>
236             <h3>3. Цели обработки</h3>
237             <ol>
238                 <li>Идентификация пользователей и обеспечение работы
                Платформы.</li>
239                 <li>Заключение и исполнение договоров аренды между
                пользователями.</li>
240                 <li>Формирование договоров с использованием
                демонстрационной ЭЦП.</li>
241                 <li>Направление уведомлений, связанных с бронированиями и
                платежами.</li>
242             </ol>
243             <h3>4. Передача данных третьим лицам</h3>
244             <p>Персональные данные не передаются третьим лицам без
                согласия субъекта, за исключением случаев, предусмотренных
                законодательством РФ.</p>
245             <h3>5. Хранение и защита</h3>
246             <p>Данные хранятся на защищённых серверах. Платформа
                принимает технические и организационные меры для защиты
                персональных данных от несанкционированного доступа.</p>
247             <h3>6. Права субъекта данных</h3>
248             <p>Вы вправе запросить доступ к своим данным, их исправление
                или удаление, направив запрос через форму обратной связи
                на Платформе.</p>

```

```

249         <h3>7. Срок действия согласия</h3>
250         <p>Настоящее согласие действует с момента регистрации до
                удаления аккаунта пользователем или истечения 5 лет с
                последней активности.</p>
251     </div>
252     <div class="legal-modal-footer">
253         <button class="btn-primary" id="privacyModalAccept" type="
                button">Понятно</button>
254     </div>
255 </div>
256 </div>
257
258 <!-- ===== Модальное окно передачи / возврата / СБП-возврата / отказа
                ===== -->
259 <div class="handover-overlay" id="handoverOverlay">
260     <div class="handover-modal">
261         <div class="handover-modal-title" id="handoverModalTitle">
                Подтверждение</div>
262         <div class="handover-modal-sub" id="handoverModalSub"></div>
263         <!-- Поле загрузки фото (показывается если нужно) -->
264         <div id="handoverPhotoSection">
265             <label class="handover-photo-label" for="handoverPhotoInput">
266                 <svg width="20" height="20" fill="none" stroke="
                        currentColor" stroke-width="2" viewBox="0 0 24 24"><
                        rect x="3" y="3" width="18" height="18" rx="3"/><
                        circle cx="8.5" cy="8.5" r="1.5"/><path d="M21 15l-5-5
                        L5 21"/></svg>
267                 <span id="handoverPhotoLabelText">Прикрепить фото товара
                        (обязательно)</span>
268             </label>
269             <input type="file" id="handoverPhotoInput" accept="image/*"
                    style="display:none">
270             <img class="handover-photo-preview" id="handoverPhotoPreview"
                    alt="Предпросмотр фото">
271             <div class="handover-photo-name" id="handoverPhotoName"></div>
272         </div>
273         <!-- Текстовое поле (для возврата/отказа) -->
274         <div id="handoverReasonSection" style="display:none">
275             <textarea class="handover-reason-input" id="
                    handoverReasonInput" placeholder="Опишите причину..."></
                    textarea>
276         </div>
277         <div class="handover-actions">
278             <button class="btn-primary" id="handoverConfirmBtn" type="
                    button">Подтвердить</button>
279             <button class="btn-outline" id="handoverCancelBtn" type="
                    button">Отмена</button>
280         </div>
281     </div>
282 </div>
283
284 <div class="contract-modal-overlay" id="contractModalOverlay">
285     <div class="contract-modal">

```

```

286         <div id="contractModalContent"></div>
287     </div>
288 </div>
289
290 <div class="support-chat-panel" id="supportChatPanel" aria-hidden="true">
291     <div class="support-chat-header">
292         <div class="support-chat-header-top">
293             <div class="support-chat-title">Помощник аренды</div>
294             <button class="support-chat-close" id="supportChatCloseBtn"
                type="button" aria-label="Закрыть чат">x</button>
295         </div>
296         <div class="support-chat-subtitle">Быстрые ответы по ключевым
                функциям сайта.</div>
297     </div>
298     <div class="support-chat-body">
299         <div class="support-chat-messages" id="supportChatMessages"></div>
300         <div class="support-questions-label">Частые вопросы</div>
301         <div class="support-questions" id="supportQuestions"></div>
302         <div class="support-chat-note">Выберите готовый вопрос выше, и
                бот покажет пошаговую инструкцию.</div>
303     </div>
304 </div>
305
306 <button class="support-chat-toggle" id="supportChatToggleBtn" type="
    button" aria-label="Открыть чат поддержки">
307     <svg viewBox="0 0 24 24" fill="none" aria-hidden="true">
308         <path d="M7 9.5h10M7 13h7m-7 6 2.9-2.2c
                .38-.29.57-.43.79-.53.19-.09.39-.15.6-.18.24-.03.47-.03.93-.03
                H16.8c1.68 0 2.52 0 3.16-.33a3 3 0 0 0 1.31-1.31c
                .33-.64.33-1.48.33-3.16V8.2c0-1.68 0-2.52-.33-3.16a3 3 0 0
                0-1.31-1.31C19.32 3.4 18.48 3.4 16.8 3.4H7.2c-1.68 0-2.52
                0-3.16.33A3 3 0 0 0 2.73 5.04C2.4 5.68 2.4 6.52 2.4 8.2v4.6c0
                1.68 0 2.52.33 3.16a3 3 0 0 0 1.31 1.31c.46.24 1.03.31 1.96.33
                Z" stroke="currentColor" stroke-width="1.8" stroke-linecap="
                round" stroke-linejoin="round"/>
309     </svg>
310 </button>
311
312 <script src="utils.js"></script>
313 <script src="auth.js"></script>
314 <script src="items.js"></script>
315 <script src="bookings.js"></script>
316 <script src="chat.js"></script>
317 <script src="notifications.js"></script>
318 <script src="support.js"></script>
319 </body>
320 </html>

```

auth.js

```

1 function getAccessToken() { return localStorage.getItem('access_token'); }
2
3
4 function getRefreshToken() { return localStorage.getItem('refresh_token'); }

```

```

5
6
7 function setTokens(access, refresh, userId, username, email, userType,
  profileData) {
8     localStorage.setItem('access_token', access);
9     if (refresh) localStorage.setItem('refresh_token', refresh);
10    if (userId) localStorage.setItem('user_id', userId);
11    if (username) localStorage.setItem('username', username);
12    if (email) localStorage.setItem('user_email', email);
13    if (userType) localStorage.setItem('user_type', userType);
14
15    if (profileData) {
16        localStorage.setItem('is_superuser', profileData.is_superuser
            ? '1' : '0');
17        if (profileData.full_name) localStorage.setItem('full_name',
            profileData.full_name);
18        if (profileData.entrepreneur_name) localStorage.setItem('
            entrepreneur_name', profileData.entrepreneur_name);
19        if (profileData.company_name) localStorage.setItem('
            company_name', profileData.company_name);
20        if (profileData.passport_series) localStorage.setItem('
            passport_series', profileData.passport_series);
21        if (profileData.passport_number) localStorage.setItem('
            passport_number', profileData.passport_number);
22        if (profileData.inn) localStorage.setItem('inn', profileData.
            inn);
23        if (profileData.ogrnip) localStorage.setItem('ogrnip',
            profileData.ogrnip);
24        if (profileData.kpp) localStorage.setItem('kpp', profileData.
            kpp);
25    }
26 }
27
28
29
30 function clearTokens() {
31     localStorage.removeItem('access_token');
32     localStorage.removeItem('refresh_token');
33     localStorage.removeItem('username');
34     localStorage.removeItem('user_id');
35     localStorage.removeItem('user_email');
36     localStorage.removeItem('user_type');
37     localStorage.removeItem('full_name');
38     localStorage.removeItem('entrepreneur_name');
39     localStorage.removeItem('company_name');
40     localStorage.removeItem('passport_series');
41     localStorage.removeItem('passport_number');
42     localStorage.removeItem('inn');
43     localStorage.removeItem('ogrnip');
44     localStorage.removeItem('kpp');
45     localStorage.removeItem('is_superuser');
46 }
47
48

```

```

49
50 function isAuthenticated() { return !!getAccessToken(); }
51
52     async
53
54 function refreshAccessToken() {
55     const refresh = getRefreshToken();
56     if (!refresh) return false;
57     try {
58         const response = await fetch(`${API_BASE}/token/refresh/`, {
59             method: 'POST',
60             headers: { 'Content-Type': 'application/json' },
61             body: JSON.stringify({ refresh })
62         });
63         if (response.ok) {
64             const data = await response.json();
65             localStorage.setItem('access_token', data.access);
66             return true;
67         }
68     } catch (e) {}
69     return false;
70 }
71
72     async
73
74 function attachPasswordToggle(buttonId, inputId) {
75     const button = document.getElementById(buttonId);
76     const input = document.getElementById(inputId);
77     if (!button || !input) return;
78
79     button.addEventListener('click', () => {
80         const isPassword = input.type === 'password';
81         input.type = isPassword ? 'text' : 'password';
82         button.textContent = isPassword ? '□' : '□';
83         button.setAttribute('aria-label', isPassword ? 'Скрыть пароль'
84             : 'Показать пароль');
85     });
86
87
88
89 function getSavedProfile() {
90     return {
91         username: localStorage.getItem('username'),
92         email: localStorage.getItem('user_email'),
93         user_type: localStorage.getItem('user_type') || 'individual',
94         full_name: localStorage.getItem('full_name') || '',
95         entrepreneur_name: localStorage.getItem('entrepreneur_name')
96             || '',
97         company_name: localStorage.getItem('company_name') || '',
98         passport_series: localStorage.getItem('passport_series') || '
',
99         passport_number: localStorage.getItem('passport_number') || '
',

```

```

99         inn: localStorage.getItem('inn') || '',
100         ogrnip: localStorage.getItem('ogrnip') || '',
101         kpp: localStorage.getItem('kpp') || ''
102     };
103 }
104
105     async
106
107     function getUserProfile() {
108         try {
109             const response = await apiRequest('/users/profile/');
110             if (response.ok) {
111                 const data = await response.json();
112                 if (data.username) localStorage.setItem('username', data.username);
113                 if (data.email) localStorage.setItem('user_email', data.email);
114                 if (data.user_type) localStorage.setItem('user_type', data.user_type);
115                 if (data.full_name !== undefined) localStorage.setItem('full_name', data.full_name || '');
116                 if (data.entrepreneur_name !== undefined) localStorage.setItem('entrepreneur_name', data.entrepreneur_name || '');
117                 if (data.company_name !== undefined) localStorage.setItem('company_name', data.company_name || '');
118                 if (data.passport_series) localStorage.setItem('passport_series', data.passport_series);
119                 if (data.passport_number) localStorage.setItem('passport_number', data.passport_number);
120                 if (data.inn) localStorage.setItem('inn', data.inn);
121                 if (data.ogrnip) localStorage.setItem('ogrnip', data.ogrnip);
122                 if (data.kpp) localStorage.setItem('kpp', data.kpp);
123                 return data;
124             }
125         } catch (e) {}
126         return getSavedProfile();
127     }
128
129     async
130
131     function updateProfile(profileData) {
132         try {
133             const response = await apiRequest('/users/profile/', {
134                 method: 'PUT',
135                 body: JSON.stringify(profileData)
136             });
137             if (response.ok) {
138                 const data = await response.json();
139                 if (data.username) localStorage.setItem('username', data.username);
140                 if (data.email) localStorage.setItem('user_email', data.email);

```

```

141         if (data.full_name !== undefined) localStorage.setItem('
            full_name', data.full_name || '');
142         if (data.entrepreneur_name !== undefined) localStorage.
            setItem('entrepreneur_name', data.entrepreneur_name ||
                '');
143         if (data.company_name !== undefined) localStorage.setItem
            ('company_name', data.company_name || '');
144         if (data.passport_series) localStorage.setItem('
            passport_series', data.passport_series);
145         if (data.passport_number) localStorage.setItem('
            passport_number', data.passport_number);
146         if (data.inn) localStorage.setItem('inn', data.inn);
147         if (data.ogrnip) localStorage.setItem('ogrnip', data.
            ogrnip);
148         if (data.kpp) localStorage.setItem('kpp', data.kpp);
149         return { success: true, data };
150     } else {
151         const detail = await readApiError(response, 'Не удалось
            обновить профиль. ');
152         return { success: false, error: { detail } };
153     }
154 } catch (error) {
155     return { success: false, error: { detail: 'Ошибка подключения
        ' } } };
156 }
157 }
158
159 async
160
161 function showProfileValidationError(msg) {
162     const msgDiv = document.getElementById('profileMessage');
163     msgDiv.innerHTML = `<div class="error-message-box
        validation-error-highlight"> По результатам проверки
        вы ввели некорректные данные:<br><strong>${escapeHtml
            (msg)}</strong></div>`;
164     msgDiv.scrollIntoView({ behavior: 'smooth', block: '
        center' });
165 }
166
167 const result = await updateProfile(updateData);
168 const msgDiv = document.getElementById('profileMessage');
169 if (result.success) {
170     msgDiv.innerHTML = '<div class="success-message">
        Профиль успешно обновлён!</div>';
171     localStorage.setItem('username', updateData.username);
172     localStorage.setItem('user_email', updateData.email);
173     setTimeout(() => msgDiv.innerHTML = '', 2000);
174     updateNavBar();
175 } else {
176     const errorMessage = extractApiErrorMessage(result.error,
        'Не удалось обновить профиль. ');
177     msgDiv.innerHTML = `<div class="error-message-box"> ${
        escapeHtml(errorMessage)}</div>`;
178 }

```

```

179     });
180
181     document.getElementById('passwordForm').addEventListener('submit'
182     , async (e) => {
183         e.preventDefault();
184         const msgDiv = document.getElementById('profileMessage');
185         const newPassword = document.getElementById('newPassword').
            value;
186         const confirmPassword = document.getElementById('
            confirmPassword').value;
187
188         if (newPassword !== confirmPassword) {
189             msgDiv.innerHTML = '<div class="error-message-box">
                Пароли не совпадают</div>';
190             return;
191         }
192
193         if (newPassword.length < 4) {
194             msgDiv.innerHTML = '<div class="error-message-box">
                Пароль должен быть не менее 4 символов</div>';
195             return;
196         }
197
198         const result = await changePassword(document.getElementById('
                oldPassword').value, newPassword);
199         if (result.success) {
200             msgDiv.innerHTML = '<div class="success-message">
                Пароль успешно изменён!</div>';
201             document.getElementById('oldPassword').value = '';
202             document.getElementById('newPassword').value = '';
203             document.getElementById('confirmPassword').value = '';
204             setTimeout(() => msgDiv.innerHTML = '', 2000);
205         } else {
206             const errorMessage = extractApiErrorMessage(result.error,
                'Не удалось изменить пароль. ');
207             msgDiv.innerHTML = `<div class="error-message-box"> ${
                escapeHtml(errorMessage)}</div>`;
208         }
209     });
210
211     document.getElementById('backToMainFromProfile')?.
        addEventListener('click', () => showMainPage());
212
213     // Кнопка «Заполнить по ИНН» в профиле
214     if (userType === 'entrepreneur') {
215         document.getElementById('profileFillByInnBtn')?.
            addEventListener('click', () => fillByInn('profile', '
                entrepreneur'));
216     } else if (userType === 'legal') {
217         document.getElementById('profileFillByInnBtn')?.
            addEventListener('click', () => fillByInn('profile', '
                legal'));
218     }

```

```

219
220     async
221
222     function renderRegisterExtraFields() {
223         const userType = document.getElementById('regUserType')?.
            value || 'individual';
224         const box = document.getElementById('registerExtraFields');
225         if (!box) return;
226
227         if (userType === 'individual') {
228             box.innerHTML = `
229                 <div class="user-type-info">
230                     <h4> Профиль физического лица</h4>
231                     <div class="input-group"><label>ФИО *</label><
                        input type="text" id="regFullName" maxlength="
232                         255" placeholder="Иванов Иван Иванович"></div>
233                     <div class="input-group"><label>Серия паспорта</
                        label><input type="text" id="regPassportSeries
234                         " placeholder="Серия паспорта (4 цифры)"></div
                        >
235                     <div class="input-group"><label>Номер паспорта</
                        label><input type="text" id="regPassportNumber
236                         " placeholder="Номер паспорта (6 цифр) "></div
                        >
237                     <div class="input-group"><label>ИНН</label><input
238                         type="text" id="regInn" placeholder="ИНН (12
239                         цифр)"></div>
240                 </div>
241             `;
242         } else if (userType === 'entrepreneur') {
243             box.innerHTML = `
244                 <div class="user-type-info">
245                     <h4> Профиль ИП</h4>
246                     <div class="inn-autofill-row">
247                         <div class="input-group" style="flex:1;margin
248                             -bottom:0">
249                             <label>ИНН <span style="font-weight:
250                                 normal;font-size:12px;color:#6b7280"
251                                 >(12 цифр)</span></label>
252                             <input type="text" id="regInn" maxlength=
253                                 "12" placeholder="Введите ИНН для
254                                 автозаполнения">
255                         </div>
256                         <button type="button" class="btn-fill-inn" id
257                             ="regFillByInnBtn"> Заполнить по ИНН</
258                             button>
259                     </div>
260                     <div id="regInnStatus" class="inn-status-msg"
261                         style="display:none"></div>
262                     <div class="input-group"><label>Наименование ИП</
263                         label><input type="text" id="
264                         regEntrepreneurName" maxlength="255"
265                         placeholder="ИП Иванов Иван Иванович"></div>

```

```

250         <div class="input-group"><label>ИНН</label><input
                type="text" id="regInn" placeholder="ИНН (12
                цифр)"></div>
251         <div class="input-group"><label>ОГРНИП</label><
                input type="text" id="regOgrnip" placeholder="
                ОГРНИП (15 цифр)"></div>
252     </div>
253     `;
254     document.getElementById('regFillByInnBtn').
        addEventListener('click', () => fillByInn('reg', '
        entrepreneur'));
255 } else {
256     box.innerHTML = `
257         <div class="user-type-info">
258             <h4>□ Профиль юридического лица</h4>
259             <div class="inn-autofill-row">
260                 <div class="input-group" style="flex:1;margin
                -bottom:0">
261                     <label>ИНН <span style="font-weight:
                        normal;font-size:12px;color:#6b7280"
                        >(10 цифр)</span></label>
262                     <input type="text" id="regInn" maxlength=
                        "10" placeholder="Введите ИНН для
                        автозаполнения">
263                 </div>
264                 <button type="button" class="btn-fill-inn" id
                        ="regFillByInnBtn" >□ Заполнить по ИНН/<
                        button>
265             </div>
266             <div id="regInnStatus" class="inn-status-msg"
                style="display:none"></div>
267             <div class="input-group"><label>Наименование
                организации</label><input type="text" id="
                regCompanyName" maxlength="255" placeholder="
                ООО Ромашка"></div>
268             <div class="input-group"><label>ИНН</label><input
                type="text" id="regInn" placeholder="ИНН (10
                цифр)"></div>
269             <div class="input-group"><label>КПП</label><input
                type="text" id="regKpp" placeholder="КПП (9
                цифр)"></div>
270         </div>
271         `;
272         document.getElementById('regFillByInnBtn').
            addEventListener('click', () => fillByInn('reg', '
            legal'));
273     }
274 }
275
276 renderRegisterExtraFields();
277 document.getElementById('regUserType')?.addEventListener('change'
    , renderRegisterExtraFields);
278 attachPasswordToggle('toggleRegPassword', 'regPassword');
279 attachPasswordToggle('toggleRegPassword2', 'regPassword2');

```

```

280
281 // Логика кнопки «Получить код»
282 let emailCodeSent = false;
283 let sendCodeCooldown = null;
284 document.getElementById('sendEmailCodeBtn')?.addEventListener('
    click', async () => {
285     const email = document.getElementById('regEmail').value.trim
        ();
286     if (!email) { alert('❑ Введите email'); return; }
287     const btn = document.getElementById('sendEmailCodeBtn');
288     const status = document.getElementById('emailCodeStatus');
289     btn.disabled = true;
290     btn.textContent = 'Отправка...';
291     try {
292         const res = await fetch(`${API_BASE}/users/send-email-
            code/`, {
293             method: 'POST',
294             headers: { 'Content-Type': 'application/json' },
295             body: JSON.stringify({ email })
296         });
297         const data = await res.json();
298         if (res.ok) {
299             emailCodeSent = true;
300             document.getElementById('emailCodeGroup').style.
                display = '';
301             status.style.display = '';
302             status.style.color = 'green';
303             status.textContent = '❑ Код отправлен. Проверьте
                почту (действует 15 минут).';
304             // Cooldown 60 секунд
305             let seconds = 60;
306             btn.textContent = `Отправить повторно (${seconds}с)`;
307             sendCodeCooldown = setInterval(() => {
308                 seconds--;
309                 if (seconds <= 0) {
310                     clearInterval(sendCodeCooldown);
311                     btn.disabled = false;
312                     btn.textContent = 'Отправить повторно';
313                 } else {
314                     btn.textContent = `Отправить повторно (${
                        seconds}с)`;
315                 }
316             }, 1000);
317         } else {
318             status.style.display = '';
319             status.style.color = 'red';
320             status.textContent = '❑ ' + (data.error || data.
                detail || 'Ошибка отправки кода');
321             btn.disabled = false;
322             btn.textContent = 'Получить код';
323         }
324     } catch (e) {
325         status.style.display = '';
326         status.style.color = 'red';

```

```

327         status.textContent = '❑ Ошибка подключения к серверу';
328         btn.disabled = false;
329         btn.textContent = 'Получить код';
330     }
331 });
332
333 document.getElementById('registerForm').addEventListener('submit'
, async (e) => {
334     e.preventDefault();
335     const password = document.getElementById('regPassword').value
;
336     const password2 = document.getElementById('regPassword2').
value;
337     if (password !== password2) {
338         alert('❑ Пароли не совпадают');
339         return;
340     }
341
342     const emailCode = (document.getElementById('regEmailCode')?.
value || '').trim();
343     if (!emailCodeSent) {
344         alert('❑ Сначала получите код подтверждения email —
нажмите кнопку «Получить код».');
345         return;
346     }
347     if (!emailCode) {
348         alert('❑ Введите код подтверждения, который был
отправлен на ваш email.');
```

349 return;

350 }

351

352 const userType = document.getElementById('regUserType').value

353 ;

354 const extraProfileData = {

355 full_name: normalizeWhitespace(document.getElementById('

356 regFullName')?.value || ''),

357 entrepreneur_name: normalizeWhitespace(document.

358 getElementById('regEntrepreneurName')?.value || ''),

359 company_name: normalizeWhitespace(document.getElementById

360 ('regCompanyName')?.value || ''),

361 passport_series: document.getElementById('

362 regPassportSeries')?.value || '',

363 passport_number: document.getElementById('

364 regPassportNumber')?.value || '',

365 inn: document.getElementById('regInn')?.value || '',

366 ogrnip: document.getElementById('regOgrnip')?.value || ''

367 ,

368 kpp: document.getElementById('regKpp')?.value || ''

369 };

370

371 if (userType === 'individual') {

372 if (!extraProfileData.full_name) {

373 alert('❑ Укажите ФИО.');

374 return;

```

368     }
369     if (!isValidRussianFullName(extraProfileData.full_name))
370     {
371         alert('❑ ФИО должно содержать минимум 3 слова через
372             пробелы и только русские буквы. ');
373         return;
374     }
375     if (extraProfileData.entrepreneur_name.length > 255) {
376         showRegValidationError('❑ Наименование ИП слишком
377             длинное. Максимум 255 символов. ');
378         return;
379     }
380     if (extraProfileData.company_name.length > 255) {
381         showRegValidationError('❑ Наименование организации
382             слишком длинное. Максимум 255 символов. ');
383         return;
384     }
385     // Клиентская проверка реквизитов по типу пользователя
386     if (userType === 'entrepreneur') {
387         const innVal = extraProfileData.inn;
388         const ogrnipVal = extraProfileData.ogrnip;
389         if (innVal) {
390             const innErr = validateInnClient(innVal, '
391                 entrepreneur');
392             if (innErr) { showRegValidationError('❑ ' + innErr);
393                 return; }
394         }
395         if (ogrnipVal) {
396             const ogrnipErr = validateOgrnipClient(ogrnipVal);
397             if (ogrnipErr) { showRegValidationError('❑ ' +
398                 ogrnipErr); return; }
399         }
400     } else if (userType === 'legal') {
401         const innVal = extraProfileData.inn;
402         const kppVal = extraProfileData.kpp;
403         if (innVal) {
404             const innErr = validateInnClient(innVal, 'legal');
405             if (innErr) { showRegValidationError('❑ ' + innErr);
406                 return; }
407         }
408         if (kppVal) {
409             const kppErr = validateKppClient(kppVal);
410             if (kppErr) { showRegValidationError('❑ ' + kppErr);
411                 return; }
412         }
413     }
414 }
415
416 function openForgotPasswordModal() {
417     if (forgotPasswordModal) {

```

```

413         forgotPasswordModal.style.display = 'flex';
414         // Очищаем поля при открытии
415         const emailInput = document.getElementById('forgotEmail')
416         ;
417         const codeInput = document.getElementById('forgotCode');
418         const passwordInput = document.getElementById('
419             forgotNewPassword');
420         if (emailInput) emailInput.value = '';
421         if (codeInput) codeInput.value = '';
422         if (passwordInput) passwordInput.value = '';
423     }
424 }
425
426 // Функция закрытия модального окна
427 function closeForgotPasswordModal() {
428     if (forgotPasswordModal) {
429         forgotPasswordModal.style.display = 'none';
430     }
431 }
432
433 // Клик по кнопке "Забыли пароль?"
434 const forgotPasswordBtn = document.getElementById('
435     forgotPasswordBtn');
436 if (forgotPasswordBtn) {
437     forgotPasswordBtn.addEventListener('click', (e) => {
438         e.preventDefault();
439         openForgotPasswordModal();
440     });
441 }
442
443 // Закрытие по кнопке ×
444 if (forgotPasswordCloseBtn) {
445     forgotPasswordCloseBtn.addEventListener('click',
446         closeForgotPasswordModal);
447 }
448
449 // Закрытие по клику вне модального окна
450 if (forgotPasswordModal) {
451     forgotPasswordModal.addEventListener('click', (e) => {
452         if (e.target === forgotPasswordModal) {
453             closeForgotPasswordModal();
454         }
455     });
456 }
457
458 // Отправка кода на email
459 if (forgotPasswordSendCodeBtn) {
460     forgotPasswordSendCodeBtn.addEventListener('click', async ()
=> {
461         const emailInput = document.getElementById('forgotEmail')
462         ;
463         const email = emailInput ? emailInput.value.trim() : '';

```

```

461
462     if (!email) {
463         alert('❑ Введите email');
464         return;
465     }
466
467     // Блокируем кнопку на время запроса
468     forgotPasswordSendCodeBtn.disabled = true;
469     forgotPasswordSendCodeBtn.textContent = 'Отправка...';
470
471     try {
472         const requestResult = await requestPasswordReset(
473             email);
474
475         if (!requestResult.ok) {
476             alert❑(` ${extractApiErrorMessage(requestResult.
477                 data, 'Не удалось отправить код.')}`);
478             return;
479         }
480
481         alert('❑ Код отправлен на вашу почту');
482         // Фокусируемся на поле ввода кода
483         const codeInput = document.getElementById('forgotCode
484             ');
485         if (codeInput) codeInput.focus();
486     } catch (error) {
487         alert('❑ Ошибка при отправке кода');
488     } finally {
489         forgotPasswordSendCodeBtn.disabled = false;
490         forgotPasswordSendCodeBtn.textContent = 'Отправить
491             код';
492     }
493 }));
494 }
495
496 // Подтверждение смены пароля
497 if (forgotPasswordForm) {
498     forgotPasswordForm.addEventListener('submit', async (e) => {
499         e.preventDefault();
500
501         const emailInput = document.getElementById('forgotEmail')
502             ;
503         const codeInput = document.getElementById('forgotCode');
504         const passwordInput = document.getElementById('
505             forgotNewPassword');
506
507         const email = emailInput ? emailInput.value.trim() : '';
508         const code = codeInput ? codeInput.value.trim() : '';
509         const newPassword = passwordInput ? passwordInput.value :
510             '';
511
512         if (!email || !code || !newPassword) {
513             alert('❑ Заполните все поля');
514             return;
515         }

```

```

508     }
509
510     if (newPassword.length < 6) {
511         alert('❑ Пароль должен быть не короче 6 символов');
512         return;
513     }
514
515     // Блокируем кнопку на время запроса
516     if (forgotPasswordSubmitBtn) {
517         forgotPasswordSubmitBtn.disabled = true;
518         forgotPasswordSubmitBtn.textContent = 'Подтверждение
519         ...';
520     }
521
522     try {
523         const confirmResult = await confirmPasswordReset(
524             email, code, newPassword);
525
526         if (!confirmResult.ok) {
527             alert(❑ ` ${extractApiErrorMessage(confirmResult.
528                 data, 'Не удалось изменить пароль.')} `);
529             return;
530         }
531
532         clearTokens();
533         updateNavBar();
534         alert('❑ Пароль изменён. Войдите заново с новым
535             паролем. ');
536         closeForgotPasswordModal();
537         showLoginPage();
538     } catch (error) {
539         alert('❑ Ошибка при смене пароля');
540     } finally {
541         if (forgotPasswordSubmitBtn) {
542             forgotPasswordSubmitBtn.disabled = false;
543             forgotPasswordSubmitBtn.textContent = '
544                 Подтвердить и сменить пароль';
545         }
546     }
547 }
548
549 // Обработчики для входа и переключения
550 document.getElementById('loginForm').addEventListener('submit',
551     async (e) => {
552         e.preventDefault();
553         const success = await login(
554             document.getElementById('loginUsername').value,
555             document.getElementById('loginPassword').value
556         );
557         if (success) {
558             showMainPage();
559             loadProducts(1);
560         }
561     }

```

```

556     });
557
558     document.getElementById( 'switchToRegister' )?.addEventListener( '
        click', (e) => {
559         e.preventDefault();
560         showRegisterPage();
561     });
562
563     document.getElementById( 'backToMainFromLogin' )?.addEventListener(
        'click', () => showMainPage());
564 }

```

bookings.js

```

1  function calculateRentalDays(startDate, endDate) {
2      if (!startDate || !endDate) return 0;
3      const start = new Date(startDate);
4      const end = new Date(endDate);
5      const diff = Math.round((end - start) / (1000 * 60 * 60 * 24));
6      return diff > 0 ? diff : 0;
7  }
8
9
10
11 function formatBookingStatus(status) {
12     const map = {
13         pending: 'Ожидает подтверждения',
14         confirmed: 'Подтверждена',
15         cancelled: 'Отменена',
16         completed: 'Завершена'
17     };
18     return map[status] || 'Не создана';
19 }
20
21
22
23 function formatPaymentStatus(status) {
24     const map = {
25         unpaid: 'Не оплачено',
26         pending: 'Ожидает оплаты',
27         paid: 'Оплачено',
28         failed: 'Ошибка оплаты'
29     };
30     return map[status] || '-';
31 }
32
33
34
35 function getMyBookingsForItem(itemId) {
36     try {
37         const list = await fetchPaginatedResults('/bookings/?role=
            outgoing');
38         return list.filter(booking =>
39             (
40                 String(booking.item) === String(itemId) ||

```

```

41         String(booking.item?.id) === String(itemId)
42     ) &&
43     ['pending', 'confirmed'].includes(booking.status)
44 );
45 } catch (error) {
46     return [];
47 }
48 }
49
50 async
51
52 function getIncomingBookings() {
53     try {
54         const response = await apiRequest('/bookings/?role=incoming')
55         ;
56         if (!response.ok) return [];
57         const data = await response.json();
58         return Array.isArray(data) ? data : (data.results || []);
59     } catch (error) {
60         return [];
61     }
62 }
63
64 async
65
66 function getOutgoingBookings() {
67     try {
68         return await fetchPaginatedResults('/bookings/?role=outgoing '
69         );
70     } catch (error) {
71         return [];
72     }
73 }
74
75 async
76
77 function getAllIncomingBookings() {
78     try {
79         return await fetchPaginatedResults('/bookings/?role=incoming '
80         );
81     } catch (error) {
82         return [];
83     }
84 }
85
86 async
87
88 function confirmIncomingBooking(bookingId) {
89     try {
90         const response = await apiRequest(`/bookings/${bookingId}/
91         confirm/`, {
92             method: 'POST',
93             body: JSON.stringify({})
94         });

```

```

91         const data = await response.json().catch(() => ({}));
92         return { success: response.ok, data };
93     } catch (error) {
94         return { success: false, data: { detail: 'Ошибка подключения
95             при подтверждении заявки.' } };
96     }
97 }
98 async
99
100 function cancelIncomingBooking(bookingId) {
101     try {
102         const response = await apiRequest(`/bookings/${bookingId}/
103             cancel/`, {
104                 method: 'POST',
105                 body: JSON.stringify({})
106             });
107         const data = await response.json().catch(() => ({}));
108         return { success: response.ok, data };
109     } catch (error) {
110         return { success: false, data: { detail: 'Ошибка подключения
111             при отмене заявки.' } };
112     }
113 }
114
115 function destroyRentalPickers() {
116     if (rentStartPicker) {
117         rentStartPicker.destroy();
118         rentStartPicker = null;
119     }
120     if (rentEndPicker) {
121         rentEndPicker.destroy();
122         rentEndPicker = null;
123     }
124 }
125
126 function hasSignedCurrentBookingContract() {
127     return !(
128         currentBookingContract &&
129         currentBookingContract.my_role === 'renter' &&
130         currentBookingContract.my_signed
131     );
132 }
133
134
135
136 function isCurrentBookingContractFullySigned() {
137     return !(
138         currentBookingContract &&
139         currentBookingContract.is_signed &&
140         currentBookingContract.renter_signed_at &&
141         currentBookingContract.owner_signed_at

```

```

142         );
143     }
144
145
146
147     function getSelectedRentalDates() {
148         return {
149             startDate: document.getElementById('rentStartDate')?.value ||
150                 '',
151             endDate: document.getElementById('rentEndDate')?.value || '',
152         };
153     }
154
155
156     function findBookingForRentalDates(startDate, endDate) {
157         if (!startDate || !endDate) return null;
158         return currentItemBookings.find(booking =>
159             booking.start_date === startDate &&
160             booking.end_date === endDate &&
161             ['pending', 'confirmed'].includes(booking.status)
162         ) || null;
163     }
164
165
166
167     function syncBookingForSelectedDates({ resetUi = false } = {}) {
168         const previousBookingId = currentBooking?.id || null;
169         const { startDate, endDate } = getSelectedRentalDates();
170
171         currentBooking = findBookingForRentalDates(startDate, endDate);
172
173         if (!currentBooking) {
174             currentBookingContract = null;
175             if (resetUi) {
176                 resetPaymentBox();
177                 setRentModalMessage('', '');
178             }
179             updateRentalActionButtons();
180             updateRentalSummary();
181             return null;
182         }
183
184         if (resetUi && previousBookingId !== currentBooking.id) {
185             resetPaymentBox();
186         }
187
188         await syncCurrentBookingContract();
189         applyCurrentBookingRentalMessage();
190         updateRentalSummary();
191         return currentBooking;
192     }
193
194

```

```

195
196 function updateRentalActionButtons() {
197     const createBookingBtn = document.getElementById( '
198         createBookingBtn' );
199     const openContractBtn = document.getElementById( 'openContractBtn '
200     );
201     const startPaymentBtn = document.getElementById( 'startPaymentBtn '
202     );
203     const booking = currentBooking;
204     const canOpenContract = [ 'confirmed', 'completed' ].includes(
205         booking?.status );
206     const canProceedToPayment = !!booking && booking.status === '
207         confirmed' && (
208         booking.payment_status === 'pending' ||
209         booking.payment_status === 'paid' ||
210         isCurrentBookingContractFullySigned()
211     );
212
213     if (createBookingBtn) {
214         createBookingBtn.disabled = !!booking && [ 'pending', '
215             confirmed' ].includes(booking.status);
216     }
217     if (openContractBtn) {
218         openContractBtn.disabled = !canOpenContract;
219     }
220     if (startPaymentBtn) {
221         startPaymentBtn.classList.toggle( 'is -hidden', !
222             canProceedToPayment );
223         startPaymentBtn.disabled = !canProceedToPayment || booking?.
224             payment_status === 'paid';
225         startPaymentBtn.textContent = booking?.payment_status === '
226             paid '
227             ? 'Оплата уже выполнена'
228             : 'Перейти к оплате по СБП';
229     }
230 }
231
232 async
233
234 function syncCurrentBookingContract() {
235     if (!currentBooking || !canUseContractForBooking(currentBooking))
236     {
237         currentBookingContract = null;
238         updateRentalActionButtons();
239         return null;
240     }
241
242     const result = await getContractByBooking(currentBooking.id);
243     currentBookingContract = result.success ? result.data : null;
244     updateRentalActionButtons();
245     return currentBookingContract;
246 }
247
248
249

```

```

239
240 function resetPaymentBox() {
241     currentPaymentPayload = null;
242     const box = document.getElementById('paymentBox');
243     if (!box) return;
244     box.classList.remove('active');
245     document.getElementById('paymentBank').textContent = '—';
246     document.getElementById('paymentRecipient').textContent = '—';
247     document.getElementById('paymentPhone').textContent = '—';
248     document.getElementById('paymentAmount').textContent = '—';
249     document.getElementById('paymentStatus').textContent = '—';
250     document.getElementById('paymentExpires').textContent = '—';
251     document.getElementById('paymentQrPayload').textContent = '';
252     const paymentLinkBtn = document.getElementById('paymentLinkBtn');
253     if (paymentLinkBtn) {
254         paymentLinkBtn.disabled = true;
255     }
256 }
257
258 function confirmBookingPayment() {
259     const booking = await refreshCurrentBookingState();
260     const demoMessageBox = document.getElementById('sbpDemoMessage');
261     const demoPayButton = document.getElementById('sbpDemoPayBtn');
262
263     if (!booking) {
264         setRentModalMessage('error', 'Бронь не найдена. ');
265         if (demoMessageBox) {
266             demoMessageBox.innerHTML = '<div class="rent-error">Бронь
                не найдена. Обновите окно аренды и попробуйте снова
                .</div>';
267         }
268         if (demoPayButton) {
269             demoPayButton.disabled = false;
270             demoPayButton.textContent = 'Оплатить';
271         }
272         return;
273     }
274
275     try {
276         const response = await apiRequest(`/bookings/${booking.id}/
            confirm_payment/`, {
277             method: 'POST',
278             body: JSON.stringify({})
279         });
280
281         if (!response.ok) {
282             const message = await readApiError(response, 'Не удалось
                подтвердить оплату. ');
283             setRentModalMessage('error', message);
284             if (demoMessageBox) {
285                 demoMessageBox.innerHTML = `<div class="rent-error">$
                    {escapeHtml(message)}</div>`;
286             }
287             if (demoPayButton) {

```

```

288         demoPayButton.disabled = false;
289         demoPayButton.textContent = 'Оплатить';
290     }
291     return;
292 }
293
294 const data = await response.json().catch(() => ({}));
295 currentBooking = data;
296 updateRentalSummary();
297 resetPaymentBox();
298 closeSbpDemoModal();
299 setRentModalMessage('success', 'Оплата подтверждена. Бронь
    оплачена. ');
300 setTimeout(() => {
301     closeRentalModal();
302     loadProducts(currentPage);
303 }, 1200);
304 } catch (error) {
305     setRentModalMessage('error', 'Ошибка подключения при
        подтверждении оплаты. ');
306     if (demoMessageBox) {
307         demoMessageBox.innerHTML = '<div class="rent-error">
            Ошибка подключения при подтверждении оплаты.</div>';
308     }
309     if (demoPayButton) {
310         demoPayButton.disabled = false;
311         demoPayButton.textContent = 'Оплатить';
312     }
313 }
314 }
315
316 async
317
318 function confirmDepositPayment() {
319     const booking = await refreshCurrentBookingState();
320     const demoMessageBox = document.getElementById('sbpDemoMessage');
321     const demoPayButton = document.getElementById('sbpDemoPayBtn');
322
323     if (!booking) {
324         setRentModalMessage('error', 'Бронь не найдена. ');
325         if (demoPayButton) { demoPayButton.disabled = false;
            demoPayButton.textContent = 'Оплатить'; }
326         return;
327     }
328
329     try {
330         const response = await apiRequest(`/bookings/${booking.id}/
            confirm_deposit/`, {
331             method: 'POST',
332             body: JSON.stringify({})
333         });
334
335         if (!response.ok) {

```

```

336         const message = await readApiError(response, 'Не удалось
337             подтвердить оплату залога. ');
338         setRentModalMessage('error', message);
339         if (demoMessageBox) demoMessageBox.innerHTML = `<div
340             class="rent-error">${escapeHtml(message)}</div>`;
341         if (demoPayButton) { demoPayButton.disabled = false;
342             demoPayButton.textContent = 'Оплатить'; }
343         return;
344     }
345
346     const data = await response.json().catch(() => ({}));
347     currentBooking = data;
348     currentPaymentType = 'rent';
349     updateRentalSummary();
350     updateRentalActionButtons();
351     resetPaymentBox();
352     closeSbpDemoModal();
353     applyCurrentBookingRentalMessage();
354 } catch {
355     setRentModalMessage('error', 'Ошибка подключения при
356         подтверждении залога. ');
357     if (demoPayButton) { demoPayButton.disabled = false;
358         demoPayButton.textContent = 'Оплатить'; }
359 }
360
361 }
362
363 async
364
365 function showIncomingBookingsPage(flashMessage = '') {
366     if (!isAuthenticated()) { alert('Пожалуйста, войдите в систему');
367         showLoginPage(); return; }
368
369     const [incomingBookings, outgoingBookings] = await Promise.all([
370         getIncomingBookings(),
371         getOutgoingBookings()
372     ]);
373
374     document.getElementById('app').innerHTML = `
375         <div class="bookings-page"><div class="bookings-card">
376             <div class="back-button" id="backToMainFromBookings">
377                 Назад</div>
378             <div class="bookings-title"> Заявки и мои аренды</div>
379             <div class="bookings-sub">Здесь владелец видит входящие
380                 бронирования, а арендатор может открыть завершённую
381                 аренду и оставить отзыв.</div>
382             <div id="bookingsMessage">${flashMessage}</div>
383             <div class="section-title">Входящие заявки</div>
384             ${incomingBookings.length === 0 ? '<div class="empty-
385                 message">Пока нет входящих заявок на ваши объявления
386                 .</div>' : `
387                 <div class="bookings-list">
388                     ${incomingBookings.map(booking => `
389                         <div class="booking-request" data-booking-id=
390                             "${booking.id}">

```

```

378         <div class="booking-request-head">
379             <div>
380                 <div class="booking-request-title"
381                     ">${escapeHtml(booking.
382                         item_title || `Объявление #${
383                             booking.item}`)}</div>
384                 <div class="booking-request-meta"
385                     >
386                     <div>Арендатор: ${escapeHtml(
387                         booking.renter_username ||
388                         'Пользователь')}</div>
389                     <div>Email: ${escapeHtml(
390                         booking.renter_email || '
391                         -')}</div>
392                     <div>Период: ${escapeHtml(
393                         booking.start_date || '-'
394                         )} — ${escapeHtml(booking
395                             .end_date || '-')}</div>
396                     <div>Сумма: ${formatPrice(
397                         booking.total_price || 0)
398                     }</div>
399                 </div>
400             </div>
401             <div class="booking-request-status">${
402                 {formatBookingStatus(booking.
403                     status)}}</div>
404         </div>
405         ${renderOwnerBookingActions(booking)}
406     </div>
407     `).join('')}
408 </div>
409 `}
410
411 <div class="section-title">Мои аренды</div>
412 ${outgoingBookings.length === 0 ? `<div class="empty-
413     message">У вас пока нет аренд.</div>` : `
414     <div class="bookings-list">
415         ${outgoingBookings.map(booking => `
416             <div class="booking-request" data-outgoing-
417                 booking-id="${booking.id}">
418                 <div class="booking-request-head">
419                     <div>
420                         <div class="booking-request-title"
421                             ">${escapeHtml(
422                                 getBookingItemTitle(booking))
423                             }</div>
424                         <div class="booking-request-meta"
425                             >
426                             <div>Период: ${escapeHtml(
427                                 booking.start_date || '-'
428                                 )} — ${escapeHtml(booking
429                                     .end_date || '-')}</div>

```

```

406         <div>Сумма: ${formatPrice(
              booking.total_price || 0)
            }</div>
407         <div>Оплата: ${
              formatPaymentStatus(
                booking.payment_status ||
                'unpaid')}</div>
408       </div>
409       ${getReviewSummaryMarkup(booking)
        }
410     </div>
411     <div class="booking-request-status">${
        {formatBookingStatus(booking.
        status)}}</div>
412   </div>
413   ${renderRenterBookingActions(booking)}
414 </div>
415 `).join('')
416 </div>
417 `}
418 </div></div>
419 `;
420
421 document.getElementById('backToMainFromBookings')?.
    addEventListener('click', () => showMainPage());
422 attachHandoverListeners(incomingBookings, outgoingBookings);
423 }
424
425
426
427 function getBookingSortDate(booking) {
428   return new Date(
429     booking?.start_date ||
430     booking?.created_at ||
431     '1970-01-01'
432   ).getTime();
433 }
434
435
436
437 function getBookingItemTitle(booking) {
438   return booking?.item_title || booking?.item?.title || booking?.
    item?.name || `Объявление #${booking?.item || —}`;
439 }
440
441
442
443 function canLeaveReviewForBooking(booking) {
444   if (booking?.can_leave_review !== undefined) return !!booking.
    can_leave_review;
445   return !!booking && !booking.has_review && (
446     booking.status === 'completed' ||
447     (
448       booking.status === 'confirmed' &&

```

```

449         booking.payment_status === 'paid'
450     )
451 );
452 }
453
454
455
456 function canUseContractForBooking(booking) {
457     return ['confirmed', 'completed'].includes(booking?.status);
458 }
459
460 // ————— Renders action buttons for the OWNER (арендодатель)
461
462
463 function renderOwnerBookingActions(booking) {
464     const parts = [];
465
466     if (booking.status === 'pending') {
467         parts.push(`
468             <div class="booking-request-actions">
469                 <button class="btn-primary" data-action="confirm-
470                     booking" data-id="${booking.id}">Подтвердить</
471                     button>
472                 <button class="btn-outline" data-action="cancel-
473                     booking" data-id="${booking.id}">Отклонить</button
474                 >
475             </div>`);
476     }
477
478     const actionBtns = [];
479     if (canUseContractForBooking(booking)) {
480         actionBtns.push(`<button class="btn-outline" data-action="
481             open-contract" data-id="${booking.id}" data-scope="
482             incoming">Договор / ЭЦП</button>`);
483         actionBtns.push(`<button class="btn-outline" data-action="
484             download-contract-pdf" data-id="${booking.id}">Скачать PDF
485             </button>`);
486     }
487     if (booking.status === 'confirmed' && booking.payment_status ===
488         'paid'
489         && booking.renter_pickup_confirmed && !booking.
490             owner_return_confirmed) {
491         actionBtns.push(`<button class="btn-primary" data-action="
492             owner-confirm-return" data-id="${booking.id}">
493             Подтвердить возврат товара</button>`);
494     }
495     if (booking.return_status === 'requested') {
496         actionBtns.push(`<button class="btn-primary" data-action="
497             approve-refund" data-id="${booking.id}">
498             Одобрить СБП-
499             возврат</button>`);
500         actionBtns.push(`<button class="btn-danger" data-action="
501             reject-refund" data-id="${booking.id}">Отклонить возврат</
502             button>`);
503     }
504 }

```

```

486     }
487
488     const badges = getHandoverBadges(booking);
489     const actionsHtml = actionBtns.length
490       ? `<div class="booking-request-actions">${actionBtns.join('')}
491         </div>`
492       : '';
493
494     return badges + parts.join('') + actionsHtml;
495   }
496   // ————— Renders action buttons for the RENTER (арендатор)
497
498   function renderRenterBookingActions(booking) {
499     const btns = [];
500
501     btns.push(`<button class="btn-outline" data-action="open-rental-item" data-id="${booking.id}">Открыть объявление</button>`);
502
503     if (canUseContractForBooking(booking)) {
504       btns.push(`<button class="btn-outline" data-action="open-contract" data-id="${booking.id}" data-scope="outgoing">
505         Договор / ЭЦП</button>`);
506       btns.push(`<button class="btn-outline" data-action="download-contract-pdf" data-id="${booking.id}">Скачать PDF</button>`);
507     }
508     if (booking.status === 'confirmed' && booking.payment_status === 'paid'
509       && !booking.renter_pickup_confirmed) {
510       btns.push(`<button class="btn-primary" data-action="renter-confirm-pickup" data-id="${booking.id}">Подтвердить
511         получение товара</button>`);
512     }
513     if (booking.status === 'confirmed' && booking.payment_status === 'paid'
514       && !booking.return_requested) {
515       btns.push(`<button class="btn-warning" data-action="renter-request-refund" data-id="${booking.id}">Запросить
516         возврат средств</button>`);
517     }
518     if ([ 'pending', 'confirmed' ].includes(booking.status) && !booking
519       .return_requested) {
520       btns.push(`<button class="btn-danger" data-action="renter-cancel-booking" data-id="${booking.id}">Отказаться от
521         аренды</button>`);
522     }
523     if (canLeaveReviewForBooking(booking)) {
524       btns.push(`<button class="btn-primary" data-action="leave-review" data-id="${booking.id}">Оставить отзыв</button>`);
525     }
526   }

```

```

523     const badges = getHandoverBadges(booking);
524     return badges + `<div class="booking-request-actions">${btns.join
      (' ')}</div>`;
525   }
526
527   // — Attach all handover/refund/cancel event listeners
528   // Call after any page render that shows booking cards.
529
530
531   function attachHandoverListeners(incomingBookings, outgoingBookings) {
532     const allBookings = [...(incomingBookings || []), ...(
      outgoingBookings || [])];
533
534     const findBooking = (id) => allBookings.find(b => String(b.id)
      === String(id));
535     const reload = (msg) => {
536       const page = document.querySelector('.bookings-page, .my-
        items-page');
537       if (page && page.classList.contains('my-items-page')) {
538         showMyItemsPage().then(() => {
539           if (msg) {
540             const msgEl = document.getElementById('
              myItemsMessage');
541             if (msgEl) msgEl.innerHTML = `<div class="success
              -message">${msg}</div>`;
542           }
543         });
544       } else {
545         showIncomingBookingsPage(msg ? `<div class="success-
          message">${msg}</div>` : '');
546       }
547     };
548
549     // confirm-booking
550     document.querySelectorAll('[data-action="confirm-booking"]').
      forEach(btn => {
551       btn.addEventListener('click', async () => {
552         const result = await confirmIncomingBooking(btn.dataset.
          id);
553         if (result.success) {
554           reload('☐ Заявка подтверждена. ');
555         } else {
556           alert('☐ ' + extractApiErrorMessage(result.data, 'He
            удалось подтвердить заявку. '));
557         }
558       });
559     });
560
561     // cancel-booking (owner rejects)
562     document.querySelectorAll('[data-action="cancel-booking"]').
      forEach(btn => {
563       btn.addEventListener('click', async () => {

```

```

564         const result = await cancelIncomingBooking(btn.dataset.id
565             );
566         if (result.success) {
567             reload('❑ Заявка отклонена. ');
568         } else {
569             alert('❑ ' + extractApiErrorMessage(result.data, 'Не
570                 удалось отклонить заявку. '));
571         }
572     });
573
574     // open-contract (incoming or outgoing scope)
575     document.querySelectorAll('[data-action="open-contract"]').
576         forEach(btn => {
577         btn.addEventListener('click', async () => {
578             const booking = findBooking(btn.dataset.id);
579             if (booking) await openContractFlowForBooking(booking);
580         });
581     });
582
583     // download-contract-pdf
584     document.querySelectorAll('[data-action="download-contract-pdf"]'
585         ).forEach(btn => {
586         btn.addEventListener('click', async () => {
587             await downloadContractPdfByBookingId(btn.dataset.id);
588         });
589     });
590
591     // open-rental-item / leave-review
592     document.querySelectorAll('[data-action="open-rental-item"], [
593         data-action="leave-review"]').forEach(btn => {
594         btn.addEventListener('click', async () => {
595             const booking = findBooking(btn.dataset.id);
596             if (booking) await openReviewFlowForBooking(booking);
597         });
598     });
599
600     // ❑ Арендатор подтверждает получение
601     document.querySelectorAll('[data-action="renter-confirm-pickup"]'
602         ).forEach(btn => {
603         btn.addEventListener('click', () => {
604             const bookingId = btn.dataset.id;
605             openHandoverModal({
606                 title: '❑ Подтверждение получения товара',
607                 sub: 'Прикрепите фото товара в момент получения. Это
608                     зафиксирует состояние вещи в начале аренды.',
609                 needPhoto: true,
610                 confirmLabel: '❑ Подтвердить получение',
611                 onConfirm: async ({ photo }) => {
612                     if (!photo) { alert('Пожалуйста, прикрепите фото
613                         товара'); return; }
614                     const resp = await confirmRenterPickup(bookingId,
615                         photo);
616                     closeHandoverModal();
617                 }
618             });
619         });
620     });

```

```

609         if (resp.ok) {
610             reload('☐ Получение товара подтверждено!');
611         } else {
612             const e = await resp.json().catch(() => ({}))
613             ;
614             alert('☐ ' + extractApiErrorMessage(e, 'Не
615                 удалось подтвердить получение.'));
616         }
617     });
618 });
619
620 // ☐ Арендодатель подтверждает возврат
621 document.querySelectorAll('[data-action="owner-confirm-return"]')
622     .forEach(btn => {
623         btn.addEventListener('click', () => {
624             const bookingId = btn.dataset.id;
625             openHandoverModal({
626                 title: '☐ Подтверждение возврата товара',
627                 sub: 'Прикрепите фото товара при возврате. После
628                     подтверждения бронирование будет завершено.',
629                 needPhoto: true,
630                 confirmLabel: '☐ Подтвердить возврат',
631                 onConfirm: async ({ photo }) => {
632                     if (!photo) { alert('Пожалуйста, прикрепите фото
633                         товара'); return; }
634                     const resp = await confirmOwnerReturn(bookingId,
635                         photo);
636                     closeHandoverModal();
637                     if (resp.ok) {
638                         reload('☐ Возврат подтверждён. Бронирование
639                             завершено.');
```

```

651         needPhoto: false,
652         needReason: true,
653         reasonPlaceholder: 'Причина возврата (обязательно)...',
654         confirmLabel: '☐ Запросить возврат',
655         confirmClass: 'btn-warning',
656         onConfirm: async ({ reason }) => {
657             if (!reason) { alert('Укажите причину возврата');
658                 return; }
659             const resp = await requestRefund(bookingId,
660                 reason);
661             closeHandoverModal();
662             if (resp.ok) {
663                 reload('☐ Запрос на возврат отправлен
664                     арендодателю.');
```

```

665             } else {
666                 const e = await resp.json().catch(() => ({}));
667                 alert('☐ ' + extractApiErrorMessage(e, 'Не
668                     удалось отправить запрос.');
```

```

669             }
670         });
671     });
672     // ☐ Арендодатель одобряет СБП-возврат
673     document.querySelectorAll('[data-action="approve-refund"]').
674     forEach(btn => {
675         btn.addEventListener('click', async () => {
676             if (!confirm('Одобрить СБП-возврат средств арендатору?'))
677                 return;
678             const resp = await processRefund(btn.dataset.id, true);
679             if (resp.ok) {
680                 reload('☐ СБП-возврат выполнен.');
```

```

681             } else {
682                 const e = await resp.json().catch(() => ({}));
683                 alert('☐ ' + extractApiErrorMessage(e, 'Не удалось
684                     выполнить возврат.');
```

```

694         alert('❌ ' + extractApiErrorMessage(e, 'Ошибка. '));
695     }
696 });
697 });
698
699 // ❌ Арендатор отказывается от аренды
700 document.querySelectorAll('[data-action="renter-cancel-booking"]')
701     .forEach(btn => {
702         btn.addEventListener('click', () => {
703             const bookingId = btn.dataset.id;
704             openHandoverModal({
705                 title: '❌ Отказ от аренды',
706                 sub: 'Вы собираетесь отменить аренду. Укажите причину  
— это поможет арендодателю.',
707                 needPhoto: false,
708                 needReason: true,
709                 reasonPlaceholder: 'Причина отказа...',
710                 confirmLabel: 'Отказаться от аренды',
711                 confirmClass: 'btn-danger',
712                 onConfirm: async ({ reason }) => {
713                     const resp = await renterCancel(bookingId, reason);
714                     closeHandoverModal();
715                     if (resp.ok) {
716                         reload('Аренда отменена. ');
717                     } else {
718                         const e = await resp.json().catch(() => ({}));
719                         alert('❌ ' + extractApiErrorMessage(e, 'Не  
удалось отменить аренду. '));
720                     }
721                 }
722             });
723         });
724     });
725
726 // ===== HANDOVER / RETURN / REFUND / CANCEL HELPERS =====
727
728
729
730 function getHandoverBadges(booking) {
731     let html = '';
732     if (booking.renter_pickup_confirmed) {
733         html += '<span class="handover-status-badge confirmed">  
Получение подтверждено</span> ';
734     }
735     if (booking.owner_return_confirmed) {
736         html += '<span class="handover-status-badge confirmed">  
Возврат подтверждён</span> ';
737     }
738     if (booking.return_requested && booking.return_status === 'requested') {

```

```

739         html += '<span class="handover-status-badge refund-requested'
740             '□"> СБП-возврат запрошен</span> ' ;
741     }
742     if (booking.return_status === 'completed') {
743         html += '<span class="handover-status-badge refund-done□">
744             СБП-возврат выполнен</span> ' ;
745     }
746     if (booking.return_status === 'rejected') {
747         html += '<span class="handover-status-badge" style="
748             background:#fde8e8;color:#b03020□"> Возврат отклонён</
749             span> ' ;
750     }
751     return html ? `<div style="margin-top:8px;display:flex;gap:6px;
752         flex-wrap:wrap;">${html}</div>` : '';
753 }
754
755 // --- Handover modal controller ---
756 let handoverActiveCallback = null;
757
758 function openHandoverModal({ title, sub, needPhoto = false, needReason =
759     false, reasonPlaceholder = 'Опишите причину...', confirmLabel = '
760     Подтвердить', confirmClass = 'btn-primary', onConfirm }) {
761     document.getElementById('handoverModalTitle').textContent = title
762     ;
763     document.getElementById('handoverModalSub').textContent = sub;
764     document.getElementById('handoverPhotoSection').style.display =
765         needPhoto ? '' : 'none';
766     document.getElementById('handoverReasonSection').style.display =
767         needReason ? '' : 'none';
768     document.getElementById('handoverReasonInput').placeholder =
769         reasonPlaceholder;
770     document.getElementById('handoverReasonInput').value = '';
771     document.getElementById('handoverPhotoPreview').classList.remove(
772         'visible');
773     document.getElementById('handoverPhotoName').textContent = '';
774     document.getElementById('handoverPhotoInput').value = '';
775     const confirmBtn = document.getElementById('handoverConfirmBtn');
776     confirmBtn.textContent = confirmLabel;
777     confirmBtn.className = confirmClass;
778     handoverActiveCallback = onConfirm;
779     document.getElementById('handoverOverlay').classList.add('active');
780 }
781
782 function closeHandoverModal() {
783     document.getElementById('handoverOverlay').classList.remove('
784         active');
785     handoverActiveCallback = null;
786 }

```

```

779 document.getElementById('handoverPhotoInput').addEventListener('
change', function() {
780     const file = this.files[0];
781     if (!file) return;
782     document.getElementById('handoverPhotoName').textContent = file .
        name;
783     const reader = new FileReader();
784     reader.onload = e => {
785         const preview = document.getElementById('handoverPhotoPreview
');
786         preview.src = e.target.result;
787         preview.classList.add('visible');
788     };
789     reader.readAsDataURL(file);
790 });
791
792
793 document.getElementById('handoverCancelBtn').addEventListener('click '
, closeHandoverModal);
794
795 // — Legal modals close handlers


---


796
797
798 function confirmRenterPickup(bookingId, photo) {
799     const formData = new FormData();
800     if (photo) formData.append('photo', photo);
801     return await apiRequest(`/bookings/${bookingId}/
confirm_renter_pickup/`, { method: 'POST', body: formData });
802 }
803
804 async
805
806 function requestRefund(bookingId, reason) {
807     return await apiRequest(`/bookings/${bookingId}/request_refund/`,
{ method: 'POST', body: JSON.stringify({ reason }) });
808 }
809
810 async
811
812 function processRefund(bookingId, approved) {
813     return await apiRequest(`/bookings/${bookingId}/process_refund/`,
{ method: 'POST', body: JSON.stringify({ approved }) });
814 }
815
816 async
817
818 function renterCancel(bookingId, reason) {
819     return await apiRequest(`/bookings/${bookingId}/renter_cancel/`,
{ method: 'POST', body: JSON.stringify({ reason }) });
820 }
821
822
823

```

```

824
825 function formatContractStatus(status) {
826     const map = {
827         draft: 'Черновик договора',
828         partially_signed: 'Подписан одной стороной',
829         signed: 'Подписан обеими сторонами',
830     };
831     return map[status] || 'Договор';
832 }
833
834
835
836 function renderContractSignState(title, signerName, signedAt, signatureCode)
837 {
838     if (!signedAt) {
839         return `
840             <div class="contract-sign-state">
841                 <strong>${escapeHtml(title)}</strong>
842                 <div class="contract-sign-meta">Подпись ЭЦП пока не
843                     поставлена.</div>
844             </div>
845         `;
846     }
847     return `
848         <div class="contract-sign-state">
849             <strong>${escapeHtml(title)}</strong>
850             <div class="contract-sign-meta">
851                 Подписант: ${escapeHtml(signerName || 'Не указан')}<br>
852                 Дата подписи: ${escapeHtml(formatDateRu(signedAt))}<br>
853                 Код ЭЦП: ${escapeHtml(signatureCode || '—')}
854             </div>
855         </div>
856     `;
857 }
858
859 async
860 function getContractByBooking(bookingId) {
861     try {
862         const response = await apiRequest(`/contracts/by-booking/${
863             bookingId}`);
864         if (!response.ok) {
865             const message = await readApiError(response, 'Не удалось
866                 загрузить договор. ');
867             return { success: false, data: { error: message } };
868         }
869         const data = await response.json().catch(() => ({}));
870         return { success: response.ok, data };
871     } catch (error) {
872         return { success: false, data: { error: 'Ошибка подключения
873             при загрузке договора.' } };
874     }
875 }

```

```

871     }
872 }
873
874
875
876 function downloadContractFile(contract) {
877     if (!contract?.file_url) {
878         alert('❑ PDF договора пока недоступен');
879         return;
880     }
881
882     const link = document.createElement('a');
883     link.href = contract.file_url;
884     link.download = `${contract.document_number} || 'contract'.pdf`;
885     link.rel = 'noopener';
886     document.body.appendChild(link);
887     link.click();
888     link.remove();
889 }
890
891 async
892
893 function downloadContractPdfByBookingId(bookingId) {
894     const result = await getContractByBooking(bookingId);
895     if (!result.success) {
896         alert(`❑ ${extractApiErrorMessage(result.data, 'Не удалось
897             загрузить договор.')}}`);
898         return;
899     }
900     downloadContractFile(result.data);
901 }
902
903 async
904
905 function signContractWithDemoEds(contractId, signerName, certificatePin) {
906     try {
907         const response = await apiRequest(`/contracts/${contractId}/
908             sign/`, {
909             method: 'POST',
910             body: JSON.stringify({
911                 signer_name: signerName,
912                 certificate_pin: certificatePin,
913             })
914         });
915         if (!response.ok) {
916             const message = await readApiError(response, 'Не удалось
917                 подписать договор. ');
918             return { success: false, data: { error: message } };
919         }
920         const data = await response.json().catch(() => ({}));
921         return { success: response.ok, data };
922     } catch (error) {

```

```

921         return { success: false, data: { error: 'Ошибка подключения
           при подписании договора.' } } };
922     }
923 }
924
925
926
927 function closeContractModal() {
928     currentContractData = null;
929     document.getElementById('contractModalOverlay')?.classList.remove
       ('active');
930     const container = document.getElementById('contractModalContent')
       ;
931     if (container) container.innerHTML = '';
932 }

```

items.js

```

1
2 function getItemImages(item) {
3     const images = Array.isArray(item?.images) ? item.images.map(img
       => img?.image).filter(Boolean) : [];
4     if (item?.image && !images.includes(item.image)) images.unshift(
       item.image);
5     if (!images.length) {
6         images.push('https://placeholder.co/900x700/eef2f7/2b6a7c?text=
           Нет+фото');
7     }
8     return images;
9 }
10
11
12
13 function getItemVideos(item) {
14     return [];
15 }
16
17
18
19 function isProductAvailableForFilter(product) {
20     if (availabilityFilterMode === 'all') return true;
21
22     let start = '';
23     let end = '';
24     const today = getTodayDateString();
25
26     if (availabilityFilterMode === 'today') {
27         start = today;
28         end = addDays(today, 1);
29     } else if (availabilityFilterMode === 'tomorrow') {
30         start = addDays(today, 1);
31         end = addDays(today, 2);
32     } else if (availabilityFilterMode === 'custom') {
33         if (!availabilityFilterStart) return true;
34         start = availabilityFilterStart;

```

```

35         end = availabilityFilterEnd || addDays(
36             availabilityFilterStart, 1);
37         if (end <= start) end = addDays(start, 1);
38     }
39     const bookedRanges = Array.isArray(product?.booked_ranges) ?
40         product.booked_ranges : [];
41     return !bookedRanges.some(range => rangeOverlaps(start, end,
42         range.start_date, range.end_date));
43 }
44
45 function renderItemPreview(item, reviewBooking = null) {
46     const images = getItemImages(item);
47     const bookedRanges = Array.isArray(item?.booked_ranges) ? item.
48         booked_ranges : [];
49     const freeLabels = getFreeDateLabels(bookedRanges);
50     const reviews = Array.isArray(item?.item_reviews) ? item.
51         item_reviews : [];
52     const currentUserId = String(localStorage.getItem('user_id') || '
53         ');
54     const isOwner = currentUserId && String(item?.owner || '') ===
55         currentUserId;
56     const actionButtons = !isOwner ? `
57         <div class="item-preview-actions">
58             <button class="btn-primary" id="itemPreviewRentBtn" type=
59                 "button">Арендовать</button>
60             <button class="btn-outline" id="itemPreviewChatBtn" type=
61                 "button">Написать продавцу</button>
62         </div>
63     ` : '';
64     return `
65         <div class="item-preview-layout">
66             <div>
67                 
70                 <div class="item-gallery-thumbs">
71                     ${images.map(src => ``).join('')}
74                 </div>
75             </div>
76             <div class="item-preview-panel">
77                 <div class="item-preview-price">${formatPrice(
78                     getDisplayPrice(item))} / день</div>
79                 <div style="font-size:0.85rem;color:#5a8090;margin-
80                     top:4px;">${Number(item.deposit) > 0 ? `Залог: ${
81                         formatPrice(Number(item.deposit))}` : 'Без залога'
82                     }</div>
83                 ${actionButtons}
84             <div class="item-preview-meta">

```

```

72         <div><strong>Владелец:</strong> ${escapeHtml(item
73             .owner_username || 'Не указан')}</div>
74         <div><strong>Рейтинг продавца:</strong> ${Number(
75             item.owner_rating || 0).toFixed(1)} / 5 (${
76             item.owner_reviews_count || 0} отзывов)</div>
77         <div><strong>Описание:</strong> ${escapeHtml(item
78             .description || 'Описание пока не добавлено.')}
79         </div>
80     </div>
81
82     <div class="preview-section-title">Свободные даты</
83     div>
84     <div class="date-pill-list">
85         ${freeLabels.map(label => `<span class="date-pill
86             ">${escapeHtml(label)}</span>`).join('')}
87     </div>
88
89     <div class="preview-section-title">Занятые периоды</
90     div>
91     <div class="date-pill-list">
92         ${bookedRanges.length
93             ? bookedRanges.map(range => `<span class="
94                 date-pill booked">${escapeHtml(
95                     formatDateRu(range.start_date))} - ${
96                     escapeHtml(formatDateRu(range.end_date))
97                 }</span>`).join('')
98             : '<span class="date-pill">Пока нет занятых
99                 дат</span>'}
100     </div>
101 </div>
102 </div>
103
104 <div class="preview-section-title">Отзывы</div>
105 ${reviews.length
    ? `<div class="review-list">
        ${reviews.map(review => `
            <div class="review-card">
                <div class="review-head">
                    <span class="review-author">${escapeHtml(
                        review.author_username || '
                        Пользователь')}</span>
                    <span>${'□'.repeat(Number(review.rating
                        || 0))}${'□'.repeat(Math.max(0, 5 -
                        Number(review.rating || 0)))}</span>
                </div>
                <div class="review-head">
                    <span>${formatDateRu(review.created_at)}
                    </span>
                </div>
                <div>${escapeHtml(review.comment || '
                    Пользователь поставил оценку без текста.')}
                </div>
            </div>
        `).join('')}
    `
    : ''}

```

```

106         </div>`
107         : `<div class="empty-message">У этого объявления пока нет
           отзывов.</div>`
108
109     ${reviewBooking ? `
110         <div class="review-form">
111             <button class="btn-primary" id="toggleReviewFormBtn"
               type="button">Оставить отзыв</button>
112             <div id="reviewFormFields" style="display:none;
               margin-top:14px;">
113                 <div class="preview-section-title" style="margin-
                   top:0;">Ваш отзыв</div>
114                 <div class="item-preview-subtitle">Оплата прошла
                   успешно. Поделитесь впечатлением об аренде и
                   объявлении.</div>
115                 <select id="reviewRatingInput">
116                     <option value="5">5 звезд</option>
117                     <option value="4">4 звезды</option>
118                     <option value="3">3 звезды</option>
119                     <option value="2">2 звезды</option>
120                     <option value="1">1 звезда</option>
121                 </select>
122                 <textarea id="reviewCommentInput" rows="4"
                   placeholder="Напишите короткий отзыв"></
                   textarea>
123                 <button class="btn-primary" id="submitReviewBtn"
                   type="button" data-booking-id="${reviewBooking
                   .id}">Отправить отзыв</button>
124                 <div id="reviewFormMessage" style="margin-top:12
                   px;"></div>
125             </div>
126         </div>
127     ` : ``}
128     `;
129 }
130
131 async
132
133 function openItemPreview(item) {
134     activePreviewItemId = item?.id || null;
135     let reviewBooking = null;
136     if (isAuthenticated()) {
137         const outgoingBookings = await getOutgoingBookings();
138         reviewBooking = outgoingBookings.find(booking =>
139             (String(booking.item) === String(item.id) || String(
140                 booking.item?.id) === String(item.id)) &&
141                 canLeaveReviewForBooking(booking)
142             ) || null;
143     }
144     document.getElementById('itemPreviewTitle').textContent = item.
        title || item.name || 'Объявление';

```

```

145 document.getElementById('itemPreviewSubtitle').textContent = `
    Подробная информация о товаре от ${item.owner_username || '
    владельца'}`;
146 document.getElementById('itemPreviewContent').innerHTML =
    renderItemPreview(item, reviewBooking);
147 document.getElementById('itemPreviewOverlay').classList.add('
    active');
148
149 document.querySelectorAll('[data-preview-image]').forEach(img =>
    {
150     img.addEventListener('click', () => {
151         const main = document.getElementById('
            itemPreviewMainImage');
152         if (main) main.src = img.dataset.previewImage;
153     });
154 });
155
156 document.getElementById('itemPreviewChatBtn')?.addEventListener('
    click', async () => {
157     if (!isAuthenticated()) {
158         alert('Пожалуйста, войдите в систему');
159         showLoginPage();
160         return;
161     }
162
163     closeItemPreview();
164     await startChat(item.owner, item.id);
165 });
166
167 document.getElementById('itemPreviewRentBtn')?.addEventListener('
    click', async () => {
168     if (!isAuthenticated()) {
169         alert('Пожалуйста, войдите в систему');
170         showLoginPage();
171         return;
172     }
173
174     closeItemPreview();
175     await openRentalModal(item);
176 });
177
178 document.getElementById('toggleReviewFormBtn')?.addEventListener(
    'click', () => {
179     const fields = document.getElementById('reviewFormFields');
180     const toggleBtn = document.getElementById('
        toggleReviewFormBtn');
181     if (fields) {
182         const shouldShow = fields.style.display === 'none';
183         fields.style.display = shouldShow ? 'block' : 'none';
184         if (toggleBtn) {
185             toggleBtn.textContent = shouldShow ? 'Закрыть поле
                для отзыва' : 'Оставить отзыв';
186         }
187     }

```

```

188     });
189
190     document.getElementById( 'submitReviewBtn' )?.addEventListener( '
191         click', async () => {
192         const rating = Number(document.getElementById( '
193             reviewRatingInput' )?.value || 5);
194         const comment = document.getElementById( 'reviewCommentInput' )
195             ?.value || '';
196         const messageBox = document.getElementById( 'reviewFormMessage
197             ');
198         const result = await submitReview(reviewBooking.id, rating,
199             comment);
200         if (result.success) {
201             if (messageBox) messageBox.innerHTML = '<div class="
202                 success-message"> Отзыв отправлен.</div>';
203             const refreshed = await apiRequest(`/items/${item.id}/`);
204             if (refreshed.ok) {
205                 const updatedItem = await refreshed.json();
206                 upsertCachedItem(updatedItem);
207                 setTimeout(() => openItemPreview(updatedItem), 400);
208             }
209             } else if (messageBox) {
210                 const errorMessage = extractApiErrorMessage(result.data,
211                     'Не удалось отправить отзыв. ');
212                 messageBox.innerHTML = `<div class="error -message-box">
213                     ${escapeHtml(errorMessage)}</div>`;
214             }
215         });
216     }
217
218
219
220 function closeItemPreview() {
221     activePreviewItemId = null;
222     document.getElementById( 'itemPreviewOverlay' )?.classList.remove( '
223         active' );
224     document.getElementById( 'itemPreviewContent' ).innerHTML = '';
225 }
226
227
228
229 function formatItemStatus(status) {
230     const map = {
231         pending: 'На модерации',
232         available: 'Доступно',
233         unavailable: 'Недоступно',
234         blocked: 'Заблокировано',
235         archived: 'В архиве'
236     };
237     return map[status] || 'Неизвестно';
238 }
239
240
241
242
243
244
245
246
247
248
249
250
251
252

```

```

233 function getBookedRangesForItem(itemId) {
234     try {
235         const directResponse = await apiRequest(`/items/${itemId}/
            booked_ranges/`);
236         if (directResponse.ok) {
237             const data = await directResponse.json();
238             return Array.isArray(data) ? data : (data.booked_ranges
                || []);
239         }
240     } catch (error) {}
241
242     const fallback = allProducts.find(item => String(item.id) ===
        String(itemId));
243     if (Array.isArray(fallback?.booked_ranges)) return fallback.
        booked_ranges;
244
245     return [];
246 }
247
248 async
249
250 function deleteItem(productId, options = {}) {
251     try {
252         const response = await apiRequest(`/items/${productId}/`, {
            method: 'DELETE' });
253         if (response.ok) {
254             alert(options.successMessage || '❑ Объявление удалено');
255             if (typeof options.onSuccess === 'function') {
256                 await options.onSuccess();
257             } else {
258                 loadProducts(currentPage);
259             }
260             return true;
261         } else {
262             alert(options.errorMessage || '❑ Ошибка при удалении');
263         }
264     } catch (error) {
265         alert(options.networkErrorMessage || '❑ Ошибка подключения')
266         ;
267     }
268     return false;
269 }
270
271 function uploadItemMedia(itemId, mediaFiles, token) {
272     const mediaUploadErrors = [];
273
274     if (!mediaFiles || !mediaFiles.length) return mediaUploadErrors;
275
276     for (const mediaFile of mediaFiles) {
277         const uploadFormData = new FormData();
278         uploadFormData.append('item', itemId);
279         uploadFormData.append('image', mediaFile);
280
281         const uploadResp = await apiRequest('/items/upload-image/', {

```

```

281         method: 'POST',
282         headers: { 'Authorization': `Bearer ${token}` },
283         body: uploadFormData
284     });
285
286     if (!uploadResp.ok) {
287         const uploadMessage = await readApiError(uploadResp, 'Не
                удалось загрузить фото. ');
288         mediaUploadErrors.push(`${mediaFile.name}: ${
                uploadMessage}`);
289         console.error('❑ Ошибка загрузки медиа: ', uploadMessage)
                ;
290     }
291 }
292
293 return mediaUploadErrors;
294 }
295
296 async
297
298 const confirmDeleteOverlay = document.getElementById('
    confirmDeleteOverlay');
299 const openDeleteConfirm = () => confirmDeleteOverlay?.classList.
    add('active');
300 const closeDeleteConfirm = () => confirmDeleteOverlay?.classList.
    remove('active');
301
302 document.getElementById('deleteItemFromEditBtn')?.
    addEventListener('click', openDeleteConfirm);
303 document.getElementById('cancelDeleteItemBtn')?.addEventListener(
    'click', closeDeleteConfirm);
304 document.getElementById('confirmDeleteItemBtn')?.addEventListener
    ('click', async () => {
305     const deleted = await deleteItem(editItem.id, {
306         successMessage: '❑ Объявление удалено',
307         onSuccess: async () => {
308             closeDeleteConfirm();
309             await showMyItemsPage();
310         },
311         errorMessage: '❑ Ошибка при удалении объявления',
312     });
313
314     if (!deleted) {
315         closeDeleteConfirm();
316     }
317 });
318 confirmDeleteOverlay?.addEventListener('click', (event) => {
319     if (event.target?.id === 'confirmDeleteOverlay') {
320         closeDeleteConfirm();
321     }
322 });
323
324 document.getElementById('backToMyItemsFromEdit')?.
    addEventListener('click', () => showMyItemsPage());

```


Автор ВКР

(подпись, дата)

Н. А. Баранов

Руководитель ВКР

(подпись, дата)

Е. А. Петрик

Нормоконтроль

(подпись, дата)

А. А. Чаплыгин

Место для диска